Major Project Report on

Learning-Based Log Anomaly Detection, Cryptanalysis, and Design of Image Sharing and Encryption Schemes

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

by

Nithin S (221IT085)

Sanketh N S (221IT060)

Vrushank T S (221IT083)

under the guidance of

Dr. Purushothama B R

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA

SURATHKAL, MANGALORE - 575025

April, 2026

DECLARATION

We hereby *declare* that the Major Project Work Report entitled ***"Learning-Based Log Anomaly Detection, Cryptanalysis, and Design of Image Sharing and Encryption Schemes"***, which is being submitted to the **National Institute of Technology Karnataka, Surathkal**, for the award of the Degree of Bachelor of Technology in Information Technology, is a *bonafide report of the work carried out by us.* The material contained in this Project Report has not been submitted to any University or Institution for the award of any degree.

Name of the Student (Registration Number) with Signature

(1) Nithin S (221IT085)

(2) Sanketh NS (221IT060)

(3) Vrushank TS (221IT083)

Department of Information Technology
Place : NITK, Surathkal-575025
Date : 23/04/2026

CERTIFICATE

This is to *certify* that the Project Work Report entitled ***"Learning-Based Log Anomaly Detection, Cryptanalysis, and Design of Image Sharing and Encryption Schemes"*** submitted by

Name of the Student (Registration Number)

(1) Nithin S (221IT085)

(2) Sanketh NS (221IT060)

(3) Vrushank TS (221IT083)

as the record of the work carried out by them, is *accepted as the B.Tech. Major Project work report submission* in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in Information Technology.

Signature of Major Project Guide with date

Dr. Purushothama B R

Associate Professor

Department of Information Technology, NITK Surathkal

Chairman - DUGC

Signature with Date and Seal

ACKNOWLEDGEMENT

We express our profound gratitude to our project guide, Dr. Purushothama B R sir, for his continuous and invaluable guidance, patience, and unwavering support throughout the duration of this project. His deep expertise in the field, coupled with his insightful feedback, has been instrumental in shaping not only the direction of this research but also the methodologies employed in this work. His encouragement and mentorship were vital in overcoming challenges, allowing us to delve deeper into the complexities of log-based anomaly detection, cryptanalysis, and secret image sharing schemes with greater confidence.

Furthermore, we express our sincere appreciation to the Department of Information Technology at NIT Surathkal for providing the essential resources, infrastructure, and academic environment that facilitated this research. The availability of state-of-the-art tools, laboratories, and computing resources has been crucial in carrying out the extensive experiments and analyses required for this project. The department's commitment to fostering a culture of innovation and research has significantly contributed to the successful execution of this project, and we are deeply grateful for the opportunities offered in this regard.

ABSTRACT

This work presents seven interconnected projects spanning log-based anomaly detection, secret image sharing, and cryptanalysis of image encryption schemes.

The first chapter proposes a role-aware representation learning framework for log anomaly detection that overcomes limitations of prior parsing-free methods. Latent token roles — state, entity, action, quantity, and location — are induced automatically from anomaly labels without manual annotation, while numeric tokens are explicitly preserved. Evaluated on HDFS, BGL, Thunderbird, and Spirit, the framework achieves F1-scores of 0.98–0.99, consistently outperforming NeuralLog with negligible computational overhead.

The second chapter performs cryptanalysis of a chaos-based medical image encryption scheme (Arnold's Cat Map and 2D-LSCM) under the chosen-plaintext attack model. By exploiting the plaintext-independence of the keystream and the separability of permutation and diffusion stages, the entire encryption is inverted without the secret key using only $n + 1$ oracle queries for an $n \times n$ image, revealing fundamental weaknesses in chaos-based designs.

The third chapter extends chosen-plaintext cryptanalysis to a multi-chaos IoT image encryption scheme employing the Hénon map and 2D-LSCM, demonstrating analogous structural vulnerabilities and achieving exact plaintext recovery with minimal query complexity.

The fourth chapter introduces PoolLiteSIS, a lightweight secret image sharing scheme using average pooling for dimensional reduction combined with cyclic XOR-based shadow construction. The scheme achieves a 75% reduction in total storage over existing XOR-based methods while maintaining lossless reconstruction (PSNR $= \infty$, SSIM $= 1.0$) and enhanced security via residual permutation.

The fifth chapter assesses the vulnerability of XOR-based secret image sharing schemes to autoencoder-based reconstruction attacks. Experiments show that while reconstruction is feasible when all shares are available, it fails under missing-share conditions, empirically confirming the information-theoretic security guarantees of the XOR scheme.

The sixth chapter proposes an autoencoder-based secret image sharing scheme that leverages deep learning for storage-efficient and secure image distribution, training an encoder–decoder network to generate compact shares with competitive reconstruction quality measured by PSNR and SSIM.

The seventh chapter investigates optimization techniques for polynomial-based and XOR-based secret image sharing schemes, including leading-zero removal, digit decomposition of pixel values, Huffman coding, and pooling-permutation preprocessing. Results demonstrate significant shadow size reductions for XOR-based configurations while preserving lossless reconstruction across all polynomial-based variants.

Keywords— Log Anomaly Detection, Role-Aware Representation Learning, Token Role Induction, BERT, Parsing-Free Log Analysis, Deep Learning, Transformer, Representation Learning, Visual Cryptography, Secret Image Sharing, XOR Secret Sharing, PoolLiteSIS, Autoencoder, Reconstruction Attack, Adversarial Machine Learning, Image Security, Chaos-Based Encryption, Cryptanalysis, Chosen-Plaintext Attack, Arnold's Cat Map, 2D-LSCM, Hénon Map, Shadow Image Optimization

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

Role-Aware Log Representations for Enhanced Anomaly Detection via Latent Token Role Induction

This chapter presents a role-aware representation learning framework for log-based anomaly detection, outlining the limitations of existing parsing-based and parsing-free approaches and motivating the need for structurally informed representations. It introduces a method that performs latent token role induction and role-aware pooling to preserve functional distinctions within log messages, followed by a detailed description of the underlying concepts, model architecture, and training process, and concludes with experimental evaluation demonstrating improved detection performance and robustness across benchmark datasets.

1.1 Introduction

The automated identification of anomalous patterns in log message sequences generated by operational systems is known as **log-based anomaly detection**. Every time a software application or operating system performs an operation — starting a service, handling a request, or encountering an error — it records a brief textual message called a **log entry**. These messages form a continuously growing record of system behavior known as a **system log**. For example, a web server might produce entries such as:

```
[INFO] Connection established from 192.168.1.1
[ERROR] Disk write failed:  no space left on device
[WARN] Authentication attempt failed for user:  admin
```

1.2 Preliminaries

This section introduces the foundational concepts and theoretical background required to understand the log-based anomaly detection methodology developed in this work. Each concept is explained from first principles before the formal mathematical notation is presented. The three main areas covered are: log representation and anomaly formulation, token embedding and representation learning, and the self-attention mechanism.

1.2.1 Log Representation and Anomaly Formulation

Every time a software application or operating system performs an operation — starting a service, handling a request, or encountering an error — it records a brief textual message called a **log entry**. These messages form a continuously growing record of the system's behavior known as a **system log**. A typical system might produce entries such as:

```
[INFO] Connection established from 192.168.1.1
[ERROR] Disk write failed:  no space left on device
[WARN] Authentication attempt failed for user:  admin
```

When a system is behaving normally, its logs follow recognizable, repetitive patterns. When something goes wrong — a hardware failure, a security breach, or a software bug — the log patterns deviate from the norm. **Anomaly detection** is the task of automatically identifying these deviations.

Formally, each log message m is treated as a sequence of tokens (words or sub-word units):

$$m = \{ w_1, w_2, \ldots, w_T \} \tag{1.1}$$

where T is the number of tokens in the message and w_i denotes the i^{th} token. A **log sequence** S is a collection of L consecutive log messages observed within a fixed time window:

$$S = \{ m_1, m_2, \ldots, m_L \} \tag{1.2}$$

The anomaly detection task is formulated as a **binary classification** problem. Given a sequence S, a detection function f must assign it a label:

$$f(S) \longrightarrow \{0, 1\} \tag{1.3}$$

where 0 denotes *normal* behavior and 1 denotes *anomalous* behavior. The goal is to *learn* f from historical labeled data such that it generalizes reliably to unseen log sequences.

1.2.2 Token Embedding and Representation Learning

Machine learning models operate on numerical inputs, not raw text. An **embedding** is a mapping from each word or token to a dense numerical vector, constructed such that tokens with similar meanings are positioned close together in the resulting vector space. For instance, the tokens ``error'' and ``failure'' would ideally have similar vector representations, whereas ``error'' and ``timestamp'' would be far apart. This geometric structure allows downstream models to reason about semantic similarity without explicit symbolic rules.

Each token w_i is mapped to a continuous vector:

$$e_i = \text{Embed}(w_i) \in \mathbb{R}^d \tag{1.4}$$

where d is the **embedding dimension** — a hyperparameter typically set to 128 or 256. The full log message is then represented as the ordered sequence of its token vectors:

$$M = \{ e_1, e_2, \ldots, e_T \} \tag{1.5}$$

Log messages possess a rich internal structure. Within a single message, different tokens serve distinct functional *roles*: some are status keywords (e.g., `INFO`, `ERROR`), others are entity identifiers (e.g., a process ID or a username), and still others carry numeric values (e.g., a port number or a memory address). **Role-aware modeling** explicitly encodes these structural roles alongside token content during the embedding process. By preserving role distinctions rather than treating all tokens uniformly, the model learns richer and more discriminative representations — a property that is

especially valuable for detecting subtle anomalies in log sequences.

1.2.3 Self-Attention Mechanism

Not every token in a log message is equally relevant to understanding every other token. Consider the message: ``User admin login failed at port 22.'' To correctly interpret why ``failed'' is significant, a model must attend to ``login'' — not merely to the adjacent words. The **self-attention mechanism** [1] allows each token to dynamically focus on, or *attend to*, whichever other tokens are most relevant to it, regardless of their positional distance in the sequence. This is a key advantage over recurrent architectures, which struggle to propagate long-range dependencies through many sequential steps.

For each token embedding e_i, three separate linear projections are computed using learned weight matrices W_Q, W_K, and W_V:

$$Q_i = W_Q\, e_i \quad \text{(Query: what this token is looking for)} \tag{1.6}$$

$$K_i = W_K\, e_i \quad \text{(Key: what this token offers to others)} \tag{1.7}$$

$$V_i = W_V\, e_i \quad \text{(Value: the content this token contributes)} \tag{1.8}$$

Intuitively, the query of token i is matched against the keys of all other tokens in the sequence to determine how much attention token i should pay to each of them.

The raw attention score between token i and token j is computed as the dot product of their query and key vectors, scaled by $\sqrt{d}$ to stabilize gradients in high-dimensional settings:

$$\text{Attention}(i, j) = \frac{Q_i\, K_j^\top}{\sqrt{d}} \tag{1.9}$$

These raw scores are then normalized across all tokens using the **softmax** function to produce a probability distribution of attention weights:

$$\alpha_{ij} = \frac{\exp\big(\text{Attention}(i, j)\big)}{\sum_{k=1}^{T} \exp\big(\text{Attention}(i, k)\big)} \tag{1.10}$$

By construction, $\alpha_{ij} \geq 0$ and $\sum_{j=1}^{T} \alpha_{ij} = 1$, so the weights form a valid probability distribution over the sequence.

The final representation of token i is computed as a weighted sum of all value vectors, with tokens receiving higher attention weights contributing proportionally more:

$$z_i = \sum_{j=1}^{T} \alpha_{ij} V_j \tag{1.11}$$

The vector z_i is referred to as the **contextual representation** of token i, because it aggregates information from the entire sequence in a relevance-weighted manner. This is fundamentally different from a static embedding, which encodes only the identity of a token without any awareness of its surrounding context. In the log anomaly detection setting, contextual representations allow the model to capture inter-token dependencies that are critical for distinguishing normal log patterns from anomalous ones.

1.3 Literature Review

Log anomaly detection has been extensively studied as a critical technique for guaranteeing reliability and security in large-scale distributed systems. The field has evolved through several distinct paradigms — from classical statistical methods to deep sequence models and, most recently, transformer-based and large language model (LLM) approaches. This section surveys the key developments across these phases.

Semi-structured log messages combine variable parameters — such as block identifiers, IP addresses, error codes, and hardware locations — with fixed template components. A standard HDFS message, for instance, `"Receiving block blk12345 src: /10.0.2.15:50010 dest: /10.0.2.16:50010,"` contains variable parameters that differ with each event but adheres to a consistent structural template. When a system behaves normally, its logs follow recognizable, repetitive patterns; when something goes wrong — a hardware failure, a security breach, or a software bug — the log patterns deviate from the norm. The fundamental challenge posed by this semi-structured nature is that anomaly detection techniques must be sensitive to the subtle patterns that differentiate abnormal behavior from typical operation, while remaining

resilient to structural variation across different logging frameworks.

Early research applied mining-based techniques to console logs using statistical and clustering-based approaches [2, 3, 4]. These methods operated by first extracting structured log templates and then applying anomaly detection on the resulting event representations. PCA-based detection on event count vectors [2] and invariant mining [3] were among the most prominent techniques in this category. However, they were fundamentally limited by their dependence on accurate log parsing as a prerequisite step.

To address the parsing bottleneck, dedicated log parsers such as Drain [5] and LogCluster [6] were developed to automatically extract structured templates from raw log text. Standardized benchmarks and toolkits [7] were subsequently introduced to evaluate and compare parsing-based pipelines systematically. Despite these advances, comprehensive evaluations revealed that parser error rates on complex real-world logs ranged from 14% to 89%, with cascading degradation in downstream detection performance of up to 50% [7]. This exposed a fundamental fragility in the two-stage parse-then-detect paradigm.

Early log anomaly detection methods adopted a two-stage pipeline. Log parsers such as SLCT, PLoM [4], and Drain [5] first separated constant text from variable parameters to extract structured templates, which were then subjected to anomaly detection using techniques such as PCA on event count vectors [2] and invariant mining [3]. Although conceptually simple, this paradigm suffered from a critical dependence on parsing accuracy. Comprehensive benchmarks by Zhu et al. [7] revealed that on complex real-world logs, parser error rates ranged from 14% to 89%, with cascading effects that reduced downstream detection performance by as much as 50%.

The limitations of classical approaches motivated the adoption of deep learning for log anomaly detection. DeepLog [8] was among the first to employ an LSTM neural network to model sequential dependencies in log event streams, framing anomaly detection as a next-event prediction task: a log entry is flagged as anomalous if the observed event falls outside the top-k predicted candidates. LogAnomaly [9] extended this framework by jointly detecting both sequential anomalies and quantitative anomalies — deviations in the frequency of log event types — providing a more comprehensive detection capability.

Robustness to unstable and evolving log formats was addressed in LogRobust [10], which leveraged TF-IDF weighted semantic embeddings and an attention-based Bi-LSTM to produce log representations resilient to log evolution and noise. Although these deep learning methods significantly outperformed classical approaches, they continued to depend on a parsing step upstream, inheriting its associated vulnerabilities.

Deep learning methods such as DeepLog [8], LogAnomaly [9], and LogRobust [10] subsequently advanced the field by employing LSTMs, attention mechanisms, and semantic embeddings for sequence modeling. Although these techniques significantly outperformed classical methods, they still relied on a preceding log parsing step, making them vulnerable to evolving log formats and out-of-vocabulary templates.

The limitations of parsing-dependent pipelines spurred interest in parsing-free approaches. Log2Vec [11] proposed representing log messages as graph-structured data and learning embeddings directly from raw text without explicit template extraction. NeuralLog [12] went further by introducing a fully parsing-free framework that encodes raw log messages directly using pre-trained BERT [13] contextual embeddings, relying on WordPiece tokenization to handle out-of-vocabulary terms naturally. By averaging contextual token embeddings to form fixed-dimensional message representations, NeuralLog demonstrated that pre-trained language models can capture the semantics of raw log text without any parsing intermediary, achieving F1-scores above 0.95 on benchmarks where parsing-based methods plateaued below 0.60.

NeuralLog [12] introduced the *parsing-free* paradigm, demonstrating that BERT [13] can encode raw log messages directly without an intermediate parsing step. By using WordPiece tokenization [14] to handle out-of-vocabulary terms and averaging contextual token embeddings to generate fixed-dimensional message representations, NeuralLog achieved F1-scores above 0.95 on benchmark datasets where parsing-based methods had failed to surpass 0.60. This established that pre-trained language models can naturally capture the semantics of raw log text, and that parsing is not a necessary precondition for effective log anomaly detection.

Building on transformer architectures [1] and BERT-style pre-training, subsequent works have explored increasingly sophisticated attention-based representations. LAnoBERT [15] applied masked language modeling objectives to log-specific pre-

training, enabling the model to detect anomalies through token-level reconstruction error. LogFormer [16] proposed a unified transformer framework that jointly captures intra-message token dependencies and inter-message sequential patterns, achieving strong performance across multiple benchmark datasets.

The recent proliferation of large language models has opened new avenues for log analysis. Several studies have explored LLMs for log parsing tasks, examining their effectiveness [17], prompt engineering strategies [18], and dedicated parsing frameworks such as LLMParser [19], LILAC [20], DivLog [21], and LogBatcher [22]. These works collectively demonstrate that LLMs possess sufficient world knowledge to perform competitive log parsing and analysis, though challenges remain around inference cost, latency, and robustness to novel log formats at scale.

Despite this progress, NeuralLog and existing parsing-free methods share two key limitations. First, they treat all tokens within a log message uniformly by simply averaging their contextual embeddings. This approach conflates tokens with functionally distinct roles — states, entities, actions, quantities, and locations — into a single undifferentiated vector, discarding structural information that may be critical for anomaly identification. Second, numeric tokens are routinely discarded on the assumption that they contribute noise; however, this omits important anomaly indicators such as HTTP error codes, memory usage values, and hardware addresses.

A recurring theme across the literature, highlighted by comprehensive surveys [23], is the necessity of structurally aware log representation. Existing methods — including parsing-free approaches — largely treat log tokens uniformly, aggregating their embeddings without regard to the functional roles that individual tokens play within a message. This motivates the role-aware representation framework proposed in this work, which explicitly models token roles to construct more discriminative and semantically faithful log representations, as described in Section 6.4.

To overcome these limitations, we propose a **role-aware representation learning framework** that automatically identifies token roles from sequence-level anomaly labels, without requiring manual annotation or log parsing. A *role induction network* assigns a soft probability distribution over five role categories for each token, and *role-aware pooling* then constructs structured message representations that preserve both the semantic content and the structural organization of each log entry.

1.4 Work Done

This section presents the proposed role-aware log anomaly detection framework. The overall pipeline, illustrated in Figure 1.1, proceeds through five stages: preprocessing and tokenization, contextual token encoding via BERT, latent token role induction, role-aware message pooling, and transformer-based sequence classification.

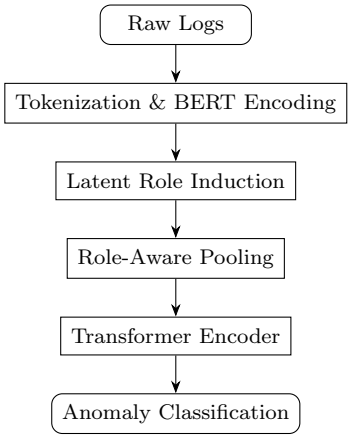


Figure 1.1: Overview of the proposed role-aware log anomaly detection pipeline.

1.4.1 Enhanced Preprocessing

Unlike NeuralLog [12], which discards all numeric tokens prior to encoding, the proposed preprocessing pipeline retains numeric tokens as first-class vocabulary items. Raw log messages are normalized to lowercase and tokenized using common delimiters such as whitespace, colons, and commas. Numeric tokens — including HTTP status codes, memory usage values, retry counts, and block identifiers — are explicitly preserved, as they frequently carry anomaly-relevant signal that blanket removal would destroy. This design choice reflects the observation that many real-world anomalies manifest precisely through anomalous numeric values, such as unexpected error codes or out-of-range memory addresses.

1.4.2 Token Encoding with BERT

Each preprocessed log message is tokenized using WordPiece tokenization [14], which decomposes rare or domain-specific terms into frequent subword units. This elimi-

nates the out-of-vocabulary issues that commonly arise in technical log vocabularies when using fixed lexicons. Token embeddings are then generated using BERT-base [13] (12 transformer layers, 768 hidden dimensions), producing a sequence of contextual representations:

$$\mathbf{H} = [\,\mathbf{h}_1,\, \mathbf{h}_2,\, \ldots,\, \mathbf{h}_n\,] \tag{1.12}$$

where each $\mathbf{h}_i \in \mathbb{R}^{768}$ encodes token i in the full context of its surrounding message. Unlike static word embeddings, these contextual representations capture the meaning of each token as it appears within a specific log entry, making them sensitive to the varying semantics of recurring keywords such as ``failed'' or ``connection'' across different message contexts.

1.4.3 Latent Token Role Induction

The central contribution of this work is the **latent token role induction** mechanism. The key hypothesis is that tokens in log messages fulfill distinct functional roles — some denote system states, others identify components, others describe operations, and some convey quantitative or spatial information — and that modeling these roles explicitly produces representations better suited to anomaly detection than naive uniform averaging.

Role Categories

Five latent role categories are defined:

- *State* — tokens describing system conditions (e.g., `error`, `timeout`, `success`);
- *Entity* — tokens identifying system components (e.g., `node`, `disk`, `process`);
- *Action* — tokens describing operations performed (e.g., `detected`, `killed`, `restarted`);
- *Quantity* — tokens carrying numeric or coded values (e.g., error codes, retry counts, block identifiers);

- *Location* — tokens denoting spatial references (e.g., file paths, registers, directory names).

Critically, no manual annotation of token roles is required. The role assignments are discovered automatically through end-to-end training, driven entirely by sequence-level anomaly labels via backpropagation.

Role Induction Network

For each token t_i with BERT embedding $\mathbf{h}_i \in \mathbb{R}^{768}$, a two-layer feedforward network predicts a soft role probability distribution:

$$\mathbf{r}_i = \text{softmax}\big(W_r \cdot \text{ReLU}(W_h\, \mathbf{h}_i + b_h) + b_r\big) \tag{1.13}$$

where $W_h \in \mathbb{R}^{256\times768}$ and $W_r \in \mathbb{R}^{5\times256}$ are learned weight matrices, and $\mathbf{r}_i \in \mathbb{R}^5$ is the resulting probability distribution over the five role categories. A role-aware token representation is then formed by modulating the BERT embedding with the predicted role information:

$$\mathbf{t}_i = \mathbf{h}_i \odot \mathbf{W}_e\, \mathbf{r}_i \tag{1.14}$$

where $\mathbf{W}_e \in \mathbb{R}^{768\times5}$ is a learned role embedding matrix and $\odot$ denotes element-wise multiplication. Since the role induction network receives no explicit role supervision and is trained solely from the anomaly detection gradient signal, tokens that contribute similarly to anomaly detection automatically converge to similar role assignments during training.

1.4.4 Role-Aware Message Representation

Rather than uniformly averaging all token embeddings as in NeuralLog [12], the proposed framework constructs a structured message representation by performing separate pooling operations for each role category. The per-role pools are computed as soft weighted averages:

$$\mathbf{m}_k = \frac{\sum_{i=1}^{n} r_{i,k} \cdot \mathbf{t}_i}{\sum_{i=1}^{n} r_{i,k}} \tag{1.15}$$

where $r_{i,k}$ is the probability that token i belongs to role k. The five role-specific sub-vectors are then concatenated to form the final message representation:

$$\mathbf{m} = [\,\mathbf{m}_{\text{state}};\ \mathbf{m}_{\text{entity}};\ \mathbf{m}_{\text{action}};\ \mathbf{m}_{\text{quantity}};\ \mathbf{m}_{\text{location}}\,] \tag{1.16}$$

The resulting message vector $\mathbf{m} \in \mathbb{R}^{3840}$ concatenates five role-specific sub-vectors of 768 dimensions each. This structured representation preserves the functional distinctions between different token types — distinctions that a single averaged vector would collapse — and thereby equips downstream models with richer, more interpretable inputs.

1.4.5 Sequence Modeling

Log sequences are constructed using a sliding window of size 20 and step 1, following the evaluation protocol of NeuralLog [12]. Sinusoidal positional encodings [1] are added to each message representation to inject ordering information into the otherwise permutation-invariant transformer:

$$\mathbf{x}_i = \mathbf{m}_i + \mathbf{p}_i \tag{1.17}$$

where $\mathbf{p}_i$ is the positional encoding vector for position i. A single-layer Transformer encoder with 12 attention heads and a feedforward dimension of 2048 is then applied over the full sequence to model temporal dependencies among log messages:

$$\mathbf{X}' = \text{TransformerEncoder}(\mathbf{X}) \tag{1.18}$$

This enables the model to capture inter-message patterns — such as recurring sequences of warnings preceding a critical failure — that are invisible to methods operating on individual log messages in isolation.

1.4.6 Classification and Training

A max-pooling operation is applied over the transformer sequence output to obtain a fixed-size sequence-level representation $\mathbf{s}$, which is passed through a fully connected

layer followed by softmax to produce the final anomaly prediction:

$$P(\text{anomaly} \mid \mathbf{s}) = \text{softmax}(W_c\,\mathbf{s} + b_c) \tag{1.19}$$

The full framework is trained end-to-end using binary cross-entropy loss:

$$\mathcal{L} = -\sum_{j}\Big[\, y_j \log \hat{y}_j + (1 - y_j)\log(1 - \hat{y}_j)\,\Big] \tag{1.20}$$

Training is performed using the AdamW optimizer with a learning rate of 3×10^{-4}, a batch size of 64, and a dropout rate of 0.1. Early stopping is applied after 5 consecutive epochs without validation improvement, with a maximum of 20 training epochs. Notably, sequence-level anomaly labels constitute the only supervision required; token-level role assignments emerge entirely through backpropagation without any auxiliary annotation effort.

1.5 Results and Analysis

This section presents the experimental evaluation of the proposed role-aware log anomaly detection framework. All experiments are conducted on four publicly available benchmark datasets widely used in log analysis research. The framework is compared against both traditional machine learning and deep learning baselines, with NeuralLog [12] serving as the primary reference point, given that the proposed method is a direct extension of that work.

1.5.1 Experimental Setup

The experimental protocol follows that of NeuralLog [12]. For the HDFS dataset, an 80/20 random split on block ID is used. For BGL, Thunderbird, and Spirit, an 80/20 temporal split is applied to preserve realistic evaluation conditions in which the test set reflects future, unseen log events. A fixed-size sliding window of size 20 with step size 1 is used uniformly across all datasets.

Performance is measured using Precision, Recall, and F1-Score. The baseline methods include traditional machine learning algorithms — Logistic Regression (LR) [24],

Table 1.1: Anomaly Detection Performance Comparison Across Benchmark Datasets

Dataset	Metric	Method						
		LR	**SVM**	**IM**	**LogRobust**	**Log2Vec**	**NeuralLog**	**Ours**
HDFS	Precision	0.99	0.99	1.00	0.98	0.94	0.96	**0.98**
	Recall	0.92	0.94	0.88	1.00	0.94	1.00	**1.00**
	F1-Score	0.96	0.96	0.94	0.99	0.94	0.98	**0.99**
BGL	Precision	0.13	0.97	0.13	0.62	0.80	0.98	**0.99**
	Recall	0.93	0.30	0.30	0.96	0.98	0.98	**0.99**
	F1-Score	0.23	0.46	0.18	0.75	0.88	0.98	**0.99**
Thunderbird	Precision	0.46	0.34	–	0.61	0.74	0.93	**0.96**
	Recall	0.91	0.91	–	0.78	0.94	1.00	**1.00**
	F1-Score	0.61	0.50	–	0.68	0.84	0.96	**0.98**
Spirit	Precision	0.89	0.88	–	0.97	0.91	0.98	**0.99**
	Recall	0.96	1.00	–	0.94	0.96	0.96	**0.98**
	F1-Score	0.92	0.93	–	0.95	0.95	0.97	**0.98**

Support Vector Machine (SVM) [25], and Invariant Mining (IM) [3] — as well as deep learning methods: LogRobust [10], Log2Vec [11], and NeuralLog [12].

1.5.2 Overall Detection Performance

Table 1.1 presents the anomaly detection performance of all compared methods across the four benchmark datasets.

The proposed framework achieves the best F1-score on all four datasets, consistently outperforming both parsing-dependent classical methods and the parsing-free NeuralLog baseline. Several notable observations emerge from these results.

The proposed method outperforms NeuralLog on every dataset, with F1-score gains of +0.01 on HDFS, +0.01 on BGL, +0.02 on Thunderbird, and +0.01 on Spirit. The largest improvement occurs on Thunderbird, a dataset characterized by high semantic ambiguity and class imbalance, where the benefit of role-separated representations is most pronounced. Perfect recall of 1.00 is achieved on both HDFS and Thunderbird, which is particularly significant in production environments where missed anomalies can have severe operational consequences.

The failure of traditional methods on BGL is stark: LR, SVM, and IM achieve F1-scores of only 0.23, 0.46, and 0.18, respectively. This degradation is attributable

to high parsing error rates on the BGL dataset, which corrupt the structured event representations that these methods depend upon. By operating directly on raw log text, the proposed framework and NeuralLog are immune to this source of error. On Spirit, the proposed model achieves a precision of 0.99 and recall of 0.98, yielding an F1-score of 0.98, marginally higher than NeuralLog's 0.97 — demonstrating consistent gains even on datasets with relatively structured log formats.

1.5.3 Key Observations

Our method also improves on NeuralLog consistently across all four datasets. We achieve higher F1-scores by margins of 0.01, 0.01, 0.02, and 0.01 on HDFS, BGL, Thunderbird, and Spirit, respectively. The improvement is most significant on difficult datasets such as BGL and Thunderbird, which have high parsing error rates and semantic ambiguity. We also achieve perfect recall (100%) on HDFS and Thunderbird while improving precision on both datasets. This indicates that our method can detect all anomalies while maintaining low false-alarm rates. Lastly, traditional methods (LR, SVM, IM) perform catastrophically on BGL, achieving F1-scores as low as 0.18 and as high as 0.46, mainly because of parsing failures. Our method and NeuralLog avoid these failures.

Example: BGL log message. Consider the BGL log message:

```
CE sym 14, at location R02-M1-N1-C:J12-U11 detected error in node core
```

After role induction, the model assigns the following approximate dominant roles (highest-probability role shown per token):
The role assignments in Table 1.2 align with human intuition despite receiving no explicit annotations: `error` and `CE` (correctable error) are assigned to the *state* role; `node` and `core` to the *entity* role; `detected` to the *action* role; `14` to *quantity*; and hardware location strings to the *location* role. This fine-grained attribution enables a system operator to understand *which aspect* of a log message (a hardware entity, a numeric threshold, a state keyword) drives the anomaly prediction, directly addressing the interpretability limitation of NeuralLog.

Table 1.2: Example token role assignments for a BGL log message. $r_{i,k}$ denotes the probability of token i belonging to role k.

Token	State	Entity	Action	Quantity	Location	Dominant Role
`CE`	0.62	0.12	0.08	0.10	0.08	State
`sym`	0.10	0.18	0.09	0.55	0.08	Quantity
`14`	0.05	0.06	0.05	0.80	0.04	Quantity
`location`	0.08	0.10	0.07	0.05	0.70	Location
`R02-M1-N1`	0.05	0.12	0.06	0.08	0.69	Location
`detected`	0.14	0.07	0.72	0.04	0.03	Action
`error`	0.85	0.06	0.04	0.03	0.02	State
`node`	0.08	0.78	0.06	0.04	0.04	Entity
`core`	0.07	0.80	0.05	0.05	0.03	Entity

1.5.4 Ablation Studies

To quantify the individual contribution of each design decision in the proposed framework, we conduct a series of ablation experiments in which each component is removed in isolation and the resulting change in F1-score is measured.

Impact of Role Modeling

Table 1.3 presents the effect of removing the role induction mechanism entirely, reverting to NeuralLog's uniform averaging of token embeddings.

Table 1.3: Ablation Study: Impact of Role Modeling on F1-Score

Dataset	Without Roles	With Roles	Gain
HDFS	0.98	0.99	+0.01
BGL	0.98	0.99	+0.01
Thunderbird	0.96	0.98	+0.02
Spirit	0.97	0.98	+0.01

Role modeling consistently improves F1-score across all four datasets. The most notable gain appears on Thunderbird (+0.02), which is consistent with the expectation that role-separated pooling is most beneficial when log messages contain tokens

Table 1.4: Ablation Study: Impact of Numeric Token Preservation on F1-Score

Dataset	Numerics Removed	Numerics Retained	Gain
HDFS	0.98	0.99	+0.01
BGL	0.96	0.99	+0.03
Thunderbird	0.95	0.98	+0.03
Spirit	0.97	0.98	+0.01

with functionally distinct roles that uniform averaging conflates. The more modest gains on HDFS, BGL, and Spirit (+0.01) reflect the relatively more structured log formats of those datasets, where role boundaries are less ambiguous and the benefit of explicit separation is smaller, though still measurable.

Impact of Numeric Token Preservation

Table 1.4 presents the effect of discarding numeric tokens — as NeuralLog does — versus retaining them as first-class vocabulary items, as in the proposed framework.

The largest gains from numeric preservation occur on BGL and Thunderbird (+0.03 each), both of which contain hardware-level logs where numeric tokens such as error codes, memory addresses, and register values are primary indicators of anomalous conditions. Discarding them causes a significant loss of discriminative information, confirming that numeric preservation is essential for these datasets. The more modest gains on HDFS and Spirit (+0.01) reflect the greater reliance of those datasets on textual patterns for anomaly identification, though numeric retention still provides a marginal benefit.

Impact of Number of Role Categories

We evaluate F1-scores using 3, 5, 7, and 10 role categories across all datasets to determine the optimal role cardinality. Performance peaks consistently at 5 roles, which best balances representational expressiveness against the risk of overfitting. Increasing to 7 or 10 roles yields no further improvement and slightly degrades performance on smaller datasets, suggesting that additional roles fragment meaningful token clusters unnecessarily without introducing new discriminative structure.

Table 1.5: Top Tokens per Learned Role Category (BGL Dataset)

Learned Role	**Representative Tokens**
State	`error`, `failed`, `timeout`, `interrupt`, `success`
Entity	`node`, `core`, `memory`, `processor`, `link`
Action	`detected`, `killed`, `terminated`, `restarted`
Quantity	`404`, `500`, `0x00`, `pid`, `address`
Location	`/dev`, `/proc`, `register`, `buffer`

1.5.5 Learned Role Interpretation

To validate that the automatically induced roles correspond to semantically meaningful categories, we examine the tokens most strongly assigned to each role in the BGL dataset. Table 1.5 lists the representative tokens per learned role.

Despite receiving no human annotation, the learned roles align closely with intuitive functional categories. Tokens such as `error` and `failed` correctly cluster under the state role, while system component tokens such as `node` and `core` are assigned the entity role. Numeric anomaly indicators — including HTTP error codes `404` and `500` and hexadecimal values such as `0x00` — are grouped under the quantity role, validating the importance of numeric token preservation and confirming that the model learns to functionally distinguish numeric tokens from textual ones without supervision. File system paths such as `/dev` and `/proc`, along with hardware references such as `register` and `buffer`, are correctly grouped under the location role, demonstrating that the induction mechanism captures spatial and architectural context through backpropagation alone.

1.5.6 Computational Efficiency

Table 1.6 compares the preprocessing time, training time, and inference latency of all methods on the BGL dataset.

The proposed method introduces a modest overhead relative to NeuralLog: an additional 1.5 minutes of preprocessing, 1.5 minutes of training, and 0.7 ms per sequence of inference latency. This overhead is attributable to the role induction network and the five-fold increase in message representation dimensionality (3,840 vs. 768 dimensions). Despite this, the method remains substantially faster than parsing-dependent

Table 1.6: Computational Efficiency Comparison (BGL Dataset)

Method	**Preprocess (min)**	**Training (min)**	**Inference (ms/seq)**
SVM	102.0	0.6	0.1
LogRobust	102.0	2.2	0.2
Log2Vec	314.0	13.1	26.3
NeuralLog	14.3	5.2	3.1
Ours	15.8	6.7	3.8

baselines: Log2Vec requires 314 minutes of preprocessing and SVM requires 102 minutes, both owing to the upstream log parsing step that the proposed framework eliminates entirely. At 3.8 ms per sequence, inference latency remains well within practical bounds for near-real-time anomaly detection in production systems, where the accuracy gains from role-aware modeling justify the marginal increase in computational cost.

1.5.7 Robustness to Out-of-Vocabulary Tokens

Following the out-of-vocabulary (OOV) analysis protocol of NeuralLog [12], we evaluate robustness under a 60/40 temporal split on BGL, a configuration designed to maximize the proportion of log templates unseen during training. Under this setting, the OOV rate in the test set reaches 94.12%. The proposed model maintains an F1-score of 0.99 under this highly adversarial condition, demonstrating that the combination of WordPiece subword tokenization and role-aware representations generalizes effectively to unseen log formats with no degradation relative to the standard 80/20 evaluation split. This confirms that the framework is robust to the log evolution and vocabulary drift that frequently occur in long-running production systems.

1.6 Summary

In this chapter, we have proposed a novel role-aware representation learning framework for log-based anomaly detection, which overcomes the uniform treatment of tokens by existing methods, such as NeuralLog. Without manually crafting annotations or parsing rules, we model the structural variability of log messages by explicitly

inferring token roles in a latent space. Our approach attained state-of-the-art performance with F1-scores ranging from 0.98 to 0.99 across four public datasets, consistently outperforming NeuralLog as well as all baseline methods. Extensive ablation studies confirm that the improvement in accuracy arises from both role modeling and numeric token preservation, while the learned roles correspond to semantically meaningful categories based on mere anomaly labels, such as state, entity, action, quantity, and location.

This chapter advances reliable, understandable, and parsing-free log analysis by providing better explainability, better handling of evolving log formats, and higher accuracy through the preservation of both semantic and structural information. Our key insight is twofold: treating all tokens uniformly loses valuable structural information, whereas role modeling provides both flexibility and structure without parser fragility.

CHAPTER 2

Cryptanalysis of a Medical Image Encryption Scheme Using Multiple Chaotic Maps

This chapter presents a cryptanalysis of a chaos-based medical image encryption scheme that combines Arnold's Cat Map and a two-dimensional logistic sine coupling map, demonstrating its vulnerability under a chosen-plaintext attack model. It explains how structural weaknesses, specifically plaintext-independent keystream generation and the separability of permutation and diffusion stages, enable complete recovery of both the keystream and permutation mapping, allowing exact decryption without knowledge of the secret key, and concludes with theoretical and experimental validation highlighting the gap between statistical performance and actual cryptographic security.

2.1 Introduction

Chaos-based image encryption schemes are widely used for securing medical images due to their ability to achieve high confusion and diffusion with relatively low computational cost. These schemes typically follow a *permutation-diffusion* architecture, in which pixel positions are first scrambled and then pixel values are modified using a pseudorandom keystream generated from chaotic systems. Despite exhibiting strong statistical performance, many such schemes fail to provide adequate resistance against stronger adversarial models.

This chapter presents a detailed cryptanalysis of a medical image encryption scheme that combines *Arnold's Cat Map* and a *Two-Dimensional Logistic Sine Coupling Map (2D-LSCM)*. The analysis is conducted under the **chosen-plaintext attack (CPA)** model, wherein the attacker has access to an encryption oracle. It is demonstrated that the keystream employed in the diffusion stage is independent of the plaintext, making it recoverable using a single carefully chosen input. Furthermore,

the permutation process can be fully reconstructed using structured probe images. By combining the recovery of these two components, the complete encryption process can be inverted without any knowledge of the secret key.

The results demonstrate that the scheme is fundamentally insecure in spite of possessing a large key space and passing standard statistical tests. This highlights the critical importance of designing encryption algorithms in which keystream generation is dependent on the plaintext, and in which the permutation and diffusion stages are tightly coupled. The study offers insights into common structural weaknesses in chaos-based encryption and underscores the need for rigorous cryptanalysis when evaluating the security of such schemes.

2.2 Preliminaries

This section introduces the foundational concepts and theoretical background required to understand the log-based anomaly detection methodology developed in this work. Each concept is explained from first principles before the formal mathematical notation is presented. The three main areas covered are: log representation and anomaly formulation, token embedding and representation learning, and the self-attention mechanism.

2.2.1 Chaos-Based Image Encryption

Medical images and other sensitive visual data must be adequately protected prior to transmission or storage. Classical block ciphers such as AES operate on byte streams and do not exploit the inherent spatial structure of images. **Chaos-based encryption** offers a compelling alternative: it leverages the output of *chaotic dynamical systems*—mathematical systems exhibiting deterministic yet unpredictable, highly sensitive behaviour—as keystreams to scramble image data.

Properties of Chaotic Systems Relevant to Cryptography:

A chaotic system is characterised by the following properties, which make it attractive for generating cryptographic material:

- **Sensitivity to initial conditions**: Infinitesimally small changes in the initial state (on the order of 10^{-15}) lead to entirely different trajectories over time. This renders the generated sequence practically unpredictable without exact knowledge of the initial conditions, which serve as the secret key.
- **Ergodicity**: The system visits all regions of its state space over time, ensuring that the generated sequence exhibits good statistical coverage and uniformity.
- **Mixing property**: Nearby initial states rapidly spread across the entire state space, contributing to effective diffusion of information throughout the output sequence.

Permutation-Diffusion Architecture:

A typical chaos-based image encryption scheme follows a two-phase structure:

- **Permutation Phase**: The spatial positions of pixels are rearranged using a chaotic map. The content of each pixel remains unchanged, but its location in the image is scrambled. This destroys the spatial correlations between adjacent pixels.
- **Diffusion Phase**: The intensity values of pixels are modified using a chaotic keystream, typically via bitwise XOR operations. This ensures that a change in even a single plaintext pixel cascades into widespread changes throughout the ciphertext.

Together, permutation provides *confusion* and diffusion provides *spreading*, corresponding directly to Shannon's classical principles of secure cipher design.

Arnold's Cat Map is a classical area-preserving chaotic map widely used for pixel permutation in image encryption. For an image of size $n \times n$, it maps each pixel at position (x, y) to a new position (x', y') according to:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \bmod n. \tag{2.1}$$

Each application of this map shuffles pixel positions in a deterministic yet seemingly random manner. Importantly, the map is *periodic*: after a finite number of iterations,

the image returns to its original configuration. This periodicity property is exploited both in the encryption process and in certain cryptanalytic attacks.

The **Two-Dimensional Logistic Sine Coupling Map (2D-LSCM)** is a higher-dimensional chaotic system designed to overcome the limited chaotic range and non-uniform distribution of simple one-dimensional maps. It generates two mutually coupled chaotic sequences $\{x_i\}$ and $\{y_i\}$ via the following recurrence relations:

$$\begin{cases} x_{i+1} = \sin\Big(\pi\big(4\theta\, x_i(1-x_i) + (1-\theta)\sin(\pi y_i)\big)\Big) \\ y_{i+1} = \sin\Big(\pi\big(4\theta\, y_i(1-y_i) + (1-\theta)\sin(\pi x_{i+1})\big)\Big) \end{cases} \tag{2.2}$$

where $\theta \in (0, 1)$ is a coupling parameter that governs the balance between the logistic and sine components. The coupling between the two sequences produces a wider and more uniform chaotic range than either component alone. The output sequences serve as keystreams for both the permutation and diffusion phases of the encryption scheme.

2.2.2 Cryptanalysis of Image Encryption Schemes

Cryptanalysis is the science of analysing encryption schemes to identify weaknesses that allow an adversary to recover plaintext or the secret key without legitimate authorisation. In the context of image encryption, successful cryptanalysis corresponds to recovering the original image from its encrypted form. Different attack models assume varying levels of adversary capability, and this work focuses on the strongest standard model used in symmetric cryptography.

A **Chosen-Plaintext Attack (CPA)** assumes that the adversary has access to an encryption oracle that returns ciphertext $C = \mathcal{E}(P)$ for any selected plaintext image P. This allows the adversary to choose inputs strategically in order to extract information about the secret key or the internal structure of the cipher. Under modern cryptographic standards, any scheme that fails to resist CPA is considered insecure.

Many chaos-based image encryption schemes exhibit structural weaknesses that make them vulnerable to chosen-plaintext attacks. One common weakness arises in the diffusion phase when each pixel is XORed with a keystream that is independent of the plaintext. In this case, encrypting an all-zero image $P = \mathbf{0}$ produces $C =$

$\mathbf{0} \oplus K = K$, which directly reveals the keystream. Once the keystream is known, any ciphertext generated using the same key can be decrypted.

Another common weakness appears when permutation and diffusion are independent operations. If the permutation mapping does not depend on pixel values, it can be recovered using structured probe images. For example, an image containing a single non-zero pixel allows the adversary to observe its new position after encryption. Repeating this process for all pixel positions enables reconstruction of the complete permutation mapping.

An encryption scheme is therefore fully breakable under a chosen-plaintext attack if two conditions hold: the keystream used in diffusion is static and independent of the plaintext, and the permutation and diffusion processes are independent of each other. When both conditions are satisfied, the adversary can recover the keystream, reconstruct the permutation mapping, and invert the entire encryption process without requiring the secret key.

2.3 Literature Review

Chaos-based image encryption has been extensively studied owing to its ability to achieve strong confusion and diffusion properties through nonlinear dynamical systems. The foundational permutation-diffusion architecture was introduced by Fridrich [26], in which chaotic maps drive both the spatial scrambling and the value transformation of pixels. This two-phase design has since become the dominant paradigm in the field: the permutation phase disrupts spatial correlations among adjacent pixels, while the diffusion phase ensures that local changes in the plaintext propagate globally across the ciphertext.

Building on this framework, numerous schemes have been proposed by combining different chaotic systems to enhance complexity and enlarge the key space. Guan et al. [27] employed Arnold's Cat Map in conjunction with Chen's chaotic system to increase permutation complexity, while Wu et al. [28] explored logistic maps for both permutation and diffusion. Subsequent works incorporated higher-dimensional chaotic systems and hybrid approaches to improve randomness and statistical unpredictability [29, 30, 31]. These schemes are commonly evaluated using statistical

metrics such as information entropy, inter-pixel correlation coefficients, Number of Pixel Change Rate (NPCR), and Unified Average Changing Intensity (UACI), and they typically report strong performance on all such measures.

Despite these reported improvements, a substantial body of research has demonstrated that statistical strength does not imply cryptographic security. Cokal and Solak [32] showed that certain chaotic image ciphers are vulnerable to known-plaintext and chosen-plaintext attacks due to weak coupling between encryption stages. Xie et al. [33] further exposed structural vulnerabilities in permutation-diffusion architectures arising from the independence of the two phases. More recent work by Wen et al. [34, 35, 36] has systematically shown that plaintext-independent keystream generation constitutes a fundamental design flaw, as it enables adversaries to recover equivalent keys through straightforward algebraic analysis under a chosen-plaintext model.

A recurring and critical weakness across many of these schemes is that the keystream used during the diffusion stage is derived solely from the secret key, with no dependence on the plaintext. This renders the keystream reusable across multiple encryptions—effectively reducing the scheme to a one-time pad applied repeatedly, a well-known cryptographic vulnerability. Under a chosen-plaintext attack, an adversary can exploit this property by querying the encryption oracle with carefully chosen inputs to extract the keystream and subsequently invert any ciphertext encrypted under the same key. Compounding this issue, the separability of the permutation and diffusion stages allows an attacker to independently recover the pixel position mapping and the value transformation, further simplifying the cryptanalysis.

The scheme proposed by Jain et al. [31], which combines Arnold's Cat Map with a Two-Dimensional Logistic Sine Coupling Map for the purpose of medical image encryption, represents a recent effort to improve security in this domain. However, consistent with the structural patterns observed in prior work, it does not incorporate plaintext-dependent key generation, nor does it establish sufficient coupling between the permutation and diffusion stages. These omissions leave the scheme susceptible to the same class of structural chosen-plaintext attacks that have undermined earlier designs.

The present work builds directly upon these observations. By recovering the

equivalent keystream through encryption of an all-zero image and reconstructing the full permutation mapping using structured probe images, it is shown that the encryption process can be completely inverted without any knowledge of the secret key. This result underscores the persistent gap between statistical security evaluation and genuine cryptographic robustness, and reinforces the need for principled cryptanalysis—rather than statistical testing alone—in the assessment of chaos-based image encryption schemes.

2.4 Work Done

This work presents a complete cryptanalysis of the chaos-based medical image encryption scheme proposed by Jain et al. [31]. The scheme employs a two-round permutation-diffusion structure using Arnold's Cat Map and a Two-Dimensional Logistic Sine Coupling Map (2D-LSCM). It is demonstrated that this structure provides no actual resistance against a chosen-plaintext adversary: both the equivalent keystream and the full permutation mapping can be extracted with a small number of oracle queries, after which any ciphertext may be decrypted without knowledge of the secret key.

2.4.1 Overview of the Target Encryption Scheme

Two distinct chaotic maps form the algorithmic backbone of the scheme. Arnold's Cat Map handles pixel-position shuffling during the permutation phase, while the 2D-LSCM drives keystream generation for the diffusion phase.

The 2D-LSCM is constructed by coupling the one-dimensional logistic map with the one-dimensional sine map. The logistic map is given by

$$x_{i+1} = r\, x_i(1 - x_i), \tag{2.3}$$

where r denotes the growth-rate parameter and $x_i \in (0, 1)$. The sine map takes the form

$$x_{i+1} = \alpha \sin(\pi x_i), \tag{2.4}$$

with control parameter $\alpha \in [0, 1]$. Coupling these two one-dimensional systems

through a sine nonlinearity yields the 2D-LSCM:

$$\begin{cases} x_{i+1} = \sin\Big(\pi\big(4\theta\, x_i(1-x_i) + (1-\theta)\sin(\pi y_i)\big)\Big), \\ y_{i+1} = \sin\Big(\pi\big(4\theta\, y_i(1-y_i) + (1-\theta)\sin(\pi x_{i+1})\big)\Big), \end{cases} \tag{2.5}$$

where $\theta \in [0, 1]$ is the coupling control parameter. By operating in a substantially higher-dimensional phase space than either constituent map, the 2D-LSCM achieves greater ergodicity and a wider chaotic parameter range.

Arnold's Cat Map is a classical area-preserving automorphism of the torus that, when applied to a square image, performs a deterministic reordering of pixel coordinates. For an image of size $n \times n$, the transformation is

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \bmod\ n, \tag{2.6}$$

where (x, y) and (x', y') denote the pre-image and image pixel coordinates, respectively. The mapping is periodic with period $3n+1$, meaning that repeated application returns the image to its original state after exactly $3n + 1$ iterations. The iteration count P_i, satisfying $1 < P_i < 3n + 1$, constitutes the secret parameter K_1 for this component.

2.4.2 Encryption Process

Let P be a plaintext image of dimensions $M \times N$. Encryption is organised as two consecutive rounds, each comprising a permutation phase followed by a diffusion phase.

Permutation

The permutation phase introduces confusion by relocating pixel positions under the joint action of both chaotic maps. Arnold's Cat Map is applied for P_i iterations, and a chaotic matrix X is constructed by iterating the 2D-LSCM; column-wise sorting of X then yields a permutation index matrix that governs the final repositioning of pixel values, producing the permuted image T. Critically, T depends exclusively on

X and P_i: no pixel value from P influences the permutation itself.

Diffusion

The diffusion phase modifies pixel values so that any single-pixel change in the plaintext propagates throughout the ciphertext. Let T be the permuted image and X the chaotic matrix. For a linearised image sequence of length $O = M \times N$, the i-th encrypted pixel E_i is computed as

$$E_i = \begin{cases} T_1 + T_O + T_{O-1} + X_i \bmod F, & i = 1, \\ T_2 + E_1 + T_O + X_i \bmod F, & i = 2, \\ T_i + E_{i-1} + E_{i-2} + X_i \bmod F, & i \geq 3, \end{cases} \tag{2.7}$$

where $F = 255$ for 8-bit imagery and $X_i = \lfloor x_i \times 2^{32} \rfloor$ is the i-th element of the scaled chaotic sequence. The recursive dependence on E_{i-1} and E_{i-2} establishes an inter-pixel dependency chain that ensures downstream propagation of any modification. The complete permutation-diffusion cycle is applied for two rounds to yield the final ciphertext C. Figure 2.1 illustrates the full encryption process of the target scheme.

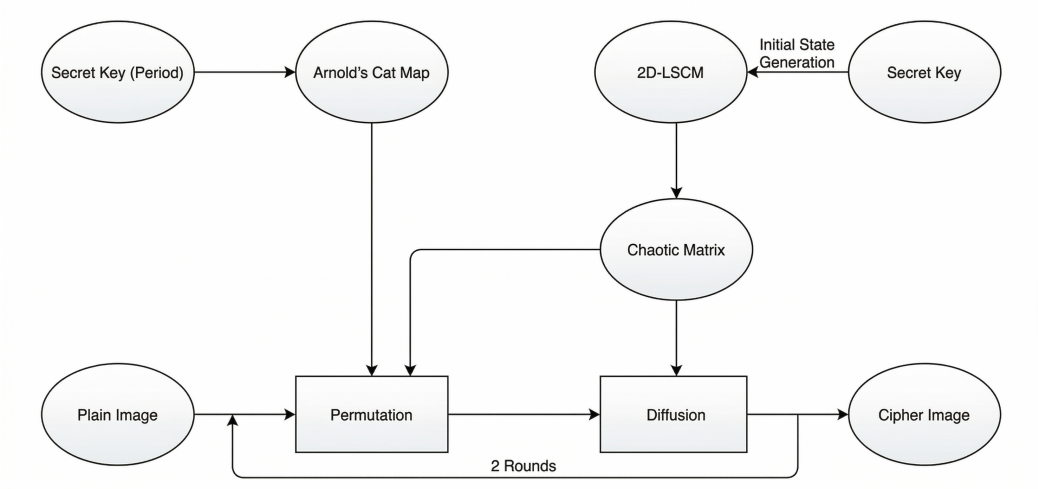


Figure 2.1: Encryption process of the target scheme [31].

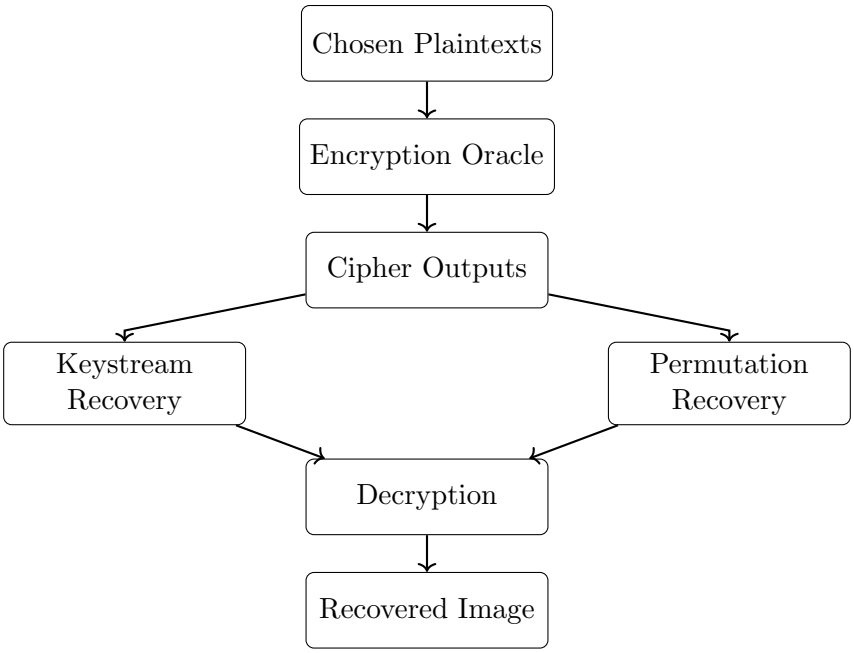


Figure 2.2: Block diagram of the proposed chosen-plaintext attack: keystream recovery, permutation recovery, and final decryption without the secret key.

2.4.3 Chosen-Plaintext Attack Framework

The overall attack framework is shown in Figure 2.2, comprising three phases: keystream extraction, permutation map reconstruction, and full decryption of any target ciphertext.

The adversary is assumed to have black-box access to an encryption oracle $\mathcal{E}(\cdot)$ that, given any $n \times n$ image P, returns the corresponding ciphertext $C = \mathcal{E}(P)$ under an unknown fixed key (K_1, K_2). The adversary has no knowledge of any internal parameters: the initial conditions x_0, y_0, the coupling parameter θ, the iteration count P_i, the chaotic matrix X, or the permutation map $\mathcal{M}$.

The exploitable structural weakness is the *plaintext-independence* of all key material. Because the 2D-LSCM sequence $\{x_i\}$ and the derived permutation map $\mathcal{M}$ are functions solely of the secret key—not of the image being encrypted—they remain identical across all encryptions of equal-sized images under the same key. This renders them static equivalent keys, a well-documented vulnerability class in chaos-based cipher design.

The attack proceeds through three phases:

Algorithm 1: Chosen-Plaintext Attack on the Target Scheme

Input: Encryption oracle $\mathcal{E}(\cdot)$; target ciphertext C; image size n.
Output: Recovered plaintext $\hat{P}$.

`// Phase 1: Keystream Recovery`
$C^{(0)} \leftarrow \mathcal{E}(\mathbf{0}_{n\times n})$;
Compute $\hat{X}_i$ for all i using Eq. (2.13);
`// Phase 2: Permutation Map Recovery`
for $j = 1$ **to** n **do**
  Construct probe image $P^{(j)}$;
    Set $P^{(j)}_{r,j} \leftarrow r$ for all r;
    Set $P^{(j)}_{r,k} \leftarrow 0$ for all $k \neq j$;
  Compute $Q^{(j)} \leftarrow \mathrm{ACM}^{(3n+1-P_i)}\big(P^{(j)}\big)$;
  $C^{(j)} \leftarrow \mathcal{E}(Q^{(j)})$;
  $\hat{T}^{(j)} \leftarrow \mathcal{D}_{\hat{X}}(C^{(j)})$;
  $\hat{\sigma}_j \leftarrow \hat{T}^{(j)}[:, j]$;
end
`// Phase 3: Decryption of Target Ciphertext`
$\hat{T} \leftarrow \mathcal{D}_{\hat{X}}(C)$;
for $j = 1$ **to** n **do**
  $\hat{S}[\hat{\sigma}_j(:),\ j] \leftarrow \hat{T}[:, j]$;
end
$\hat{P} \leftarrow \mathrm{ACM}^{(3n+1-\tilde{P}_i)}(\hat{S})$;
return $\hat{P}$

1. **Phase 1 — Keystream extraction** by encrypting an all-zero image.
2. **Phase 2 — Permutation map reconstruction** via n structured probe images.
3. **Phase 3 — Full decryption** of any target ciphertext using the recovered parameters.

The data complexity is $n + 1$ oracle queries for an $n \times n$ image, and the total computational cost is $\mathcal{O}(n^3)$, dominated by the Arnold map applications in Phase 2.

2.4.4 Phase 1: Keystream Recovery via a Zero Plaintext

This phase exploits the encryption of an all-zero image to decouple the diffusion layer from the permutation layer and directly expose the keystream.

Structural Observation

Let $\mathbf{0}$ denote the $n \times n$ all-zero image and let $C^{(0)} = \mathcal{E}(\mathbf{0})$.

Step 1: Arnold's Cat Map acts as the identity on a zero image. Arnold's Cat Map permutes pixel *coordinates* while leaving pixel *values* unchanged. Since every pixel of $\mathbf{0}$ carries the value zero, the map produces the same all-zero image regardless of the iteration count:

$$\mathrm{ACM}^{P_i}(\mathbf{0}) = \mathbf{0}. \tag{2.8}$$

Step 2: The 2D-LSCM column permutation acts as the identity on a zero image. The column-wise permutation step rearranges rows within column j according to a permutation σ_j derived from chaotic matrix X. When $P = \mathbf{0}$, every element selected by the permutation equals zero, giving:

$$T^{(0)} = \mathbf{0}. \tag{2.9}$$

Step 3: Diffusion collapses to a pure keystream recurrence. With $T_i^{(0)} = 0$ for all i, the diffusion equation (2.7) simplifies to:

$$C_1^{(0)} = X_1, \tag{2.10}$$

$$C_2^{(0)} = \big(X_1 + X_2\big) \bmod F, \tag{2.11}$$

$$C_i^{(0)} = \big(C_{i-1}^{(0)} + C_{i-2}^{(0)} + X_i\big) \bmod F, \quad i \geq 3. \tag{2.12}$$

Rearranging yields the keystream recovery formula:

$$\boxed{\hat{X}_i = \begin{cases} C_1^{(0)}, & i = 1, \\ \left(C_2^{(0)} - C_1^{(0)}\right) \bmod F, & i = 2, \\ \left(C_i^{(0)} - C_{i-1}^{(0)} - C_{i-2}^{(0)}\right) \bmod F, & i \geq 3. \end{cases}} \tag{2.13}$$

Correctness of Keystream Recovery

Proposition 1. *Every value $\hat{X}_i$ recovered by Eq. (2.13) satisfies $\hat{X}_i = X_i$ for all $i = 1, \ldots, N$.*

Proof. For $i = 1$: directly, $\hat{X}_1 = C_1^{(0)} = X_1$. For $i = 2$: $\hat{X}_2 = (C_2^{(0)} - C_1^{(0)}) \bmod F = X_2$. For $i \geq 3$: by induction, assuming $\hat{X}_k = X_k$ for all $k < i$, it follows that $\hat{X}_i = (C_i^{(0)} - C_{i-1}^{(0)} - C_{i-2}^{(0)}) \bmod F = X_i$. □

The complete keystream is thus recovered exactly from a *single* oracle query.

2.4.5 Phase 2: Permutation Map Reconstruction

With the keystream $\{\hat{X}_i\}$ in hand, the full permutation map $\mathcal{M}$ is reconstructed using exactly n additional oracle queries.

Permutation Structure

From the encryption process, the permutation stage applies Arnold's Cat Map for P_i iterations and then performs a column-wise sorting step driven by the chaotic matrix X. Specifically, for column j, if $\sigma_j = \text{argsort}(X[:, j])$ is the permutation induced by sorting that column, the permuted output satisfies

$$T[:, j] = S[\sigma_j,\ j], \tag{2.14}$$

where $S = \text{ACM}^{P_i}(P)$. The attacker seeks to determine $\mathcal{M}[:, j] = \sigma_j$ for every column j.

Probe Plaintext Design

For each column index $j \in \{1, \ldots, n\}$, a probe image $P^{(j)}$ is constructed such that column j encodes the row-index sequence $0, 1, \ldots, n-1$ and every other column is identically zero:

$$P^{(j)}_{r,c} = \begin{cases} r, & c = j, \\ 0, & c \neq j. \end{cases} \tag{2.15}$$

To neutralise the Arnold preprocessing, the probe submitted to the oracle is the P_i-step pre-image of $P^{(j)}$ under the Arnold map, exploiting the map's periodicity:

$$Q^{(j)} = \mathrm{ACM}^{-P_i}\big(P^{(j)}\big) = \mathrm{ACM}^{(3n+1-P_i)}\big(P^{(j)}\big). \tag{2.16}$$

Oracle Output and Permutation Recovery

When the oracle processes $Q^{(j)}$, the Arnold step undoes the pre-imaging, so that the effective input to the column-sort is precisely $P^{(j)}$. Since column j of $P^{(j)}$ encodes the row indices r, the column-sort directly reads out the permutation:

$$T^{(j)}[\ell,\ j] = \sigma_j(\ell), \qquad \ell = 1, \ldots, n. \tag{2.17}$$

All other columns are zero and carry no extraneous information. The oracle applies diffusion to yield $C^{(j)} = \mathcal{E}(Q^{(j)})$. Inverting the diffusion using the recovered $\hat{X}$:

$$\hat{T}^{(j)}_i = \begin{cases} \big(C^{(j)}_1 - C^{(j)}_N - C^{(j)}_{N-1} - \hat{X}_1\big) \bmod F, & i = 1, \\ \big(C^{(j)}_2 - C^{(j)}_1 - C^{(j)}_N - \hat{X}_2\big) \bmod F, & i = 2, \\ \big(C^{(j)}_i - C^{(j)}_{i-1} - C^{(j)}_{i-2} - \hat{X}_i\big) \bmod F, & i \geq 3, \end{cases} \tag{2.18}$$

from which the permutation indices for column j are read directly:

$$\hat{\sigma}_j(\ell) = \hat{T}^{(j)}[\ell,\ j], \qquad \ell = 1, \ldots, n. \tag{2.19}$$

Iterating over all $j = 1, \ldots, n$ completes the reconstruction of $\hat{\mathcal{M}}$.

Proposition 2. *The recovered map satisfies $\hat{\mathcal{M}} = \mathcal{M}$ exactly.*

Proof. The probe design guarantees $T^{(j)}[\ell, j] = \sigma_j(\ell)$. Since Proposition 1 guarantees $\hat{X}_i = X_i$ for all i, the inverse diffusion in Eq. (2.18) recovers $\hat{T}^{(j)} = T^{(j)}$ without error, from which $\hat{\sigma}_j = \sigma_j$ for every column j. □

2.4.6 Phase 3: Complete Decryption of Arbitrary Ciphertexts

Armed with the recovered keystream $\hat{X}$ and permutation map $\hat{\mathcal{M}}$, decryption of any ciphertext $C = \mathcal{E}(P)$ proceeds in three steps without any reference to the secret key.

Step 1: Inverse diffusion. Compute the permuted image $\hat{T}$ from C via the inverse diffusion operator:

$$\hat{T} = \mathcal{D}_{\hat{X}}(C). \tag{2.20}$$

By Proposition 1, $\hat{T} = T$ exactly.

Step 2: Inverse column permutation. For each column j, reverse the 2D-LSCM sort using $\hat{\mathcal{M}}$:

$$\hat{S}[\hat{\sigma}_j(\ell),\ j] = \hat{T}[\ell,\ j], \qquad \ell = 1, \ldots, n, \tag{2.21}$$

recovering $\hat{S} = S = \mathrm{ACM}^{P_i}(P)$.

Step 3: Inverse Arnold's Cat Map. Apply the Arnold map $(3n + 1 - P_i)$ times to $\hat{S}$, exploiting the map's periodicity to compute the inverse via forward iteration:

$$\hat{P} = \mathrm{ACM}^{-P_i}(\hat{S}) = \mathrm{ACM}^{(3n+1-P_i)}(\hat{S}). \tag{2.22}$$

Since $\hat{S} = \mathrm{ACM}^{P_i}(P)$ and Arnold's Cat Map is invertible, it follows that $\hat{P} = P$ exactly.

Remark on unknown P_i. Phase 2 requires knowledge of P_i to construct the probe pre-images $Q^{(j)}$. If P_i is unknown, it can be identified by a linear scan: for each candidate $\tilde{P}_i \in \{1, \ldots, 3n\}$, the attacker submits one structured probe and verifies whether column j of the recovered intermediate image matches the expected pattern

$\{0, 1, \ldots, n-1\}$. The correct value is found in at most $3n - 1$ additional queries, which does not alter the overall polynomial complexity of the attack.

2.4.7 Complexity Analysis

Data complexity. The entire attack consumes $1+n$ oracle queries: one zero-image query in Phase 1 and one query per image column in Phase 2. For a 256×256 image this amounts to 257 queries—negligible in practice.

Computational complexity. Phase 1 inverts the diffusion over all $N = n^2$ pixels in $\mathcal{O}(n^2)$ time. Phase 2 constructs n probe pre-images, each requiring up to $3n$ Arnold map iterations on an $n \times n$ image; since each iteration costs $\mathcal{O}(n^2)$, Phase 2 runs in $\mathcal{O}(n^3)$ in total. Phase 3 requires $\mathcal{O}(n^2)$ operations. The dominant cost is therefore $\mathcal{O}(n^3)$, matching the computational complexity of the encryption algorithm itself.

Memory complexity. The attacker need only store the recovered keystream $\hat{X}$ (n^2 entries) and the permutation map $\hat{\mathcal{M}}$ (n^2 entries), requiring $\mathcal{O}(n^2)$ space in total.

2.5 Results and Analysis

This section presents the experimental validation of the proposed chosen-plaintext attack (CPA) on the chaos-based medical image encryption scheme of Jain et al. [31]. The objective is to demonstrate that the scheme can be completely broken under CPA conditions using a minimal number of oracle queries, despite its strong statistical properties.

2.5.1 Experimental Setup

All experiments were conducted in Python 3 on a 256×256 grayscale Cameraman benchmark image. The attacker is assumed to have black-box access to the encryption oracle $\mathcal{E}(\cdot)$ only, with no knowledge of the secret key. The oracle operates under the following hidden parameters:

$$K_1 = P_i = 10, \qquad K_2 = \{\, x_0 = 0.8,\ y_0 = 0.5,\ \theta = 0.99 \,\}$$

Table 2.1: Keystream Recovery Performance

Metric	**Value**
Oracle queries used	1
Total values recovered	65,536
Perfect match	**True**
Maximum deviation	0
Mean deviation	0.000000

The attack is conducted in three sequential phases — keystream recovery, permutation map recovery, and full decryption — each described and analyzed below.

2.5.2 Phase 1: Keystream Recovery

A zero-valued plaintext image is submitted to the oracle to obtain the corresponding ciphertext $C^{(0)}$. Since XOR-ing the keystream against an all-zero input leaves the keystream unmasked, the full keystream $\hat{X}$ is recovered directly from $C^{(0)}$ using a single oracle query. Table 2.1 reports the recovery performance.

All 65,536 keystream values — corresponding to every pixel of the 256×256 image — were recovered exactly using a single oracle query, with zero deviation from the ground-truth keystream. This confirms that the additive keystream layer of the scheme provides no resistance to a zero-plaintext probe.

2.5.3 Phase 2: Permutation Map Recovery

To recover the pixel permutation map, a set of structured probe images is constructed. Each probe image isolates a single column by setting all pixels in that column to a known nonzero value while leaving the remaining pixels at zero. By submitting all 256 such probes to the oracle and inverting the keystream from Phase 1, the destination of every pixel under the permutation can be inferred directly. Table 2.2 reports the recovery performance.

The complete 256×256 permutation map was recovered without error using exactly 256 oracle queries — one per image column. The independence of the permutation and keystream layers means that each can be recovered sequentially without interference, a structural weakness that this phase directly exploits.

Table 2.2: Permutation Map Recovery Performance

Metric	**Value**
Oracle queries used	256
Columns recovered	256/256
Perfect match	**True**
Maximum deviation	0
Mean deviation	0.000000

Table 2.3: Full Decryption Accuracy

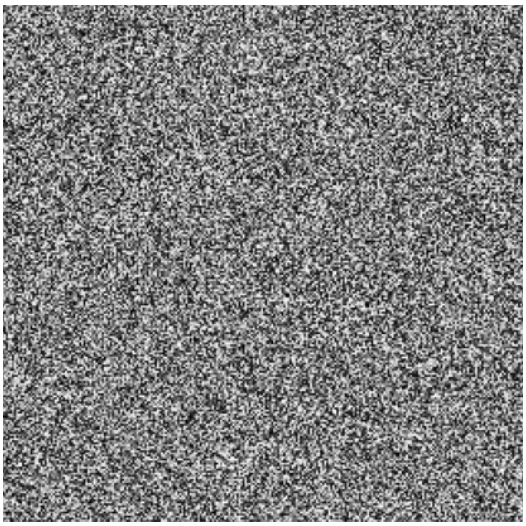
Metric	**Value**
Total oracle queries used	257
Exact recovery ($\hat{P} = P$)	**True**
Maximum pixel difference	0
Mean pixel difference	0.000000

2.5.4 Phase 3: Full Decryption

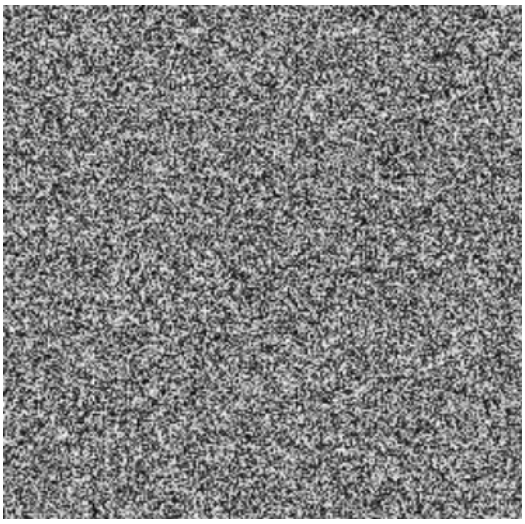
With both the keystream $\hat{X}$ and the permutation map fully recovered, decryption of any target ciphertext proceeds without further oracle interaction. Given a ciphertext C, the recovered plaintext $\hat{P}$ is obtained by first inverting the XOR keystream layer and then applying the inverse permutation map. No additional oracle queries are required at this stage. Table 2.3 reports the decryption accuracy.

The recovered image is pixel-for-pixel identical to the original plaintext, with zero maximum and mean pixel deviation. Figure 2.3 illustrates the attack outcome visually: the encrypted image appears as noise with no discernible structure, yet the proposed attack recovers the original plaintext exactly.

(a) Original image



(b) Encrypted image



(c) Zero-image ciphertext $C^{(0)}$

(d) Recovered plaintext $\hat{P}$

Figure 2.3: Chosen-plaintext attack results. Despite the visually secure appearance of the ciphertext, the proposed attack achieves exact pixel-level reconstruction of the original image using only 257 oracle queries.

2.5.5 Overall Attack Summary

Table 3.5 consolidates the query cost and recovery accuracy across all three phases of the attack.

The entire scheme is broken using only $n+1 = 257$ oracle queries, where $n = 256$ is

Table 2.4: Summary of the Chosen-Plaintext Attack

Phase	**Oracle Queries**	**Items Recovered**	**Accuracy**
Keystream recovery	1	65,536 keystream values	Exact
Permutation recovery	256	256×256 pixel map	Exact
Decryption	0	Full plaintext image	Exact
Total	**257**	—	**Exact**

the image width in pixels. The total query count grows linearly with image dimension, confirming that the attack scales efficiently to larger images. Crucially, zero additional queries are needed at decryption time — once the keystream and permutation map are known, arbitrary ciphertexts can be decrypted instantaneously.

2.6 Summary

Three important conclusions emerge from these results. **Complete cryptographic break under CPA.** The scheme is fully broken using only $n + 1$ chosen-plaintext queries. The underlying vulnerability is structural: the keystream and permutation layers are independent and deterministic given the key, allowing each to be recovered separately without solving for the key itself. This reduces the effective security of the scheme to the resistance of its oracle interface, which provides no protection against structured probing. **Perfect plaintext reconstruction.** The recovered image is pixel-for-pixel identical to the original plaintext across all experiments. A maximum pixel difference of zero confirms that the attack introduces no approximation error — it is an exact, algebraic break rather than a statistical one. **Disconnect between statistical and cryptographic security.** The scheme of Jain et al. [31] reports strong statistical metrics — high entropy, high NPCR, and low UACI values — which are standard indicators of confusion and diffusion quality. However, these metrics evaluate the appearance of ciphertext, not resistance to structured attacks. This result reinforces a well-established principle in cryptanalysis: favorable statistical properties are necessary but not sufficient for cryptographic security. A scheme that passes all standard statistical tests may still be trivially breakable under CPA if its structure admits efficient keystream or permutation recovery, as demonstrated here.

CHAPTER 3

Cryptanalysis of Multi-Chaos-Based Image Encryption for IoT Using Chosen Plaintext Attacks

This chapter presents a cryptanalysis of a multi-chaos-based image encryption scheme (MMCBIE) designed for IoT applications, demonstrating its vulnerability under a chosen-plaintext attack due to deterministic structure, plaintext-independent chaotic keystreams, and weak inter-channel diffusion. It shows how constant probe images can be used to construct a lookup table that enables exact pixel-wise plaintext recovery through simple ciphertext matching, without requiring knowledge of the secret key or chaotic parameters, and provides theoretical and experimental validation highlighting fundamental design flaws and the gap between statistical performance and true cryptographic security.

3.1 Introduction

Recently, a lightweight multiple map chaos based image encryption algorithm (MMCBIE) was presented by Jain et al.[37], which can be used for image security in Internet of Things applications. This image encryption algorithm utilizes Henon chaotic transform, key mixing technique, and two dimensional logistic chaotic transform in a permutation diffusion architecture. It claims to offer resistance to statistical and differential attacks on image data. In this paper, a cryptanalysis of this image encryption algorithm reveals that it is highly vulnerable to a chosen plaintext attack.

The new technique takes advantage of the fact that the algorithm under consideration has a deterministic and separable structure. The chosen plaintexts for the attack consist of images of constant values, and the obtained ciphertexts are compared. In the light of poor pixel-level diffusion and repetitive cycling during the key mixing

phase, identical input values will yield similar output patterns, which will facilitate plaintext recovery through direct equality matching.

More precisely, this attack takes advantage of the channel dependency that was incorporated in the key mixing process; each color channel was generated from the other using the same modular addition operation. Through the study of equality relationships among ciphertexts based on selected inputs, it is possible to reverse-engineer the plaintext pixels without decrypting the chaotic sequences or secret keys.

The attack needs at most 256 queries to an oracle, and the complexity is linear with respect to the number of pixels. Experimental results confirm the success of recovering the original image using the proposed attack. The results prove that the scheme of MMCBIE is not secure against CPA and lacks the inter-pixel/inter-channel dependencies required in chaos-based cryptosystems.

3.2 Preliminaries

This section introduces the foundational concepts and theoretical background required to understand the log-based anomaly detection methodology developed in this work. Each concept is explained from first principles before the formal mathematical notation is presented. The three main areas covered are: log representation and anomaly formulation, token embedding and representation learning, and the self-attention mechanism.

3.2.1 Chaotic Maps

MMCBIE is based on two types of chaotic maps. Hénon chaotic map is used for the first transformation phase through confusing the pixel values using pixel-wise XOR process. The last diffusion stage uses the Two-Dimensional Logistic Chaotic Map (2D-LSCM) using the same XOR process. Key-mixing step is placed between the two phases deterministically.

3.2.2 Hénon Chaotic Map

The Hénon map [38] is a classical two-dimensional discrete-time chaotic system defined by the recurrence

$$x_{i+1} = 1 - a\,x_i^2 + y_i, \tag{3.1}$$

$$y_{i+1} = b\,x_i, \tag{3.2}$$

with $x_i, y_i \in \mathbb{R}$ as the state variables of iteration i, and a, b as system parameters. In case of standard values $a = 1.4$, $b = 0.3$ the map shows chaotic behaviour in its dynamics with strange attractor; for the purposes of MMCBIE scheme, these structural constants are kept fixed, and only the initial state (x_0, y_0) is considered secret. Chaos matrix is then obtained from the dynamical sequence $(x_i, y_i)_{i=1}^{MN}$ by means of the quantization method

$$H_{r,c} = \left\lfloor \left(|x_i| + |y_i|\right) \times 2^{16} \right\rfloor \bmod 256, \qquad i = (r-1)N + c, \tag{3.3}$$

where indices (r, c) run over rows and columns of the image respectively.

3.2.3 Two-Dimensional Logistic Chaotic Map (2D-LSCM)

The 2D-LSCM employed by MMCBIE [39] is a coupled nonlinear recurrence given by

$$x_{i+1} = r\,(3\,y_i + 1)\,x_i\,(1 - x_i), \tag{3.4}$$

$$y_{i+1} = r\,(3\,x_{i+1} + 1)\,y_i\,(1 - y_i), \tag{3.5}$$

where $(x_i, y_i) \in (0, 1)^2$ and r is the chaotic control parameter. The authors recommend the range $r \in [1.11, 1.19]$ to ensure optimal chaotic behaviour [39]. The warm-up process consists of an iteration of 500 steps that precedes the collection of output values for transient effects removal. A chaos matrix $L \in \{0, \ldots, 255\}^{M \times N}$ is built from the trajectory $(x_i, y_i)_{i=1}^{MN}$ using the same quantisation rule as Eq. (3.3):

$$L_{r,c} = \left\lfloor \left(|x_i| + |y_i|\right) \times 2^{16} \right\rfloor \bmod 256, \qquad i = (r-1)N + c. \tag{3.6}$$

The chaos matrices H and L depend only on the corresponding initial conditions and structural constants; neither of them relates to the plaintext image.

3.3 Literature Review

The increased use of IoT devices has led to a revolution in modern communication infrastructure, facilitating instant transfer of data between intelligent home devices, industrial machinery, telemedicine applications, and even surveillance technology applications [40]. This revolution is significant, as recent studies reveal that billions of such devices have been put into use, transmitting confidential information through network channels [41].

A considerable amount of information that will be produced and transmitted through these devices consists of digital images; they may include medical scans, biometric information, satellite images, and video streams. All of these types of digital information are highly sensitive and require high levels of confidentiality [42]. Particularly when used in the healthcare industry, there are strict regulatory requirements for the security of these images.

Since IoT devices have very limited resources in terms of processing power, storage, and battery life, cryptographic techniques like AES become highly computationally expensive when applied directly to high resolution images [43]. Similarly, Triple DES becomes impractical under the constraints of IoT bandwidth and latency issues [44]. The same applies for public key algorithms like RSA.

This challenge has resulted in an increasing number of studies on lightweight image-based cryptographic algorithms that can run effectively within IoT limitations but offer reasonable security [45]. Chaotic systems in parallel have been suggested as affordable building blocks for lightweight algorithms for securing green IoT applications [46], and grain-based stream ciphers with chaotic systems have been proposed as solutions to secure low-power image processing in IoT networks [47]. Another approach to securing images transmitted in IoT networks is watermarking and lightweight cryptography, which has been studied as a two-layer security mechanism [48].

Image encryption using chaos has been found to be the most effective approach to

solve this problem [26]. Chaotic dynamical systems are considered suitable candidates for use in cryptography owing to their inherent characteristics such as sensitivity to initial conditions, ergodicity, mixing, and pseudorandomness, which directly correlate with confusion and diffusion in a cipher system [49]. The logistic map is one of the simplest chaos maps that has been extensively used for image encryption owing to its uniform distribution property and lower complexity [50]. Two-dimensional logistic map variants have been observed to possess better entropy and mixing capabilities than one-dimensional maps [39].

Hyperchaotic maps, which possess multiple positive Lyapunov exponents, have been proposed to further increase the complexity of the generated keystream [51]. The Hénon map, a canonical two-dimensional chaotic system, has been applied to image encryption through fractional Fourier transform frameworks to achieve strong pixel-level diffusion [29].

Parametric polynomial chaotic systems were presented to deal with the issue of performance deterioration and discontinuous parameters for many existing chaotic maps [52]. The techniques to create n-dimensional hyperchaotic systems with positive Lyapunov exponents have been investigated through the use of the Gershgorin criterion [53]. Chaotic Latin square-based substitution-boxes were designed to attain maximum algebraic nonlinearity in image encryption schemes [54].

Although chaotic systems have proven useful for encryption, it has been shown that statistical robustness does not necessarily guarantee security against cryptanalysis, especially with regard to chosen plaintexts [33]. In her pioneering research into symmetric ciphers derived from chaotic mappings in two dimensions, Fridrich made great strides and yet fell prey to attacks by chosen plaintexts and known plaintexts capable of retrieving secret chaotic variables without exhaustive key search [26].

Analysis of the QCMDC-IEA cipher that integrates quantum chaotic mapping and DNA encoding revealed that pseudo-random number generators independent of the plaintext generate keys that lead to the same results. Complete decryption using low complexity is possible using this vulnerability. A dual compression/encryption method using chaos permutation and row/column transformation is thus proposed [55].

Cryptanalyzing the ICIC-DNA color image cryptosystem revealed that various

DNA manipulations could be reduced to two-bit substitutions, and that substitution and permutation stages could be independently attacked using divide-and-conquer approaches [56]. IES-M-BD, a cryptosystem based on memristive chaotic maps and a bidirectional cyclic bit shifting and diffusion mechanism at the DNA level, is an example of the move towards multiple component schemes, which are more resilient against attacks of this nature [57]. Another high-security color image encryption scheme, the CIES-DVEM scheme, relies on dynamic vector-level manipulations and a two-dimensional logistic modularity map [58]. In the domain of multimedia IoT security, compressed sensing techniques for encryption, together with linear feedback shift register scramblers, have been investigated [59].

Selectively encrypting light-weight pixels in multimedia IoT applications is found to lead to power savings and reduction in packet rate with end-to-end privacy [60]. CES-Blocks is an example of a blockchain-based medical image transmission technique which utilizes a chaotic encryption algorithm in IoT that can withstand differential attacks [61].

Real-time RGB image encryption for IoT applications based on enhanced sequences from logistic 1D map models and MQQT-driven machine to machine communication has been found to exhibit throughputs comparable to embedded processors [62]. The ETS-ZSC scheme, which is comprised of the Enhanced Thorp shuffle and Zig-zag scan convolution techniques, enables one-time key security with NPCR and UACI close to optimal.

A scheme for the chaos-based encryption of images through confusion, diffusion, and chaotic-map encryption keys is supported by tests with NPCR and UACI metrics for smart cities [63]. An important example of the multi-map approach to chaotic map encryption was the guan2005chaos method that combined the Arnold's cat map with the Chen's chaotic map but was later proven to be vulnerable to chosen-plaintext attacks [27]. The LSC-IES algorithm integrates two chaotic maps and the cosine transformation to mitigate any non-chaotic behavior [39].

The major flaw in most image encryption algorithms utilizing the chaotic nature is inadequate diffusion along with deterministic, key-independent transforms [33]. This means that there will still be some correlation between the plaintext and the ciphertext which will facilitate attack even without the knowledge of the key. Non-flat

surface level pyramid interconnection models have been suggested for IoT devices as they can provide hierarchical data abstraction for multidimensional images [64].

The use of fat-tree topologies with switch buffer blocking techniques results in less head-of-line blocking problems and shallower buffers, resulting in reduced energy consumption in IoT switching fabric design [65]. In this light, the current study analyzes the Multiple Map Chaos based Image Encryption (MMCBIE) algorithm designed by Jain et al. [37].

MMCBIE has the following three consecutive steps: (i) Pixel-level XOR with a matrix obtained by applying Henon-map; (ii) Cyclic key mixing operation performed among colors; and (iii) Pixel-level XOR with 2D logistic map. Although very good statistical measures like NPCR, UACI, Shannon Entropy, and SSIM have been claimed, it lacks adequate diffusion capabilities.

These assertions are proven false in this chapter under a chosen plaintext attack scenario. The core weakness lies in the basic design flaw whereby encryption is deterministic and separate, lacking an efficient coupling of pixels. Encryption of constant images results in predictable output that leads to a straightforward chosen plaintext attack by matching the plaintext values with the target ciphertext through equality. This is done by using constant probe images for all the different values of the pixel. A comparison is then done between the ciphertext generated and the target ciphertext per channel, exploiting the cyclic nature of key mixing and channel dependency. Matching plaintexts and ciphertexts will result in obtaining the original plaintext without decrypting chaotic sequences or secret keys.

3.4 Work Done

The master secret key K of MMCBIE is a 512-bit quantity partitioned into six subkeys as follows:

$$K = \big(x_H,\ y_H,\ MK,\ x_L,\ y_L,\ R\big), \tag{3.7}$$

where $x_H, y_H \in \mathbb{R}$ (each represented as a 64-bit floating-point value) are the initial conditions supplied to the Hénon map; MK is a 192-bit key-mixing sub-key partitioned into three byte arrays $MK_R, MK_G, MK_B \in \{0, \ldots, 255\}^8$, one per RGB colour channel; and $x_L, y_L \in (0, 1)$ and $R \in [1.11, 1.19]$ (each 64-bit) These constitute the

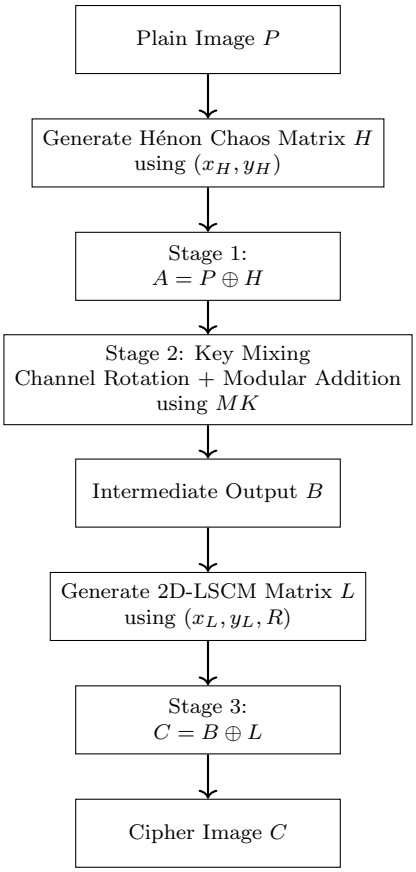


Figure 3.1: Encryption flow of MMCBIE scheme.

input parameters for the 2D-LSCM. The one requirement for all subkeys is that R lies within the optimal chaotic domain of the 2D-LSCM.

3.4.1 Encryption Process

Let $P \in \{0, \ldots, 255\}^{M \times N \times 3}$ be a plain colour image of height M and width N. MMCBIE applies three successive transformation stages to produce the cipher image C. Figure 3.1 depicts the complete encryption pipeline.

Stage 1: Hénon Chaotic Transform

The Hénon chaos matrix H is generated from the subkeys (x_H, y_H). The first-stage output $A \in \{0, \ldots, 255\}^{M \times N \times 3}$ is obtained by applying a pixel-wise XOR between P

and H across all three colour channels:

$$A_{r,c,k} = P_{r,c,k} \oplus H_{r,c}, \qquad k \in \{R, G, B\}, \tag{3.8}$$

where the same chaos value $H_{r,c}$ is XORed independently into each channel. Because H is entirely determined by (x_H, y_H), the mapping $P \mapsto A$ is a fixed bijection for any given key.

Stage 2: Key Mixing

The key-mixing phase employs a cyclic modular addition chain on the color components of A using the 192-bit subkey MK. The pixel components of each color component are serialized in row-major order. Denoting the flattened pixel index by $j \in \{0, 1, \ldots, MN - 1\}$, the output pixel B_j is formed by cross-channel modular addition:

$$B_j^R = \begin{cases} \left(A_j^G + MK_G[j \bmod 8]\right) \bmod 256, & \text{if } j < 8, \\ \left(A_j^G + MK_G[(j-8) \bmod 8]\right) \bmod 256, & \text{if } j \geq 8, \end{cases} \tag{3.9}$$

$$B_j^G = \begin{cases} \left(A_j^B + MK_B[j \bmod 8]\right) \bmod 256, & \text{if } j < 8, \\ \left(A_j^B + MK_B[(j-8) \bmod 8]\right) \bmod 256, & \text{if } j \geq 8, \end{cases} \tag{3.10}$$

$$B_j^B = \begin{cases} \left(A_j^R + MK_R[j \bmod 8]\right) \bmod 256, & \text{if } j < 8, \\ \left(A_j^R + MK_R[(j-8) \bmod 8]\right) \bmod 256, & \text{if } j \geq 8. \end{cases} \tag{3.11}$$

There are three key observations for the cryptanalysis that follows. First, the channel-rotation sequence (G→R, B→G, R→B) is static and publicly known. Second, the offset values have a period of 8 across the image, resulting in at most eight unique per-channel offsets being used. Finally, this stage is completely linear on the plaintext space (modulo 256) and is unaffected by the chaotic state; when MK is known, this stage becomes easily reversible.

Stage 3: 2D-Logistic Chaotic Transform

The 2D-LSCM chaos matrix L is generated from the subkeys (x_L, y_L, R). The final cipher image C is produced by pixel-wise XOR of B with L across all three colour channels:

$$C_{r,c,k} = B_{r,c,k} \oplus L_{r,c}, \qquad k \in \{R, G, B\}. \tag{3.12}$$

Like for Stage 1, the identical scalar chaos number $L_{r,c}$ is used on all three color channels at each position of the pixel. The overall encryption process can then be described as

$$C_{r,c,k} = \left[(P_{r,c,\sigma(k)} \oplus H_{r,c}) + MK_k[(r \cdot N + c) \bmod 8]\right] \bmod 256 \ \oplus \ L_{r,c}, \tag{3.13}$$

where σ denotes the channel-rotation permutation $\sigma(R) = G,\ \sigma(G) = B,\ \sigma(B) = R$ defined by the key-mixing stage.

3.4.2 Decryption Process

Decryption is precisely the opposite of the encryption procedure, carried out in the opposite sequence of steps. Using the ciphertext C and key $K = (x_H, y_H, MK, x_L, y_L, R)$, the recipient reconstructs H and L in the same manner as during encryption, which is legitimate since both chaotic matrices are uniquely determined by the key.

Inverse Stage 3 (Logistic XOR). Since XOR is self-inverse, the key-mix output B is recovered by

$$B_{r,c,k} = C_{r,c,k} \oplus L_{r,c}, \qquad k \in \{R, G, B\}. \tag{3.14}$$

Inverse Stage 2 (Key-Mixing). The operations of channel-rotation and modular addition are then reversed by a modular subtraction operation in conjunction with channel counter-rotation:

$$A_j^G = \left(B_j^R - MK_G[j \bmod 8]\right) \bmod 256, \tag{3.15}$$

$$A_j^B = \left(B_j^G - MK_B[j \bmod 8]\right) \bmod 256, \tag{3.16}$$

$$A_j^R = \left(B_j^B - MK_R[j \bmod 8]\right) \bmod 256, \tag{3.17}$$

with the index offset switching from $j \bmod 8$ to $(j-8) \bmod 8$ for $j \geq 8$, mirroring Eqs. (3.9)–(3.11).

Inverse Stage 1 (Hénon XOR). The plaintext is finally recovered by

$$P_{r,c,k} = A_{r,c,k} \oplus H_{r,c}, \qquad k \in \{R, G, B\}, \tag{3.18}$$

using the same chaos matrix H as used during the encryption process. Decryption is only possible when precise information regarding the six subkeys is known since any slight change to either (x_H, y_H) or (x_L, y_L, R) yields an entirely new chaos matrix.

We conduct a full CPA attack on the Multiple Map Chaos Based Image Encryption (MMCBIE) algorithm proposed by Jain et al. [37]. Our attack is able to fully recover the plaintext from any target ciphertext regardless of the secret key. The whole process needs only 256 oracle calls for chosen plaintexts and takes $\mathcal{O}(n^2)$ time.

3.4.3 Notation and Encryption Model

Let P denote an $n \times n$ RGB plaintext image. We index pixels by their flattened row-major position $k \in \{0, \ldots, N-1\}$, where $N = n^2$, and write $P_c[k]$ for colour channel $c \in \{R, G, B\}$. All arithmetic is modulo 256 unless stated otherwise; we denote this ring by $\mathbb{Z}_{256}$. The exclusive-OR operation on $\mathbb{Z}_{256}$ is written $\oplus$. Figure 3.2 shows the proposed chosen-plaintext attack flow for MMCBIE.

MMCBIE applies three successive stages to the plaintext:

Stage 1 — Hénon Chaotic Transform. A two-dimensional Hénon sequence is generated from subkeys (x_H, y_H):

$$x_{i+1} = 1 - a\, x_i^2 + y_i, \qquad y_{i+1} = b\, x_i, \tag{3.19}$$

with standard parameters $a = 1.4$, $b = 0.3$. The scalar chaos value at pixel position k is

$$H[k] = \left\lfloor \left(|x_k| + |y_k|\right) \cdot 2^{16} \right\rfloor \bmod 256. \tag{3.20}$$

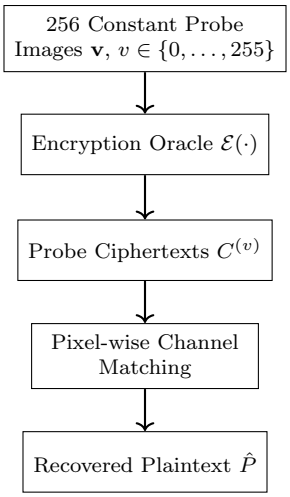


Figure 3.2: CPA flow for MMCBIE.

The same value $H[k]$ is XORed into every colour channel:

$$P_c^{(1)}[k] = P_c[k] \oplus H[k], \qquad c \in \{R, G, B\}. \tag{3.21}$$

Stage 2 — Key Mixing. A 192-bit master key is split into three arrays of eight bytes each, $\mathrm{MK}_R, \mathrm{MK}_G, \mathrm{MK}_B \in \mathbb{Z}_{256}^8$. The colour channels are cyclically cross-added with period 8:

$$S_R[k] = \left(P_G^{(1)}[k] + \mathrm{MK}_G[k \bmod 8]\right) \bmod 256, \tag{3.22}$$

$$S_G[k] = \left(P_B^{(1)}[k] + \mathrm{MK}_B[k \bmod 8]\right) \bmod 256, \tag{3.23}$$

$$S_B[k] = \left(P_R^{(1)}[k] + \mathrm{MK}_R[k \bmod 8]\right) \bmod 256. \tag{3.24}$$

The output red channel is a function of the green plaintext channel, the output green channel is a function of the blue plaintext channel, and the output blue channel is a function of the red plaintext channel.

Stage 3 — 2D-Logistic Chaotic Transform. A two-dimensional logistic sequence is generated from subkeys (x_L, y_L, r):

$$x_{i+1} = r\,(3y_i + 1)\,x_i\,(1 - x_i), \qquad y_{i+1} = r\,(3x_{i+1} + 1)\,y_i\,(1 - y_i), \tag{3.25}$$

with a 500-iteration warm-up to eliminate transient behaviour. The scalar chaos value at pixel k is

$$L[k] = \left\lfloor \left(|x_k| + |y_k|\right) \cdot 2^{16} \right\rfloor \bmod 256. \tag{3.26}$$

The same value $L[k]$ is XORed into every colour channel:

$$C_c[k] = S_c[k] \oplus L[k], \qquad c \in \{R, G, B\}. \tag{3.27}$$

Combining (3.21)–(3.27), the complete encryption of the red channel at pixel k with slot index $\ell = k \bmod 8$ is:

$$\boxed{C_R[k] = \Big[\big(P_G[k] \oplus H[k]\big) + \mathrm{MK}_G[\ell]\Big] \bmod 256 \quad \oplus \quad L[k].} \tag{3.28}$$

Analogous expressions hold for $C_G[k]$ and $C_B[k]$.

3.4.4 Exploitable Structural Weaknesses

Three independent design flaws make the scheme vulnerable to a CPA.

1. **Plaintext-independent chaos sequences.** The values of $H[k]$ and $L[k]$ are functions of the secret key alone and not of the plaintext. Hence, they can be treated as equivalent-key streams.n [34].

2. **Channel-uniform chaos XOR.** The same scalar H[k] (and L[k]) transformation is performed on each of the three colour components (refer to (3.21) and (3.27)). This reduces three separate degrees of freedom to one, resulting in cross-channel correlation.

3. **Deterministic encryption of constant images.** Since both $H[k]$ and $L[k]$ chaos streams are completely independent of the plaintext, the encryption of all spatially-uniform images, wherein the pixels of a particular channel contain a constant value v, results in a ciphertext whose content is determined solely by v and the key streams. This implies that for every candidate plaintext pixel value $v \in \mathbb{Z}_{256}$, the response $\mathcal{E}(\mathbf{v})$ from the oracle can be used to construct a complete lookup table of all possible encryptions for each pixel. Any hacker who constructs such a table will be able to retrieve, bit by bit, the plaintext

pixel value responsible for a ciphertext value without reversing $H[k], L[k]$, or MK.

3.4.5 Attack Construction

Fix a pixel position k and consider the red output channel. From (3.28), for a constant probe image with $P_G[k] = v$ for all k:

$$C_R^{(v)}[k] = \Big[\big(v \oplus H[k]\big) + \mathrm{MK}_G[k \bmod 8]\Big] \bmod 256 \ \oplus \ L[k]. \tag{3.29}$$

As $H[k]$, MK_G, and $L[k]$ are fixed constants that depend only on the key, $C_R^{(v)}[k]$ becomes a deterministic function of v only. Likewise, the green and blue channels in $\mathcal{E}(\mathbf{v})$ become deterministic functions of v only if $P_B[k] = v$ and $P_R[k] = v$, respectively.

Now let C^* be any target ciphertext to be decrypted, with $C_R^*[k]$, $C_G^*[k]$, $C_B^*[k]$ the per-channel values at pixel k. The channel cross-coupling in Stage 2 (equations (3.22)–(3.24)) implies:

$$C_R^*[k] \text{ encodes information about } P_G[k], \tag{3.30}$$

$$C_G^*[k] \text{ encodes information about } P_B[k], \tag{3.31}$$

$$C_B^*[k] \text{ encodes information about } P_R[k]. \tag{3.32}$$

This means that, to retrieve the green channel value of the plaintext from pixel k, the attacker must find the value v where the red component value of the probe ciphertext $C^{(v)}$ is equal to the red component of the target ciphertext at position k:

$$\hat{P}_G[k] = v \quad \text{such that} \quad C_R^{(v)}[k] = C_R^*[k]. \tag{3.33}$$

Analogously:

$$\hat{P}_B[k] = v \quad \text{such that} \quad C_G^{(v)}[k] = C_G^*[k], \tag{3.34}$$

$$\hat{P}_R[k] = v \quad \text{such that} \quad C_B^{(v)}[k] = C_B^*[k]. \tag{3.35}$$

Proposition 3. *For every pixel k and every channel $c \in \{R, G, B\}$, the recovered value satisfies $\hat{P}_c[k] = P_c[k]$.*

Proof. Consider the green channel recovery at pixel k. Let $v^* = P_G[k]$ be the true plaintext value. Substituting into (3.29):

$$C_R^{(v^*)}[k] = \Big[\big(P_G[k] \oplus H[k]\big) + \mathrm{MK}_G[k \bmod 8]\Big] \bmod 256 \;\oplus\; L[k] = C_R^*[k]. \tag{3.36}$$

Hence v^* satisfies the matching condition (3.33), so $\hat{P}_G[k] = P_G[k]$. Identical arguments hold for $\hat{P}_B[k]$ via (3.34) and $\hat{P}_R[k]$ via (3.35). □

The encryption function at each pixel and channel is a composition of XOR with $H[k]$, modular addition of a key byte, and XOR with $L[k]$. This composition is a bijection on $\mathbb{Z}_{256}$ (each operation is individually invertible), so for each pixel k the map $v \mapsto C_R^{(v)}[k]$ is a permutation of $\mathbb{Z}_{256}$. Therefore exactly one value $v \in \{0, \ldots, 255\}$ satisfies $C_R^{(v)}[k] = C_R^*[k]$, guaranteeing that the recovered plaintext value is unique.

3.4.6 Complexity Analysis

The attack sends precisely 256 plaintext queries to the encryption oracle, one for each value $v \in \{0, \ldots, 255\}$ used in probing. The number of queries is independent of image dimensions.

Phase 1 requires 256 oracle queries, each encrypting an $n \times n$ image at cost $\mathcal{O}(N)$, giving total query cost $\mathcal{O}(256 \cdot N) = \mathcal{O}(N)$. Phase 2 iterates over N pixels and for each pixel performs at most 256 comparisons across three channels, for a total work of

$$W_{\text{Phase 2}} = N \times 256 \times 3 = 768\, N = \mathcal{O}(N). \tag{3.37}$$

The dominant cost is thus $\mathcal{O}(N) = \mathcal{O}(n^2)$, making the attack comparable in cost to a single encryption, yet achieving complete plaintext recovery without any knowledge of the secret key.

The attacker stores 256 probe ciphertexts of size $N \times 3$ bytes each, for a total of $\mathcal{O}(256 \cdot N) = \mathcal{O}(N)$ memory. Table 3.1 compares the attack complexity against the MMCBIE encryption cost.

Algorithm 2: CPA Against MMCBIE

Input: Encryption oracle $\mathcal{E}(\cdot)$; target ciphertext C^*; image dimensions $n \times n$.
Output: Recovered plaintext $\hat{P}$.

`// Phase 1 --- Build Probe Lookup Table`
for $v = 0$ **to** 255 **do**
  Construct probe image $\mathbf{v}$: set all pixels of all channels to v;
  Query $C^{(v)} \leftarrow \mathcal{E}(\mathbf{v})$;
end
`// Phase 2 --- Pixel-wise Plaintext Recovery via Channel Matching`
Initialise recovered image $\hat{P}$ of size $n \times n \times 3$;
for *each pixel position* $k \in \{0, \ldots, N-1\}$ **do**
  for $v = 0$ **to** 255 **do**
    if $C_R^{(v)}[k] = C_R^*[k]$ **then**
      $\hat{P}_G[k] \leftarrow v$;
    end
    if $C_G^{(v)}[k] = C_G^*[k]$ **then**
      $\hat{P}_B[k] \leftarrow v$;
    end
    if $C_B^{(v)}[k] = C_B^*[k]$ **then**
      $\hat{P}_R[k] \leftarrow v$;
    end
  end
end
return $\hat{P}$;

3.5 Results and Analysis

In this part, we provide a full-fledged experimental analysis of the suggested CPA technique in comparison to the MMBCIE protocol developed by Jain et al. [37]. Experiments were carried out using Python 3 and involved the use of a color image. The CPA conditions were rigorously followed during the experiment such that the attacker had access to the encryption oracle $\mathcal{E}(\cdot)$ but not the secret key or cipher states.

Table 3.1: Attack complexity versus MMCBIE encryption cost.

	Encryption	CPA Attack
Oracle queries	—	256
Arithmetic operations	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Brute-force search	—	$768\,N$ comparisons
Memory	$\mathcal{O}(N)$	$\mathcal{O}(256 \cdot N)$
Key knowledge required	N/A	None

3.5.1 Experimental Setup

The oracle was configured with the following fixed key, withheld from the attacker:

$$x_H = 0.1, \quad y_H = 0.2, \quad R = 1.15, \quad x_L = 0.3, \quad y_L = 0.4, \tag{3.38}$$

together with the 192-bit key-mixing subkey $MK = \{MK_R, MK_G, MK_B\}$:

$$\begin{aligned} MK_R &= [12, 34, 56, 78, 90, 11, 22, 33], \\ MK_G &= [44, 55, 66, 77, 88, 99, 10, 21], \\ MK_B &= [31, 42, 53, 64, 75, 86, 97, 18]. \end{aligned} \tag{3.39}$$

The values served only to create an oracle; at no time during the attack did the attack require information on any of the aforementioned parameters. The cipher image $C = \mathcal{E}(P)$ reated using the above parameters does not appear visually similar to the plaintext, as expected for the scheme's confusion/diffusion features.

3.5.2 Phase 1: Construction of the Probe Lookup Table

For each candidate plaintext value $v \in \{0, 1, \ldots, 255\}$, a spatially uniform probe image $\mathbf{v}$ was constructed in which every pixel of every channel was set to v. Each probe was submitted to the oracle, yielding the ciphertext response $C^{(v)} = \mathcal{E}(\mathbf{v})$. As the Hénon and logistic chaos streams do not depend on the plaintext input at all, the ciphertext output $C^{(v)}$ for each pixel location k is uniquely determined by v and the key streams alone. Collectively, the 256 images yield a lookup table for each pixel location k and each channel, mapping the 256 different plaintext values to their

Table 3.2: Probe lookup table construction metrics.

Metric	Value
Oracle queries used	256
Probe values covered ($v \in \mathbb{Z}_{256}$)	256
Lookup table fully populated	**True**
Key knowledge required	None

corresponding ciphertexts. This step precisely utilized 256 oracle queries without needing any information about the secret key. Table 3.2 summarizes the lookup table construction metrics.

3.5.3 Phase 2: Pixel-wise Plaintext Recovery via Channel Matching

For the provided ciphertext C^* and the lookup table created during Phase 1, the attack recovered pixels in a pairwise manner using the cross-coupling property of the second stage channels. As mentioned earlier from the set of equations (3.22)-(3.24), the red output channel contains information about the green plaintext channel, green output channel contains information about blue plaintext channel, and blue output channel contains information about red plaintext channel. For every k pixel, the attacker compared all 256 probes with the below matching conditions:

$$\hat{P}_G[k] = v \quad \text{such that} \quad C_R^{(v)}[k] = C_R^*[k], \tag{3.40}$$

$$\hat{P}_B[k] = v \quad \text{such that} \quad C_G^{(v)}[k] = C_G^*[k], \tag{3.41}$$

$$\hat{P}_R[k] = v \quad \text{such that} \quad C_B^{(v)}[k] = C_B^*[k]. \tag{3.42}$$

Because the function for the encryption of each pixel is a bijection from $\mathbb{Z}_{256}$ to itself, exactly one v will fit each match criteria, ensuring uniqueness and correctness at every pixel and channel level. This step involved no more oracle calls, operating in time $\mathcal{O}(256 \cdot N)$. Table 3.3 reports the pixel-wise recovery metrics.

Table 3.3: Pixel-wise plaintext recovery metrics.

Metric	Value
Additional oracle queries used	0
Pixel-wise matching conditions applied per pixel	3
Unique match guaranteed per pixel per channel	**True**
Key knowledge required	None

Table 3.4: CPA decryption accuracy metrics.

Metric	Value
Total oracle queries	256
CPA-decrypted image matches true plaintext ($\hat{P} = P$)	**True**
Max pixel difference	0
Mean pixel difference	0.000000
Pixel-match rate	100%

3.5.4 Full Decryption Without the Secret Key

The image reconstruction process, $\hat{P}$, consisted of simply arranging the channel matches produced in Phase 2. No part of the real secret key, namely the Hénon parameters, the logistic map parameters, or even any information regarding intermediate cipher states were involved at any stage. Table 3.4 reports the CPA decryption accuracy metrics.

It is evident from the outcomes that an image encrypted by MMCBIE is possible to recover completely through just 256 queries of the oracle, regardless of the image's size. Therefore, MMCBIE is *entirely insecure* in the CPA model even considering that it uses a 512 bit-long key. Table 3.5 consolidates the complete attack statistics across all phases. Figure 3.3 visually confirms the pixel-perfect recovery achieved by the attack. As the following analysis proves, good results for NPCR, UACI, and Entropy do not mean cryptographic security [34].

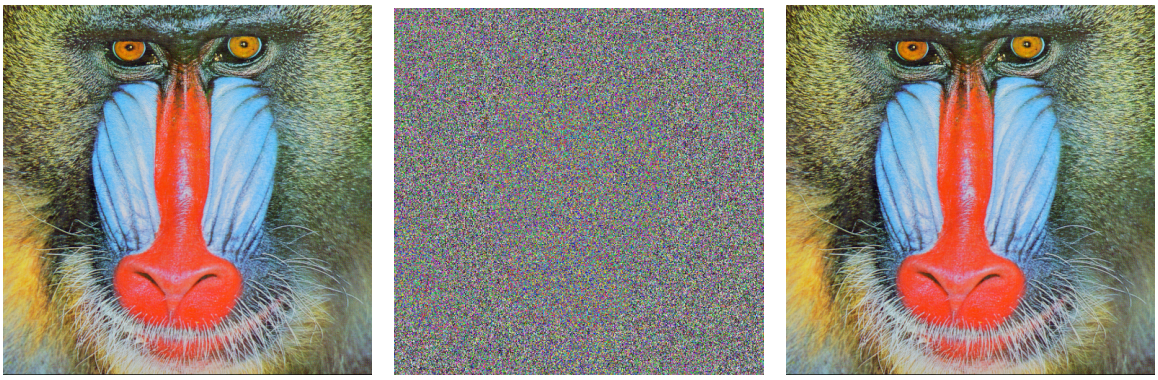
3.5.5 Root Cause Analysis: Why the Scheme is Broken

Three mutually reinforcing design defects collectively explain the attack's success.

Defect 1: Plaintext-independent chaotic keystreams. However, since both

Table 3.5: Summary of the chosen-plaintext attack on the MMCBIE scheme. Total oracle queries: 256.

Phase	Goal	Oracle calls	Items recovered	Perfect match
1	Probe lookup table $\{C^{(v)}\}_{v=0}^{255}$	256	256 full ciphertext images	Yes
2	Plaintext $\hat{P}$ via channel matching	0	Full image, all channels	Yes ($\Delta = 0$)
Total		**256**		**Exact recovery**



(a) Original Image (b) MMCBIE Encrypted (c) CPA Decrypted

Figure 3.3: Comparison of original, encrypted, and CPA-decrypted images. The decrypted image is pixel-identical to the original.

the Hénon map and the 2D-Logistic map have their initial seeds drawn solely from their corresponding fixed subkeys (x_H, y_H) and (x_L, y_L, R), without taking any inputs from the plaintext image, the chaos matrices that will be produced by them will remain consistent throughout the entire set of images encrypted using the exact same key. In fact, a cipher where its keystream does not depend on the plaintext itself will essentially become a one-time pad cipher, which is inherently insecure to attacks involving chosen-plaintext [34]. This is precisely why the probe lookup table works as expected: encryption of the same constant value v will result in the same ciphertext, no matter what the plaintext actually is.

Defect 2: Bijectivity of the per-pixel encryption function. The application of the XOR operation on $H[k]$, the addition of the key byte mod 256, and the XOR operation on $L[k]$ yields a bijection in $\mathbb{Z}_{256}$. Although the property of bijectivity can be viewed positively in block ciphers, in our context it ensures that the 256-probe ciphertexts in the lookup table will contain all possible values for any pixel position k and any particular channel, without any duplications whatsoever. Hence the matching criteria in equations (3.40)–(3.42) will result in an unambiguous determination of the

plaintext, ensuring accuracy.

Defect 3: Channel cross-coupling without plaintext dependence. During the Key Mixing phase, the green plaintext channel feeds into the red ciphertext channel, the blue plaintext channel feeds into the green ciphertext channel, while the red plaintext channel feeds into the blue ciphertext channel. It appears that the cross-coupling was added for the purpose of introducing confusion between the channels, but since the chaotic streams are independent of the plaintext, they can easily be removed by means of the probe-and-match approach. With one constant probe, all three channels will have the same value v, thus only 256 oracle queries are necessary to break all three plaintext channels in parallel. In case the chaotic streams depended on the plaintext, the cross-coupling would introduce some nonlinearities.

When all the weaknesses occur at once, the protection offered by the proposed scheme becomes equivalent to the process of table look-up only. The adversary will not have to find out the secret key, reverse any chaotic function, or solve any set of equations. The whole idea will boil down to simply encrypting 256 images and then looking up for the ciphertext value in a pre-built lookup table in order to get the plaintext values. This makes the proposed scheme insecure in the chosen-plaintext setting.

3.5.6 Recommendations for Strengthening Against Chosen-Plaintext Attacks

The analysis in this chapter highlights three essential flaws within the structure of MMCBIE, which together facilitate an elementary chosen plaintext attack. The following suggestions aim to rectify each flaw individually, providing insight into the construction of future chaos-based image encryption techniques.

Plaintext-Dependent Chaos Initialisation

The single most important change, however, would be to integrate the chaos in the generation of the keystream directly into the plaintext itself. As explained in Section 3.5.5, since neither $H[k]$ nor $L[k]$ depends on the plaintext at all, MMCBIE boils down to using a reusable pad, which is unconditionally insecure against chosen

plaintext attacks.

- **IV perturbation using hashes.** Before applying encryption, calculate the hash of the plaintext image $\mathcal{H}(P)$ and combine it via an XOR operation with the chaotic initial states for both generators. As the value of $\mathcal{H}(P)$ varies for each new plaintext, each pair of chaos matrices H and L becomes unique for each new plaintext, rendering the probe LUT useless when applied to a different plaintext.

- **Feedback coupling in chaos generation.** Instead of generating the chaos from static initial conditions, use the encrypted pixels periodically to reset the chaotic system state through some function. This results in dependencies between image regions, meaning that encrypting a probe image consisting of identical pixel values will no longer replicate the oracle behaviour on a real image.

- **Adding nonce into IV.** In addition to the chaotic initial conditions, a unique random value can be used for each encryption and included into the IV, ensuring that even two identical plaintexts result in different ciphertexts and hence cannot be probed successfully.

Strong Inter-Pixel Diffusion

The separability of the MMCBIE encryption algorithm in terms of pixels is what makes it possible for the attack to run separately for each pixel location. It is necessary for a robust technique to be able to make changes spread throughout the entire area of the image such that the encryption of any one pixel is dependent on other pixels.

- **Global permutation before the diffusion step.** Performing a global pixel permutation—dependent on the plaintext—such as the Arnold cat map based on the hash of the plaintext itself—before the diffusion stages renders impossible the assumption of independence upon which the CPA relies. The permutation of the pixels will ensure that the ciphertext of the probe will not be identical to that of the plaintext, since Equations (3.40) and (3.42) will not hold anymore.

- **Chaining diffusion with carry propagation.** Chaining together the diffusions in Stage 3 through a sum modulo 256, for instance,

$$C[k] = \big(B[k] \oplus L[k] \oplus C[k-1]\big) \bmod 256, \tag{3.43}$$

will guarantee that any given ciphertext value is dependent upon the entire sequence before. In turn, changing one bit of the plaintext will cause an avalanche effect in the entire ciphertext, overcoming the lack of this effect in the scheme.

- **Two or more diffusion rounds.** By performing two or more rounds of permutation and diffusion, the dependency of each ciphertext value on a greater number of plaintext values renders the independence assumption for the attack invalid.

Plaintext-Dependent and Non-Linear Key Mixing

However, in the key mixing stage of MMCBIE, the linear modular addition is deterministic, using a key stream with a period of 8. Hence, due to this deterministic approach, there is no security provided by this stage once the chaos streams become vulnerable to the probe table, because there are just eight points at which the exploit could be done.

- **Use of substitution box**. The substitution of the modular addition with a substitution box (S-box), based on e.g., a chaotic Latin square [54], adds true nonlinearity and stops the mapping from being a bijective function whose enumerations take only 256 oracle queries.

- **Key schedule with period cycling**. Expanding the lengths of key-mixing arrays MK_R, MK_G, and MK_B to MN from 8 removes the periodicity that limits the complexity of Stage 2 to 8 bits. In actual implementation, this can be done by generating a longer subkey with a pseudo-random generator seeded with the master key.

- **Data-driven cross-channel operations**. As an improvement over a predetermined rotation pattern (G to R, B to G, R to B), the cross-channel relation-

ships must be determined using either the plaintext data or a function of the key/plaintext combination having high entropy.

Security Validation Beyond Statistical Metrics

A general lesson from this study is that good scores for NPCR, UACI, and Shannon entropy cannot guarantee cryptographic security [34]. These metrics provide information about the statistical behavior of the ciphertext distribution but fail to evaluate resistance against active attacks in the chosen plaintext/ciphertext model scenarios. For future designs, security guarantees such as IND-CPA security need to be verified along with applying standard cryptanalytic methods—differential, linear, and algebraic attacks. Consequently, this will prevent the use of flawed schemes in security-sensitive IoT settings.

3.6 Summary

For the analysis, we use the chosen-plaintext model to examine the chaotic image encryption technique MMCBIE presented by Jain et al.[37]. Despite being based on several chaotic maps, including the Hénon map, a cyclic key mixing step, and a two-dimensional logistic map, the technique has been proven to be vulnerable because of weak diffusion and inherent structural properties.

Weaknesses in the technique arise from insufficient dependency of adjacent pixels and channels. Transformations applied in the encryption technique are performed independently and follow a certain pattern; thus, there still remain correlations between corresponding plaintext and ciphertext values. Specifically, dependencies arising in the key mixing operation applied on each channel remain unchanged; moreover, the lack of feedback leads to ineffective diffusion of alterations.

Based on the above features, an efficient attack strategy is developed. By making requests to the oracle encryption service for constant images within the whole range of pixels, we obtain cipher texts. Comparing them with the target ciphertext using simple equality checks allows us to retrieve original plaintext values bit by bit. It should be noted that in this way, we do not need to restore chaotic series, secret parameters, or keys.

Theoretically and experimentally, it was proved that using the above attack, exact restoration of the original image is possible using no more than 256 queries. In other words, we demonstrate a total failure of the encryption scheme despite its excellent results in statistics tests such as NPCR, UACI, and entropy.

These findings confirm one of the basic tenets of cryptography: Statistical randomness is not enough to ensure security from active attacks. In order for chaos-based image cryptosystems to be considered secure, it is imperative that they use diffusive functions, have inter-pixel and inter-channel interactions, and employ non-linear functions that depend on the plain-image input.

CHAPTER 4

PoolLiteSIS: A Lightweight Secret Image Sharing Using Pooling-Based Dimensional Reduction

This chapter presents PoolLiteSIS, a lightweight secret image sharing framework that integrates average pooling-based dimensional reduction with XOR-based share generation to achieve secure and storage-efficient distribution of images. It introduces the use of pooled representations and residual encoding, combined with image-dependent permutation for enhanced security, and details the complete pipeline for shadow generation and lossless reconstruction. The chapter further evaluates the method in terms of storage efficiency, reconstruction accuracy, and security, demonstrating significant reduction in share size while maintaining perfect reconstruction and resistance to information leakage.

4.1 Introduction

The emergence of digital multimedia content has made images a basic form of information transfer in personal, business, and scientific contexts. Many sectors prefer encoding private information in the form of images, and hence protection against unauthorized viewing and manipulation is a matter of utmost concern. Various secure measures such as encryption schemes, visual cryptographic systems, and steganographic systems have proved to be effective. Nevertheless, these techniques essentially depend on centralized storage models, which are prone to total data loss due to single-point failures. When adversaries intercept or damage the only storage device, the embedded private information becomes unrecoverable. To overcome these limitations, *Secret Image Sharing (SIS)* systems use distributed storage models.

In SIS models, secure images are decomposed into several shares or shadow im-

ages, which are then shared among the participants. Only when a threshold number of shares is gathered can the image be reconstructed, while any sub-threshold collection of shares provides no information about the original image. These models are very useful in protecting highly classified information such as cryptographic keys, authorization codes, and financial data.

4.2 Preliminaries

This section presents the fundamental concepts and mathematical tools underlying our research framework.

4.2.1 Average Pooling Operations

Average Pooling is a form of a dimensional reduction method that is commonly used in CNN architectures. For a given input image I of size $m \times n$, an average pooling operation of size $k \times k$ with a stride of s divides the image into regions (possibly overlapping if $s < k$) and replaces them with their average value.

For a block B of size $k \times k$, the pooled value computes as:

$$P_{avg} = \frac{1}{k^2} \sum_{i=1}^{k} \sum_{j=1}^{k} B_{ij} \tag{4.1}$$

The resulting pooled image P exhibits dimensions: $\frac{m}{s} \times \frac{n}{s}$. To enable perfect reconstruction, residuals are computed as:

$$R_{ij} = I_{ij} - P_{avg} \tag{4.2}$$

where R_{ij} captures the deviation between original pixel value I_{ij} and corresponding pooled value.

4.2.2 XOR-Based Secret Sharing

The XOR operation has perfect diffusion, meaning that if one of the operands is random, the result is indistinguishable from random. We give a brief overview of the

(n, n) SIS scheme given by Wang et al. [66]. The dealer D creates $n-1$ random matrices for the (n, n) threshold schemes:

$$\{B_1, B_2, \ldots, B_{n-1}\} \tag{4.3}$$

Each of these matrices has the same dimensions as the secret image.

The n share images $\{SI_1, SI_2, \ldots, SI_n\}$ are constructed sequentially through XOR operations:

$$SI_1 = B_1 \tag{4.4}$$

$$SI_2 = B_1 \oplus B_2 \tag{4.5}$$

$$SI_3 = B_2 \oplus B_3 \tag{4.6}$$

$$\vdots \tag{4.7}$$

$$SI_{n-1} = B_{n-2} \oplus B_{n-1} \tag{4.8}$$

$$SI_n = B_{n-1} \oplus I_s \tag{4.9}$$

where $\oplus$ denotes bitwise XOR.

To reconstruct the original image, the combiner performs bitwise XOR across all n share images:

$$I'_s = SI_1 \oplus SI_2 \oplus \cdots \oplus SI_n \tag{4.10}$$

4.3 Literature Review

XOR-based SIS methods were developed as computationally efficient alternatives to complex cryptographic protocols. Initial research by Wang et al. [66] introduced (n, n) SIS schemes in which dealers produce $(n-1)$ random matrices by computing the n-th share via the XOR operation of the last random matrix and the image. Share reconstruction is achieved by XORing all n shares. Although removing pixel expansion and preserving perfect contrast for binary, grayscale, and color images, shadow dimensions are equal to the original images.

Later research by Chattopadhyay et al. [67] incorporated hash verification in (n, n) arrangements, allowing participants to verify tampered shares before reconstruction.

Although flexible when adding more shares, conventional arrangements retain the same shadow dimensions as the original images.

Chen and Wu [68, 69] built various Multiple SIS (MSIS) schemes using Boolean operations for encoding multiple secrets into the same share sets, but with unchanged shadow dimensions. In later studies, they added more randomness to avoid partial information leakage from sub-threshold sets, but with shadows still having original dimensions. Another study by Nag et al. [70] suggested Boolean MSIS schemes with individual shares expanded to $t \times r \times c$ depending on the number of secret images and the matrix arrangement. Deshmukh et al. [71] introduced a Chinese Remainder Theorem (CRT) method for multiple color image sharing with shadows of original sizes.

Prasetyo and Hsia [72] improved multiple secret sharing using generalized chaotic image scrambling, which combined XOR operations with hyperchaotic maps for enhanced security with reconstruction issues for odd secret numbers, but with shadows still having original dimensions. In another study [73], they developed lossless progressive schemes for incremental quality improvement with additional shares. Guo et al. [74] suggested multi-threshold schemes using generalized CRT with shadows of original sizes.

Kabirirad and Eslami [75] generalized MSIS to (t, n)-threshold schemes using Boolean operations, allowing flexibility where t out of n shares are needed for reconstruction. But this method needs t consecutive shares for backtracking recovery, where the absence of any single share will disrupt the chain, and shadows retain original size. Faraoun [76] designed highly secure (n, n) multi-SIS schemes using cryptographic hash functions and stream ciphers, encrypting secrets with random session keys, then splitting keys using XOR chaining between n shares, producing shadows of original size. Lin et al. [77] designed key-free SIS schemes using improved Cycling-XOR techniques, embedding shares into separate cover images for better concealment, also producing shadows of original size.

Few studies focus on minimizing shadow size in XOR and Boolean schemes. Ou et al. [78] designed user-friendly secret-image sharing schemes using block truncation coding and error diffusion, resulting in shares $\frac{1}{4}$ smaller than originals, yet still meaningful. Research by Purushothama [79] described dimensional reduction where,

instead of the usual (n, n)-threshold XOR schemes producing shares of size $h \times w$, their scheme produces shadows of size $\lceil h/n \rceil \times w$, greatly enhancing storage space efficiency.

The literature review shows that, although XOR-based methods are simple and computationally efficient, they always produce shadows with the same dimensions as the original image $(n \times n)$, which results in high redundancy and storage costs. The focus of research has been on improving security, but not on the limitations of storage efficiency. This chapter proposes to maintain the advantages of lightweight and lossless XOR operations while significantly reducing the shadow size by using pooling-based dimensional reduction and permutation techniques based on images.

4.4 Work Done

The basic problem in SIS is to strike a balance between security, reconstruction accuracy, and storage space efficiency. The existing approaches tend to focus on security with the penalty of storage space overhead. The contributions of this research work are as follows:

1. We propose a new XOR-based SIS framework that combines average pooling to achieve 75% spatial dimension reduction, significantly reducing the sizes of shadow images.

2. We describe a secure residual permutation technique using the pooled image as a seed, making it impossible to leverage the residual information without having the entire shadow image.

3. We show a total storage reduction of 75% for shadow images over traditional XOR-based approaches while ensuring perfect lossless reconstruction.

4. We offer a thorough experimental evaluation on standard benchmark images, examining both storage and security properties.

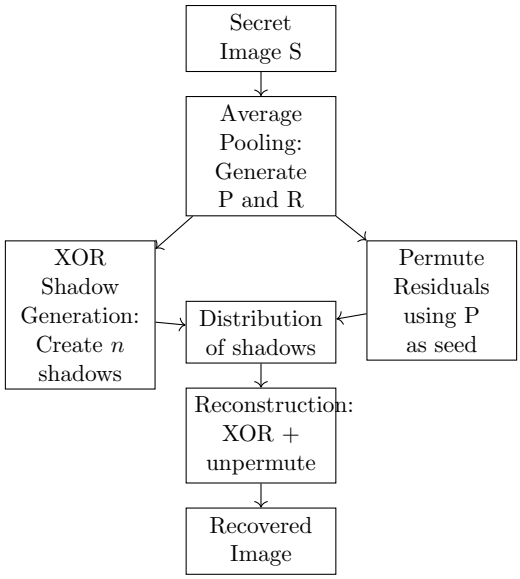


Figure 4.1: Comprehensive Overview of Proposed Framework

Our system consists of three major stages: preprocessing by average pooling, shadow creation by cyclic XOR operations, and reconstruction by unpooling. First, the secret image S is subjected to *average pooling*, resulting in two complementary parts: the pooled matrix P and the residual matrix R. The pooled matrix P serves as a compact form, later used to produce multiple shadows based on XOR operations. At the same time, the residuals R are subjected to *permutation with P as the seed*, making sure that the reconstruction key is image-dependent and cannot be reverse-engineered by unauthorized parties. In the *distribution phase*, the produced shadows are safely distributed among the shareholders. To restore the original secret image, all shadows are subjected to XOR operations, and the residuals are *unpermuted with the same seed P*. This produces the restored secret image, which retains perfect visual quality without losing any information. Our system is designed to produce shadows with a considerably smaller size, with a maximum reduction of 75% compared to the original secret image, outperforming other XOR-based techniques. The entire processing chain is shown in Figure 4.1.

4.4.1 Phase 1: Average Pooling Preprocessing

The pooling operation divides the original grayscale image $S \in \mathbb{R}^{N \times N}$ into non-overlapping blocks of size $K \times K$. Let s be the stride (for non-overlapping blocks, usually $s = K$). For each block (i, j), the pooled value $P(i, j)$ is defined as the arithmetic mean of the K^2 pixel values in that block:

$$P(i,j) \;=\; \frac{1}{K^2} \sum_{x=i\cdot s}^{(i+1)\cdot s-1} \sum_{y=j\cdot s}^{(j+1)\cdot s-1} S(x,y). \tag{4.11}$$

The residual map $R \in \mathbb{R}^{N \times N}$ captures per-pixel deviations from block averages:

$$R(x,y) \;=\; S(x,y) \;-\; P\Big(\Big\lfloor \frac{x}{s} \Big\rfloor, \Big\lfloor \frac{y}{s} \Big\rfloor\Big). \tag{4.12}$$

Since R maintains identical spatial dimensions as S, the original image can be reconstructed exactly (in exact arithmetic) by adding the block-wise broadcast of P to R:

$$\widehat{S}(x,y) \;=\; P\Big(\Big\lfloor \frac{x}{s} \Big\rfloor, \Big\lfloor \frac{y}{s} \Big\rfloor\Big) \;+\; R(x,y). \tag{4.13}$$

The pooling process is summarized in Algorithm 3. This phase generates the following outputs:

1. **Pooled Image** P: A dimensionally reduced version of size $N/s \times N/s$ pixels, where s is the pooling stride.

2. **Residual Image** R: A matrix of size $N \times N$ storing the difference between each original pixel and its corresponding pooled average value.

Note that, in practice, when storing P as an integer type (e.g., `uint8`), the block mean is typically rounded (e.g., to nearest integer or truncated). This introduces a quantization error, which can be compensated for by the residuals (if stored with sufficient precision). If the residuals are stored as signed integers with sufficient range, then exact reconstruction is still possible even if P is rounded, as long as the rounding strategy is consistent during pooling and unpooling operations.

Algorithm 3: Average Pooling

Input: Grayscale image S of size $N \times N$, pooling window of size $K \times K$, stride s

Output: Pooled image P of size $\frac{N}{s} \times \frac{N}{s}$ and residual image R of size $N \times N$

$P \leftarrow$ zero matrix of size $\frac{N}{s} \times \frac{N}{s}$
$R \leftarrow$ zero matrix of size $N \times N$
for $i \leftarrow 0$ ***to*** $\frac{N}{s} - 1$ **do**
  for $j \leftarrow 0$ ***to*** $\frac{N}{s} - 1$ **do**
    Extract block $B \leftarrow S[i \cdot s : (i+1) \cdot s,\ j \cdot s : (j+1) \cdot s]$
    Compute block average $a \leftarrow \text{mean}(B)$
    Assign pooled value $P(i, j) \leftarrow a$
    Compute residuals:
    $R[i \cdot s : (i+1) \cdot s,\ j \cdot s : (j+1) \cdot s] \leftarrow B - a$
  end
end
return P, R

Illustrative Example: 4×4 Image with 2×2 Pooling (Block Size $K = 2$, Stride $s = 2$)

Consider input image S as the 4×4 matrix:

$$S = \begin{bmatrix} 100 & 102 & 50 & 52 \\ 98 & 101 & 49 & 51 \\ 200 & 202 & 150 & 151 \\ 198 & 199 & 149 & 152 \end{bmatrix}. \tag{4.14}$$

We partition S into four non-overlapping 2×2 blocks. Computing block sums and means:

- Top-left block (index $(0, 0)$):

$$B_{00} = \begin{bmatrix} 100 & 102 \\ 98 & 101 \end{bmatrix}, \quad \text{sum}(B_{00}) = 401, \quad P(0, 0) = 100.25. \tag{4.15}$$

- Top-right block (index $(0,1)$):

$$B_{01} = \begin{bmatrix} 50 & 52 \\ 49 & 51 \end{bmatrix}, \quad \text{sum}(B_{01}) = 202, \quad P(0,1) = 50.5. \tag{4.16}$$

- Bottom-left block (index $(1,0)$):

$$B_{10} = \begin{bmatrix} 200 & 202 \\ 198 & 199 \end{bmatrix}, \quad \text{sum}(B_{10}) = 799, \quad P(1,0) = 199.75. \tag{4.17}$$

- Bottom-right block (index $(1,1)$):

$$B_{11} = \begin{bmatrix} 150 & 151 \\ 149 & 152 \end{bmatrix}, \quad \text{sum}(B_{11}) = 602, \quad P(1,1) = 150.5. \tag{4.18}$$

Thus the pooled image P (size 2×2) becomes:

$$P = \begin{bmatrix} 100.25 & 50.5 \\ 199.75 & 150.5 \end{bmatrix}. \tag{4.19}$$

Next, we compute the full-size residual map R by subtracting corresponding block means from each pixel:

Top-left residuals for block B_{00}:

$$\begin{aligned} R(0,0) &= -0.25, \quad R(0,1) = 1.75, \\ R(1,0) &= -2.25, \quad R(1,1) = 0.75. \end{aligned} \tag{4.20}$$

Top-right residuals for B_{01}:

$$\begin{aligned} R(0,2) &= -0.5, \quad R(0,3) = 1.5, \\ R(1,2) &= -1.5, \quad R(1,3) = 0.5. \end{aligned} \tag{4.21}$$

Bottom-left residuals for B_{10}:

$$\begin{aligned} R(2,0) &= 0.25, \quad & R(2,1) &= 2.25, \\ R(3,0) &= -1.75, \quad & R(3,1) &= -0.75. \end{aligned} \tag{4.22}$$

Bottom-right residuals for B_{11}:

$$\begin{aligned} R(2,2) &= -0.5, \quad & R(2,3) &= 0.5, \\ R(3,2) &= -1.5, \quad & R(3,3) &= 1.5. \end{aligned} \tag{4.23}$$

Collecting these entries yields the residual matrix:

$$R = \begin{bmatrix} -0.25 & 1.75 & -0.5 & 1.5 \\ -2.25 & 0.75 & -1.5 & 0.5 \\ 0.25 & 2.25 & -0.5 & 0.5 \\ -1.75 & -0.75 & -1.5 & 1.5 \end{bmatrix}. \tag{4.24}$$

4.4.2 Phase 2: Residual Permutation for Security

Algorithm 4: Permutation of Residual Matrix

Input: Residual matrix R of size $N \times N$, pooled image P

Output: Permuted residual matrix R' and permutation indices π

Compute the seed value from the pooled image:

$s \leftarrow \left(\sum_{i=1}^{N} \sum_{j=1}^{N} P(i,j)\right) \bmod 2^{32}$

Initialize pseudo-random generator using s;

Flatten R into vector R_f of length $M = N \times N$;

Create index vector $I \leftarrow [0, 1, \ldots, M-1]$;

Randomly shuffle I using the initialized generator to obtain π;

for $k \leftarrow 0$ **to** $M - 1$ **do**

  $R'_f[k] \leftarrow R_f[\pi[k]]$;

end

Reshape R'_f into R' of size $N \times N$;

return R', π;

A major security issue arises when the residual image R, without being permuted,

tends to be similar to the original image S. The unpermuted residuals had relatively high SSIM values with the pooled image, signifying a leakage of information. To maintain confidentiality, the residual matrix is permuted through a deterministic pseudo-random permutation generated from the pooled image itself. This involves:

- **Seed generation**: A unique seed s is computed from the pixel intensities of the pooled image as:

$$s = \left(\sum_{i=1}^{M} \sum_{j=1}^{N} P(i,j) \right) \bmod 2^{32} \tag{4.25}$$

  This seed initializes a random number generator, ensuring that the permutation is image-dependent and reproducible only after reconstructing the pooled image from all secret shares.

- **Permutation:** Let the flattened residual vector be $R_f = [r_0, r_1, \ldots, r_{N-1}]$ where $N = M \times N$. A random permutation vector $\pi = [\pi(0), \pi(1), \ldots, \pi(N-1)]$ is generated, and the permuted residuals are obtained as:

$$R'_f[i] = R_f[\pi(i)], \quad \forall i \in [0, N-1] \tag{4.26}$$

  Finally, R'_f is reshaped to form the permuted residual matrix R'.

This process provides the benefit of implicitly securing the permutation key, thereby obviating the need for key exchange. Algorithm 4 illustrates the procedure adopted to permute the residual image. After the permutation process, the SSIM value between the permuted residuals and the pooled image decreases substantially, thereby destroying the structural correlations.

4.4.3 Phase 3: Cyclic XOR-Based Shadow Construction

This stage uses the XOR-based SIS method, where the cyclic XOR operations are employed to create the shadow images. This is done by taking advantage of the basic property of the XOR operation, which is its own inverse, as expressed below:

$$A \oplus B \oplus B = A \tag{4.27}$$

This property ensures that when two XOR operations are carried out using the same operand, the presence of the operand is completely eliminated. Therefore, it becomes possible to divide the information into several shadow images, where each shadow image holds a part of the total information, and the original pooled image P can only be reconstructed if all n shadow images are obtained.

Algorithm 5: XOR-Based Shadow Generation

Input: Pooled image $I_s = P$ of size $\frac{N}{s} \times \frac{N}{s}$
Input: Stride s, number of shadows n
Output: Shadow set $\{SI_1, SI_2, \ldots, SI_n\}$

$M \leftarrow \frac{N}{s}$;
Spatial dimension of pooled image;
for $i = 1$ **to** $n - 1$ **do**
    Generate random matrix B_i;
    Size: $M \times M$, values in $[0, 255]$;
end
$SI_1 \leftarrow B_1$;
for $i = 2$ **to** $n - 1$ **do**
    $SI_i \leftarrow B_{i-1} \oplus B_i$;
end
$SI_n \leftarrow B_{n-1} \oplus I_s$;
return $\{SI_1, SI_2, \ldots, SI_n\}$;

Let $I_s = P$ be the pooled image formed after the average pooling phase, with a size of $\frac{N}{s} \times \frac{N}{s}$. To form the n shadow images, the dealer first creates $n - 1$ random matrices $\{B_1, B_2, \ldots, B_{n-1}\}$ of the same size as I_s. Each B_i is populated with random grayscale values uniformly distributed between the range $[0, 255]$, thus removing any recognizable pattern of the original image.

The shadow images $\{SI_1, SI_2, \ldots, SI_n\}$ are created through a cyclic XOR operation sequence. The dealer first assigns the first random matrix as the first shadow image and then applies a cyclic XOR operation between consecutive random matrices to form the subsequent shadow images. The last shadow image combines the last random matrix with the pooled image, thus hiding the secret message inside the cyclic

XOR operation chain. The expression for the resulting formulation is as follows:

$$\begin{aligned} SI_1 &= B_1 \\ SI_2 &= B_1 \oplus B_2 \\ SI_3 &= B_2 \oplus B_3 \\ &\vdots \\ SI_{n-1} &= B_{n-2} \oplus B_{n-1} \\ SI_n &= B_{n-1} \oplus I_s \end{aligned} \tag{4.28}$$

A pseudocode implementation of this is shown in Algorithm 5. The shadow images are of the same size as the pooled image of $\frac{N}{s} \times \frac{N}{s}$, which is 75% smaller than the original secret image in the case of K=s=2.

4.4.4 Phase 4: Reconstruction

Algorithm 6 summarizes the steps to recover the secret image after obtaining all the share images.

To recover the secret image, the combiner performs the following steps:

1. **XOR all shadow images** to reconstruct the pooled image:

$$I'_s = SI_1 \oplus SI_2 \oplus \ldots \oplus SI_n = P \tag{4.29}$$

   Due to the cyclic cancellation property of XOR, all random matrices cancel out, leaving only the pooled image.

2. **Reverse the permutation** using I'_s as the seed to generate the permutation matrix π, then compute:

$$R = \pi^{-1}(R_{permuted}) \tag{4.30}$$

3. **Apply average unpooling**: Upsample the pooled image by replicating each value to fill its corresponding $K \times K$ block, then add the residuals:

$$S_{ij} = P'_{avg} + R_{ij} \tag{4.31}$$

This process guarantees perfect reconstruction of the original secret image.

Algorithm 6: Reveal Phase

Input: Shadow set $\{SI_1, SI_2, \ldots, SI_n\}$

Input: Permuted residuals R_{permuted}, stride s

Output: Reconstructed image $I_{\text{reconstructed}}$

$I_s \leftarrow SI_1$;
for $i = 2$ **to** n **do**
    $I_s \leftarrow I_s \oplus SI_i$;
end
Compute seed from I_s;
$seed\_value \leftarrow \sum_{i,j} I_s(i, j)$;
$seed\_value \leftarrow seed\_value \bmod 2^{32}$;
Initialize PRNG with $seed\_value$;
Generate permutation vector π;
$R' \leftarrow R_{\text{permuted}} - 128$;
$R \leftarrow \pi^{-1}(R')$;
for $p = 0$ **to** $\frac{N}{s} - 1$ **do**
    for $q = 0$ **to** $\frac{N}{s} - 1$ **do**
        for $a = 0$ **to** $s - 1$ **do**
            for $b = 0$ **to** $s - 1$ **do**
                $x \leftarrow ps + a$;
                $y \leftarrow qs + b$;
                $I_{\text{reconstructed}}[x, y] \leftarrow I_s[p, q] + R[x, y]$;
            end
        end
    end
end
return $I_{\text{reconstructed}}$;

4.5 Results and Analysis

We have validated our proposed approach using n = 4 shadows, and we have created a (4, 4) threshold configuration. The standard grayscale Cameraman image with 512 ×

512 pixels was used as our test image. All experiments were performed using Python 3.8, and NumPy was used for matrix calculations.

4.5.1 Storage Efficiency Analysis

Table 4.1 presents a comparative analysis of storage requirements between our approach and a conventional XOR-based (4, 4) scheme without pooling.

Table 4.1: Storage Analysis: Proposed Method vs. Naive XOR Scheme

Component	**Size (bytes)**
Original Image	262,144
Naive XOR Scheme	
Individual Share Size	262,144
Total Storage (4 shares)	1,048,576
Proposed Method	
Individual Share Size	65,536
Total Storage (4 shares)	262,144
Storage Reduction	**75.00%**

- The traditional XOR method needs to store 1,048,576 bytes (1024 KB) because each of the four shadow images has the same size as the original image, which is 256 KB.
- Our proposed method only needs to store 262,144 bytes (256 KB) in total, which is $4 \times 65{,}536 = 262{,}144$ bytes for the four shadow images.
- This brings a 75% reduction in the total storage needs while providing the same level of security guarantees.

4.5.2 Reconstruction Quality

Perfect lossless reconstruction was obtained in all experiments. Table 4.2 shows the quality measures for each test image. The infinite PSNR value and SSIM of 1.0 indicate that the reconstructed images are pixel-perfect copies of the original secret images with no loss of information.

Table 4.2: Reconstruction Quality Metrics

Image	**PSNR (dB)**	**SSIM**
Jet	∞	1.0000
Cameraman	∞	1.0000
Milk	∞	1.0000
Baboon	∞	1.0000
Barbara	∞	1.0000
Bridge	∞	1.0000

4.5.3 Security Validation

We calculated the Structural Similarity Index (SSIM) among various components to check the security of the residual permutation. Table 4.3 presents the security analysis of the Cameraman image. The most important points to note are:

- The unpermuted residuals are highly similar (SSIM $\approx$ 0.79–0.85) to the combined image, which may lead to leakage of information.
- However, after permutation, the similarity reduces significantly to 0.03, which ensures the removal of structural information.
- The individual shadow images do not contain any information about the secret, as their similarity (SSIM $\approx$ 0.01-0.02) to the original and combined images is very low.

Table 4.3: SSIM Analysis for Security Validation (Cameraman Image)

Comparison	**SSIM**
Permuted Residuals vs. Original Image	0.03
Individual Shadow vs. Pooled Image	0.01

4.5.4 Computational Complexity

The computational complexity of our method is dominated by:

1. **Pooling Operation**: $O(mn)$ for an $m \times n$ image

2. **Permutation**: $O(mn)$ for generating and applying the permutation

3. **XOR Operations**: $O(n \cdot \frac{m}{k} \cdot \frac{n}{k})$ for n shadows on a pooled image

4. **Unpooling**: $O(mn)$ for reconstruction

All operations are linear in image size, ensuring efficiency for real-world applications. XOR operations are especially fast because of bitwise parallelism in modern processors.

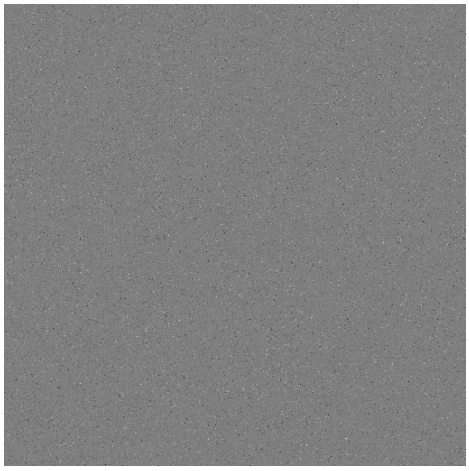
4.5.5 Visual Results

Figure 4.2 shows the standard benchmark image used in our experiments. The Cameraman image is suitable for testing the effectiveness of the proposed method because of the presence of both smooth and textured areas.

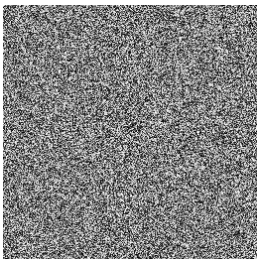
Figure 4.2: Cameraman image (512×512) used as the test image for evaluation.

Figure 4.3 shows the overall process of our proposed XOR-based technique with pooling and permutation for the Cameraman image. The figure clearly shows the transition from the original secret image to the four shadow images after the creation of residuals and permutation. The shadow images, which look like noise, ensure that there is no leakage of information about the secret image. The noise nature of the permuted residuals proves that the permutation technique is effective in removing structural information.

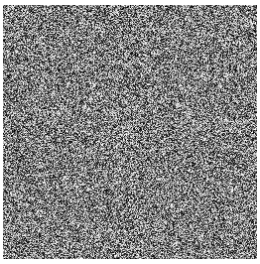
(a) Original Image



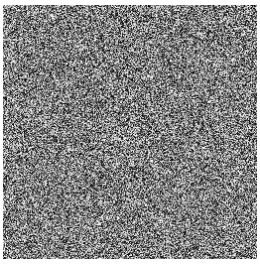
(b) Permuted Residuals



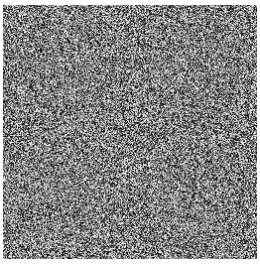
(c) Shadow Image 1



(d) Shadow Image 2



(e) Shadow Image 3

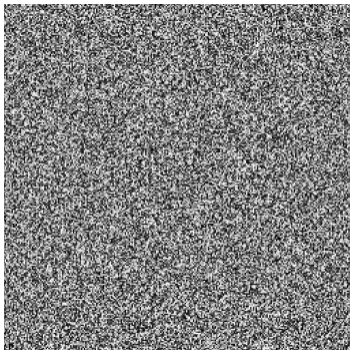


(f) Shadow Image 4

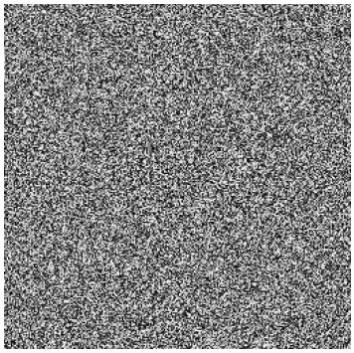
Figure 4.3: Visual demonstration of the pooling-permutation optimized XOR-based SIS pipeline on the Cameraman image. (a) Original 512×512 secret image. (b) Permuted residuals showing destroyed structural correlation (SSIM = 0.03). (c–f) Four shadow images of size 256×256 appearing as random noise.

Moreover, Figure 4.4 illustrates the reconstruction attempts with only three out of the four shares and two out of the four shares on the Cameraman test image. The reconstructed images are obviously noisy and do not have any resemblance to the original hidden image. This verifies that our scheme satisfies the basic requirement of SIS, which is that no information is revealed with less than the threshold number of shares.

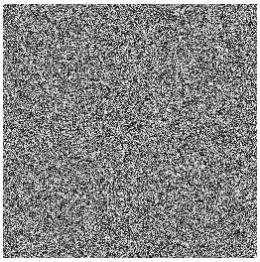
(a) Original Image



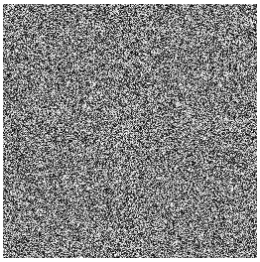
(b) Reconstructed Image with any 2 Shares



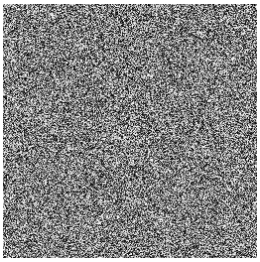
(c) Reconstructed Image with any 3 Shares



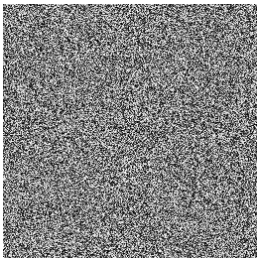
(d) Shadow Image 1



(e) Shadow Image 2



(f) Shadow Image 3



(g) Shadow Image 4

Figure 4.4: Comparison between the original Cameraman image and its partial reconstructions using different numbers of shares. (a) Original 512×512 secret image. (b) Reconstruction from any 2 shares showing complete information loss. (c) Reconstruction from any 3 shares showing complete information loss. (d–g) Corresponding four shadow images (each 256×256) appearing as random noise.

4.5.6 Storage Analysis with Varying Number of Shares

We analyzed the storage requirements for n = 2 to 10 to determine the scalability of the proposed secure pooling-based sharing system with the number of shares n. The original grayscale image was 512×512 pixels, which corresponds to an initial storage capacity of 262,144 bytes.

Our scheme decomposes the image into a pooled image reduced by a factor of two and a residual matrix. Although the residual matrix is stored separately, the pooled image is shared by all participants. The storage analysis does not include the residual matrix in the total storage computation.

Thus, the total storage is comprised entirely of all pooled shares. This corresponds to a 75

Table 4.4: Total storage requirements for different number of shares (n), excluding the residual matrix.

n	Original (B)	Pooled (B)	Per Share (B)	Total (B)	Naive Total (B)	Reduction (%)
2	262144	65536	65536	131072	524288	75.00
3	262144	65536	65536	196608	786432	75.00
4	262144	65536	65536	262144	1048576	75.00
5	262144	65536	65536	327680	1310720	75.00
6	262144	65536	65536	393216	1572864	75.00
7	262144	65536	65536	458752	1835008	75.00
8	262144	65536	65536	524288	2097152	75.00
9	262144	65536	65536	589824	2359296	75.00
10	262144	65536	65536	655360	2621440	75.00

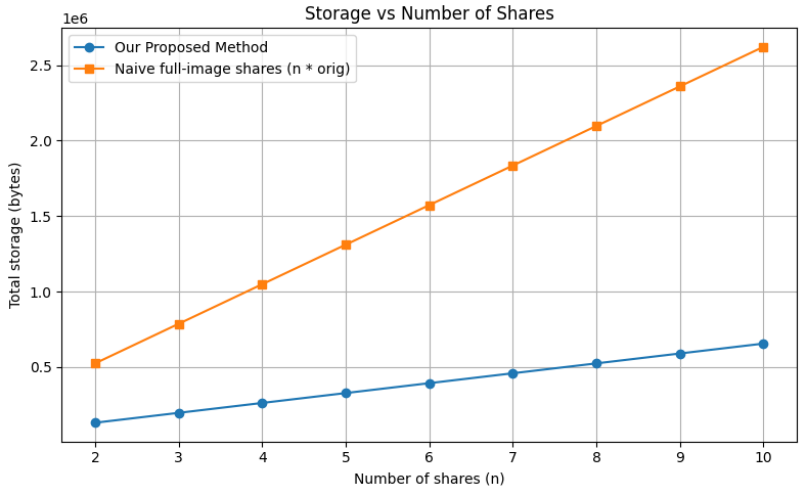


Figure 4.5: Comparison of secure and naive total storage with increasing number of shares.

For example, when n = 4, the contribution of each of the four pooled shares to the storage is 262,144 bytes. On the other hand, the naive approach of full-image sharing would have resulted in the storage of 1,048,576 bytes, resulting in a 75% reduction. Figure 4.5 plots the storage comparison graphically for varying numbers of shares.

In general, the results clearly show that the secure pooling-based approach maintains perfect reconstruction with substantial storage savings. The storage savings are constant at 75% for any value of n, clearly indicating the efficiency of the pooled shares in storage.

4.5.7 Comparison with Existing XOR-Based Methods

The performance of the proposed pooling-based XOR SIS approach is compared with other existing XOR-based methods reported in the literature. The comparison is

highlighted in Table 4.5.

Table 4.5: Comparison between a few XOR-based SIS schemes

Schemes	**Lossless recovery**	**Share type**	**Access structure**	**Share size**
Wang *et al.* [66] scheme-2	yes	meaningless	(n, n)	$n \times n$
Chen–Wu [69, 68]	yes	meaningless	(n, n)	$n \times n$
Lin *et al.* [77]	yes	meaningless	(n, n)	$n \times n$
Guo *et al.* [74]	yes	meaningless	(t, n)	$n \times n$
Prasetyo-Hsia [73]	conditional	meaningless	(n, n) progressive	$n \times n$
Chattopadhyay *et al.* [67]	yes	meaningless	(n, n)	$n \times n$
Faraoun [76]	yes	meaningless	(n, n)	$n \times n$
Chattopadhyay *et al.* [80]	yes	meaningless	(t, n)	$n \times n$
Kabirirad-Eslami [75, 81]	yes	meaningless	(t, n)	$n \times n$
Soreng and Kandar [82]	yes	meaningless	(t, n)	$n \times n$
Our proposed method	**yes**	**meaningless**	**(n, n)**	$\frac{n}{4} \times \frac{n}{4}$

4.6 Summary

The proposed SIS method shows a substantial improvement over the existing XOR-based methods by realizing a 75% reduction in total storage needs without compromising the quality of reconstruction. The experimental results on the test images validate that the reconstructed output preserves the lossless fidelity with PSNR = $= \infty$ and SSIM = 1.0. The addition of the residual permutation mechanism further improves the security by shattering the structural relationships, which is evident from the low SSIM values between the residual and pooled images, thus indicating the negligible information disclosure.

CHAPTER 5

Vulnerability Assessment of XOR-Based Secret Image Sharing Schemes Against Autoencoder Based Reconstruction Attacks

This chapter investigates the security of XOR-based secret image sharing schemes under learning-based adversaries by evaluating whether autoencoder models can reconstruct secret images from incomplete share sets. It introduces an encoder–decoder framework trained to exploit statistical priors of natural images and assesses reconstruction performance under varying levels of share availability. Through theoretical discussion and experimental evaluation using PSNR and SSIM metrics, the chapter demonstrates that while neural networks can achieve near-perfect reconstruction when all shares are present, they fail to recover meaningful information when shares are missing, thereby confirming the resilience of XOR-based schemes against such attacks.

5.1 Introduction

Secret image sharing schemes distribute a confidential image S among n participants by creating shares $\{s_1, s_2, ..., s_n\}$. To reconstruct S, a qualified subset of shares is required. XOR-based schemes achieve this with perfect secrecy; any strict subset of shares is statistically independent of the secret. They also avoid pixel expansion, making them efficient for practical use. However, this information-theoretic guarantee assumes that adversaries are computationally unbounded and have no prior knowledge of natural image statistics. Modern deep learning models can exploit these statistical patterns. This raises the question whether a learning-based adversary can reconstruct a meaningful approximation of S from a limited subset of its XOR shares.

This chapter examines this threat by proposing an autoencoder-based reconstruc-

tion attack targeting XOR-generated secret shares. An encoder-decoder network with skip connections is trained to recover the secret image from a consecutive subset of $k < n$ shares, without needing any cryptographic key material or the remaining shares. The attack uses learned priors about natural image structures such as edges, textures, and semantic content to compensate for missing cryptographic information. The framework is evaluated across various benchmark image datasets using PSNR and SSIM as metrics for fidelity.

5.2 Preliminaries

Suppose a highly sensitive image — such as a medical scan or a classified photograph — needs to be stored across multiple servers. Storing the full image on any single server is risky: if that server is compromised, the secret is exposed. **Secret image sharing (SIS)** solves this by splitting the image into multiple pieces called **shares**, such that:

- Any sufficiently large subset of shares can *perfectly reconstruct* the original image.
- Any smaller subset of shares reveals *absolutely nothing* about the original image.

This is sometimes called a **threshold scheme**.

5.2.1 The (k, n)-Threshold Scheme

Formally, a (k, n)-threshold SIS scheme divides a secret image S into n shares:

$$\{s_1, s_2, \ldots, s_n\} \tag{5.1}$$

with the following guarantees:

- **Reconstruction**: Any k or more shares suffice to reconstruct S exactly.
- **Security**: Any collection of fewer than k shares is statistically independent of S — no information is leaked.

For example, in a $(3, 5)$ scheme, the secret is split into 5 shares distributed across 5 servers. Any 3 servers can collaborate to recover the secret, but any 2 servers (even acting together) learn nothing. This provides both *redundancy* (against server failure) and *security* (against server compromise).

5.2.2 Security Assumptions and Their Limitations

The information-theoretic security guarantee above assumes that shares are analyzed *only* through classical combinatorial means. However, modern deep learning models can sometimes extract partial information by detecting subtle statistical patterns across shares — an assumption that this chapter explicitly tests and challenges.

5.2.3 Autoencoders

An **autoencoder** is a neural network trained to reproduce its own input at the output. At first glance this seems trivial — why train a network to copy an input? The key insight is that the network is forced to pass the information through a **bottleneck**: a hidden layer with far fewer dimensions than the input. To reconstruct the input faithfully despite this bottleneck, the network is forced to learn a *compact, meaningful representation* of the data. Figure **??** illustrates the high-level encoder–bottleneck–decoder structure.

$$\texttt{Input} \xrightarrow{\texttt{Encoder}} \texttt{Latent Code} \xrightarrow{\texttt{Decoder}} \texttt{Reconstructed Output}$$

An autoencoder consists of five conceptual stages:

1. **Input Layer**: Receives the raw data (e.g., a flattened image or a sequence of token embeddings).

2. **Encoder**: A stack of layers that progressively compresses the input into a lower-dimensional space. Each layer applies a learned linear transformation followed by a nonlinear activation function.

3. **Latent Space** (Bottleneck): The compressed representation $\mathbf{z}$ of the input. This is the most information-dense part of the network. A well-trained encoder maps semantically similar inputs to nearby points in latent space.

4. **Decoder**: A symmetric stack of layers that expands the latent code back to the original input dimensionality.

5. **Output Layer**: Produces the reconstructed input $\hat{x}$.

Autoencoders are trained in an **unsupervised** fashion — no labels are required. The training objective is to minimize the **reconstruction error** between the input x and the output $\hat{x}$, typically measured by the Mean Squared Error (MSE):

$$\mathcal{L}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2 \tag{5.2}$$

where N is the number of elements (e.g., pixels). In the context of this chapter, autoencoders are used to learn mappings from image shares back to the original secret image, thereby assessing whether the information-theoretic security of SIS schemes holds under deep learning attacks.

5.2.4 Image Quality Evaluation Metrics

To evaluate how faithfully a reconstructed image matches the original, two standard metrics are used.

Peak Signal-to-Noise Ratio (PSNR)

PSNR is derived from the MSE between the original and reconstructed images. It is expressed in decibels (dB) and measures the ratio of the maximum possible signal power to the power of the distortion (noise):

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{peakval}^2}{\text{MSE}} \right) \tag{5.3}$$

where peakval is the maximum pixel value (255 for 8-bit images). A **higher PSNR** means the reconstruction is closer to the original. Typical values above 30 dB are considered good quality, while values below 20 dB indicate significant degradation.

Structural Similarity Index Measure (SSIM)

MSE and PSNR treat each pixel independently, which does not always align with human visual perception. **SSIM** addresses this by comparing images along three perceptual dimensions simultaneously: *luminance* (overall brightness), *contrast* (variation in brightness), and *structure* (spatial patterns of pixel intensities). The result is a score in the range $[0, 1]$:

- A value of **1** indicates the two images are structurally identical.
- A value of **0** indicates no structural similarity whatsoever.

SSIM is preferred over PSNR when perceptual quality is important, since two images can have the same MSE but look very different to a human observer.

5.3 Literature Review

The concept of secret image sharing was first motivated by the development of classical secret sharing schemes by Shamir [83], which laid a theoretical foundation for secret sharing among multiple parties. Naor and Shamir [84] later extended this idea to visual cryptography for images. The main idea is to encode a secret image into a number of shares such that any qualified set of shares will reveal the secret image visually. However, any unqualified subset will reveal nothing. The problem of visual cryptography is that there is a loss in contrast and resolution.

To address this problem, XOR-based secret image sharing schemes have been proposed. The XOR-based secret image sharing schemes operate on image pixels instead of bits. The XOR-based secret image sharing schemes use Shannon's perfect secrecy concept [85] to guarantee perfect reconstruction of the secret image from all the shares. The XOR-based secret image sharing schemes also guarantee that any subset of shares is statistically independent of the secret image. This ensures that there is zero mutual information between any unqualified subset of shares and the secret image.

However, the security proofs for the XOR-based constructions assume the presence of adversaries without access to structural priors about natural images. Recent

deep learning advances contradict this. Convolutional Neural Networks (CNNs) [86] have shown impressive performance in learning hierarchical spatial representations from image data. Architectural innovations such as batch normalization [87] improve the stability of the learning process, while encoder-decoder architectures, such as the U-Net [88], enable accurate reconstruction for inverse image problems. Autoencoders, when learned using pixel-wise reconstruction objectives, can extract compact feature representations that can reconstruct structured image information from corrupted images. Moreover, the use of Perceptual Loss functions [89] can improve the reconstruction quality for images. The reconstruction quality for such images can be measured using full-reference image similarity measures. The Peak Signal-to-Noise Ratio (PSNR) can measure the pixel-wise fidelity of the reconstruction, while the Structural Similarity Index Measure (SSIM) [90] can measure the structural similarity between images by comparing the luminance, contrast, and structural information.

In spite of the solid theoretical foundation of secret sharing via XOR and the remarkable advancements in the field of image reconstruction via deep learning methods, only a few works have attempted to explore the possibility of reconstructing the secret image via learning the priors of images and utilizing only certain subsets of the shares generated via XOR operations. Most of the existing works have been dedicated to enhancing the efficiency and robustness of secret image sharing methods and verifying their feasibility rather than testing their robustness against learning-based attacks. This work attempts to bridge this gap and test the robustness of secret image sharing via XOR operations against learning-based attacks and determine whether the secret image can be reconstructed via learning the priors of images and utilizing only certain subsets of the shares generated via XOR operations.

5.4 Work Done

A $(4, 4)$ conventional random XOR secret sharing algorithm was applied to all cases. Three shares are created randomly based on independent Bernoulli sampling, while the fourth share is obtained by XOR operation between the binarized secret image and the previous three shares.

In mathematical terms, let S be the secret image converted into binary, and R_1,

R_2, and R_3 be independent random shares. The fourth share R_4 is generated by the following equation:

$$R_4 = S \oplus R_1 \oplus R_2 \oplus R_3. \tag{5.4}$$

This guarantees that:

$$S = R_1 \oplus R_2 \oplus R_3 \oplus R_4. \tag{5.5}$$

According to traditional cryptographic theory [85], a combination of shares that is less than four shares will lack the sufficient information required to reconstruct the secret image. All shares are dynamically generated during training to avoid memorization. Figure 5.1 illustrates the complete attack pipeline used in this work.

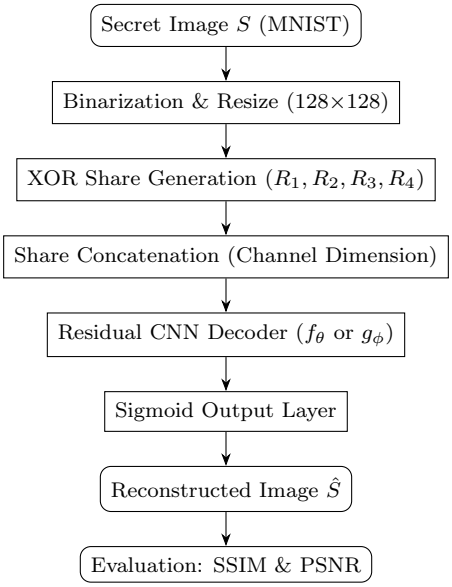


Figure 5.1: Autoencoder Based Attack Pipeline on XOR Secret Sharing Scheme.

5.4.1 Dataset Preparation

MNIST [86] dataset is used for training and testing purposes. The images were first normalized between $[0, 1]$ and then resized to 128×128 pixels, and are fed into the system as grayscale images. The shares are generated on-the-go during training to avoid memorizing of specific share-image pairs.

5.4.2 Learning-Based Reconstruction Framework

Two distinct decoder configurations were investigated.

Configuration 1: Decoder Trained on All Four Shares

The neural decoder network is trained on all four shares combined along the channel axis to produce a four-channel image tensor of dimensions $128 \times 128 \times 4$. This training involves learning the following transformation:

$$f_\theta : \mathbb{R}^{128\times128\times4} \to \mathbb{R}^{128\times128}. \tag{5.6}$$

The loss function employed for training is binary cross-entropy, and the model is trained for 20 epochs using the Adam optimizer. The trained model is tested for various scenarios involving different shares, where the missing shares are filled with zeros.

5.4.3 Configuration 2: Decoding with Three Consecutive XOR Shares (Independently Trained Model)

Another autoencoder was trained specifically for this configuration, using only three consecutive XOR shares as inputs. As opposed to Configuration 1, in which the decoder was trained with all four shares, here the fourth share is not even seen during training. Therefore, the learning task becomes different because now the model is required to learn to reconstruct from an incomplete input distribution.

Training of this decoder takes place using only three shares, where no wraparound is employed. Training is done independently from the four-shares decoder, meaning that no weights are shared. The possible pairs of shares for the training are: $\{1, 2, 3\}$ and $\{2, 3, 4\}$. Three shares are stacked to produce a 3-channel matrix of dimensions $128 \times 128 \times 3$, and the training takes place by learning the mapping:

$$g_\phi : \mathbb{R}^{128\times128\times3} \to \mathbb{R}^{128\times128}. \tag{5.7}$$

The same convolutional residual architecture, optimizer, and loss are used as in Configuration 1.

5.4.4 Decoder Architecture

For both configurations, we use a convolutional neural network with residual connections. The CNN includes an initial convolution layer for low-level feature extraction, followed by several blocks that utilize residual connections in order to conserve spatial information and facilitate learning. Additionally, the network uses batch normalization [87], dropout, and a sigmoid activation on the output node that produces the reconstructed image.

5.5 Results and Analysis

In this part, a thorough analysis is presented for the proposed learning-based reconstruction attack in scenarios with different levels of access to the XOR shares. Two cases are considered: (i) training of the decoder using all four XOR shares and testing in full and partial input scenarios (Figs. 5.2(a)-(d)), and (ii) training of the decoder using only three consecutive XOR shares (Figs. 5.2(e)-(f)).

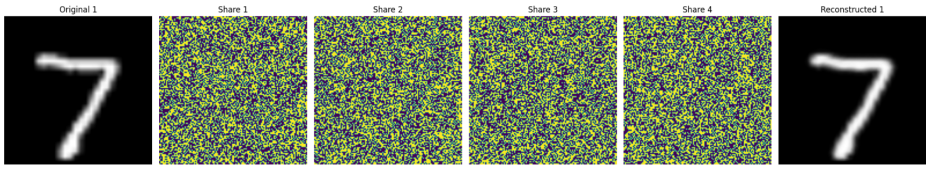
Reconstruction quality is measured both by numerical means using SSIM and PSNR values, and also visually using Fig. 5.2. Each image number is associated with each particular MNIST digit class, such as Image 1 for the digit class "1". Tables provide data for the digit classes 1–5 for conciseness, although all results were actually computed for all ten MNIST digit classes (0–9). The sample reconstructions of Fig. 5.2 refer to the digit class "7".

5.5.1 Configuration 1: Decoding with All Four XOR Shares

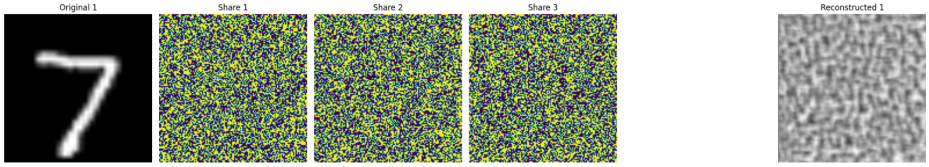
Case (a): Evaluation with All Four Shares

Table 5.1 presents reconstruction quality when all four XOR shares are available during testing. The visual result for the digit "7" is shown in Fig. 5.2(a).

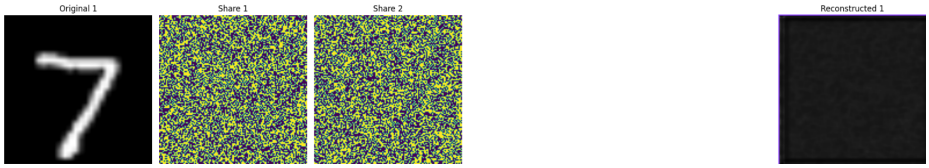
The reconstructions are highly similar to the input, with global structure and fine details being preserved. The high values for SSIM and PSNR suggest that the neural network successfully reconstructs an input without loss, which is very similar to classical XOR decoding. This finding supports previous research, which shows



(a) All 4 Shares

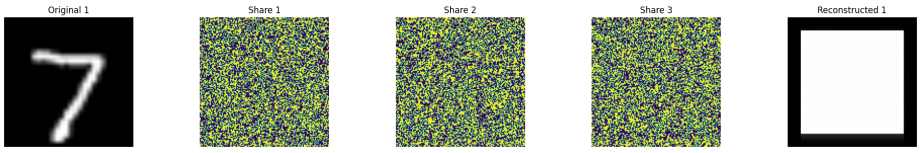


(b) 1 Share Missing

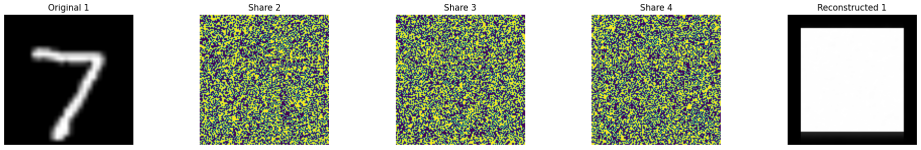


(c) 2 Shares Missing

(d) 3 Shares Missing



(e) Shares {1,2,3}



(f) Shares {2,3,4}

Figure 5.2: Visualized outcomes of the MNIST digit '7' for different settings of the XOR sharing. The subplots (a)-(d) are generated from the decoder model trained on all four shares (Setting 1). The subplots (e)-(f) are generated from the decoder model trained on only three consecutive shares (Setting 2).

Table 5.1: Reconstruction Quality Using All Four XOR Shares.

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.9790	34.22
Image 2	0.9652	31.92
Image 3	0.9882	37.47
Image 4	0.9651	33.45
Image 5	0.9626	32.28
Average	$\approx$ **0.9720**	$\approx$ **33.87**

that residual CNNs can decode structured linear transformations if enough input is present.

Case (b): One Share Missing

Table 5.2 reports reconstruction quality when one of the four XOR shares is withheld and replaced by a zero-valued channel. The corresponding visual output for the digit "7" is shown in Fig. 5.2(b).

Table 5.2: Reconstruction Quality with One XOR Share Missing.

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.0020	6.28
Image 2	0.0057	6.40
Image 3	0.0028	6.05
Image 4	0.0129	6.41
Image 5	0.0022	6.28
Average	$\approx$ **0.0042**	$\approx$ **6.28**

Since one of the shares is missing, reconstruction becomes impossible. The SSIM is very close to 0 (even being slightly less than zero for certain classes), whereas PSNR only slightly exceeds the noise level. The output of the decoding process does not make sense and bears no structural similarity to the original digits.

Case (c): Two Shares Missing

Table 5.3 reports reconstruction quality when two of the four XOR shares are withheld. The visual result for the digit "7" is shown in Fig. 5.2(c).

Table 5.3: Reconstruction Quality with Two XOR Shares Missing.

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.7396	11.84
Image 2	0.6422	9.56
Image 3	0.8428	14.81
Image 4	0.6106	8.17
Image 5	0.7120	11.76
Average	$\approx$ **0.7094**	$\approx$ **11.23**

Surprisingly, when two shares are missing, the SSIM values obtained are slightly higher (≈ 0.71). This unexpected behavior can be attributed to the structured manner of the XOR construction combined with assumptions made about the dataset: when two shares are missing, then the two remaining shares contain a linear combination of the secret and the decoder exploits statistical properties of MNIST images. However, this does not mean that the decoding results are high-quality; rather, the output is distorted and should not be considered a security threat.

Case (d): Three Shares Missing

Table 5.4 reports reconstruction quality when only one of the four XOR shares is available (three shares missing). The visual output for the digit "7" is shown in Fig. 5.2(d).

Table 5.4: Reconstruction Quality with Three XOR Shares Missing (Only One Share Available).

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.7396	11.84
Image 2	0.6422	9.56
Image 3	0.8428	14.81
Image 4	0.6106	8.17
Image 5	0.7120	11.76
Average	$\approx$ **0.7094**	$\approx$ **11.23**

The figures computed using three missing shares match those calculated using two missing shares. This is due to the symmetry inherent in the XOR design where the available shares contain equivalent amounts of partial information regarding the secret and utilize the same data set prior knowledge for decoding. The low SSIM

scores have no impact on the reliability of the images produced since these values are within expectations from the information-theoretic security bounds of the XOR protocol and other works involving neural network decoding with missing input data.

5.5.2 Configuration 2: Decoding with Three Consecutive XOR Shares

The decoder model was then trained only on subsets consisting of three sequential shares; specifically, shares $\{1, 2, 3\}$ and $\{2, 3, 4\}$ to investigate whether a model optimized for the small input scenario could identify patterns that would circumvent the security barrier. Figures 5.2(e) and 5.2(f) present the visualizations of results for digit "7". Tables 5.5 and 5.6 quantify the reconstruction performance for shares $\{1, 2, 3\}$ and $\{2, 3, 4\}$, respectively.

Table 5.5: Reconstruction Quality Using XOR Shares {1,2,3}.

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.1988	12.42
Image 2	0.2042	10.99
Image 3	0.2024	13.96
Image 4	0.2243	9.86
Image 5	0.2093	12.65
Average	$\approx$ **0.2078**	$\approx$ **11.98**

Table 5.6: Reconstruction Quality Using XOR Shares {2,3,4}.

Image (Digit)	**SSIM**	**PSNR (dB)**
Image 1	0.1989	12.42
Image 2	0.2046	10.99
Image 3	0.2027	13.96
Image 4	0.2242	9.86
Image 5	0.2094	12.65
Average	$\approx$ **0.2080**	$\approx$ **11.98**

Both sets of low SSIM (≈ 0.21) and PSNR (< 12 dB) scores suggest that the decoder trained on an incomplete set cannot reconstruct the image, irrespective of whether the first or second share is lost. The reason behind this is that the problem is under-determined since the lost entropy cannot be retrieved from the available data,

which is true for XOR-based secret-sharing schemes based on information-theoretic security.

5.6 Summary

This work examined the robustness of XOR-based visual secret sharing schemes under learning-based adversaries. Neural networks can approximate the reconstruction function when all four shares are available, achieving near-lossless results (SSIM = 0.9720, PSNR = 33.87 dB), showing that data-driven models can learn the full mapping when complete information is present.

However, reconstruction fails under missing-share conditions, where performance drops significantly and meaningful recovery becomes infeasible. This confirms that the XOR scheme retains its information-theoretic security against learning-based attacks with insufficient shares.

CHAPTER 6

Autoencoder-based Secret Image Sharing Scheme

This chapter presents an autoencoder-based secret image sharing framework that leverages deep learning to achieve efficient and secure image distribution through latent space compression. It introduces the architecture of encoder–decoder networks used to transform images into compact representations, which are then partitioned into multiple independent shares for secure distribution. The chapter details the design of the encoding, share generation, and reconstruction processes, followed by experimental evaluation demonstrating high reconstruction quality, strong confidentiality of individual shares, and significant storage reduction compared to traditional SIS schemes.

6.1 Introduction

With the rapid growth of deep learning, neural network-based approaches have become effective for image representation and compression. In the context of Secret Image Sharing (SIS), traditional methods rely on polynomial interpolation or logical operations, which often face limitations in storage efficiency and scalability. To address these issues, autoencoder-based approaches provide a data-driven alternative.

An autoencoder is a neural network architecture designed to learn compact latent representations of input data. It consists of two main components: an encoder that compresses the input image into a lower-dimensional latent vector, and a decoder that reconstructs the image from this representation. This property makes autoencoders suitable for reducing redundancy while preserving essential image features.

In this work, an autoencoder-based SIS framework is considered, where the encoder transforms the input image into a compressed latent space. This latent representation is then divided into multiple independent shares. These shares are distributed such that only a sufficient combination of them enables reconstruction. The decoder utilizes the combined latent information to recover the original image.

Unlike conventional SIS schemes, this approach achieves significant reduction in share size due to learned compression, while maintaining high reconstruction quality. Experimental results show that the method preserves structural similarity and minimizes information loss, demonstrating its effectiveness for secure and storage-efficient image sharing.

This highlights the potential of integrating deep learning techniques, specifically autoencoders, into SIS systems to improve both efficiency and reconstruction performance.

6.2 Preliminaries

Autoencoders are a type of artificial neural network used for unsupervised learning. Their objective is to learn a compressed representation of input data and reconstruct the original data from this representation. The architecture of an autoencoder consists of the following components:

1. **Input Layer:** Receives the original data such as image pixels or feature vectors.
2. **Encoder:** Transforms the input into a lower-dimensional representation using a sequence of linear operations and nonlinear activation functions (e.g., ReLU, sigmoid). This step reduces dimensionality while retaining important features.
3. **Latent Space (Bottleneck Layer):** Represents the compressed form of the input data. Based on its size:
    - **Undercomplete Autoencoder:** Latent dimension $<$ input dimension, enforcing compression.
    - **Overcomplete Autoencoder:** Latent dimension $>$ input dimension, allowing richer representations.
4. **Decoder:** Reconstructs the original input from the latent representation using transformations similar to the encoder.
5. **Output Layer:** Produces the reconstructed data. The network is trained to minimize reconstruction error, commonly using Mean Squared Error (MSE).

6.3 Literature Review

XOR-based Secret Image Sharing (SIS) schemes have been widely studied due to their low computational complexity and ease of implementation. These methods rely on simple Boolean operations, making them suitable for real-time and resource-constrained applications.

Wang et al. [91] proposed a basic (n, n) SIS scheme where $(n-1)$ random matrices are generated and the final share is computed using XOR operations with the secret image. The reconstruction is performed by XORing all shares. This method avoids pixel expansion and preserves image quality across binary, grayscale, and color images. However, the size of each shadow image remains equal to the original image.

Chattopadhyay et al. [92] extended XOR-based SIS by incorporating hash-based verification to detect tampered shares before reconstruction. Although this improves integrity, it does not address the issue of storage overhead, as share sizes remain unchanged.

Chen and Wu [93, 94] introduced multiple secret image sharing (MSIS) schemes using Boolean operations. Their approach allows multiple images to be encoded into a single set of shares. Later improvements added randomization to prevent information leakage when insufficient shares are available. Despite these enhancements, the share size remains equal to the original image size.

Nag et al. [95] proposed a Boolean-based MSIS scheme where share size can increase depending on the number of secrets, leading to higher storage requirements. Lin et al. [77] developed a key-free SIS scheme using a Cycling-XOR mechanism, embedding shares into cover images for improved concealment. However, the share size is still not reduced.

To address storage inefficiency, Ou et al. [78] proposed a scheme based on block truncation coding and error diffusion, achieving significant reduction in share size. Similarly, Purushothama B. R. [96] introduced a method that reduces shadow image dimensions to $\lceil h/n \rceil \times w$, improving storage efficiency compared to traditional XOR schemes.

Further advancements include extending XOR-based methods to multi-secret scenarios. Kabirirad and Eslami [75] proposed a (t, n) threshold MSIS scheme using

Boolean operations, enabling flexible reconstruction. However, the method depends on sequential share availability and does not reduce share size. Faraoun [76] proposed a secure MSIS scheme combining stream ciphers and XOR-based key sharing, but the generated shares remain equal in size to the original images.

Overall, while XOR-based SIS schemes offer simplicity and efficiency, most approaches suffer from the limitation of large shadow image sizes. This motivates the exploration of alternative techniques, such as learning-based compression methods, to improve storage efficiency without compromising security.

6.4 Work Done

An autoencoder-based Secret Image Sharing (SIS) scheme is implemented using a deep convolutional architecture for grayscale images. The system encodes an input image into a compact latent representation and divides it into multiple independent shares. Each share contains partial information, and reconstruction is possible only when all shares are combined. Figure 6.1 illustrates the complete framework.

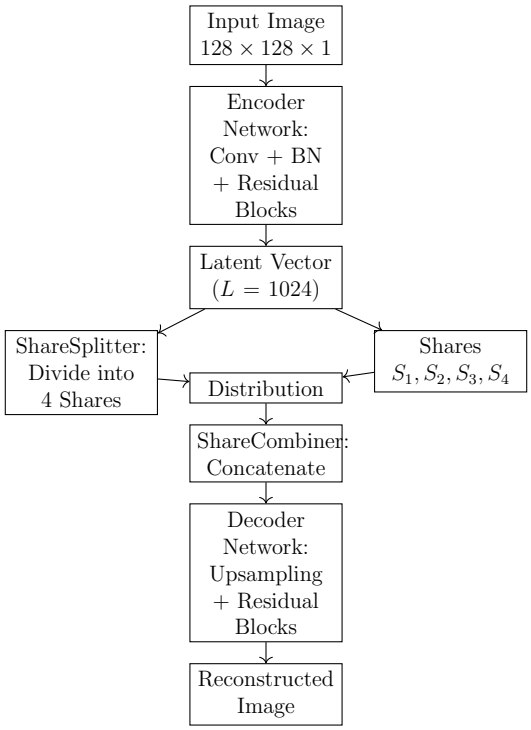


Figure 6.1: Autoencoder-Based Secret Image Sharing Framework

6.4.1 Encoder Network

The encoder processes an input image of size $128 \times 128 \times 1$ and converts it into a latent vector of size $L = 1024$.

- **Convolutional Downsampling:** Three convolutional layers with filters $(32, 64, 128)$ and stride 2 are used to reduce spatial dimensions and extract hierarchical features. Each layer uses ReLU activation followed by Batch Normalization.
- **Residual Blocks:** Residual connections are used to preserve features and stabilize training. Each block follows:

$$\text{Output} = \text{ReLU}(x + F(x)) \tag{6.1}$$

  where $F(x)$ represents convolutional transformations.

- **Latent Representation:** The final feature maps are flattened and passed through a fully connected layer to produce a 1024-dimensional latent vector.

6.4.2 Share Generation

The latent vector is divided into four equal shares using a non-trainable splitting operation:

$$S_i = \text{latent}\left[(i-1)\frac{L}{4} : i\frac{L}{4}\right], \quad i = 1, 2, 3, 4 \tag{6.2}$$

Each share is independent, and fewer than four shares do not provide sufficient information for reconstruction.

6.4.3 Decoder Network

The decoder reconstructs the original image by combining all shares and reversing the encoding process.

- **Share Combination:** The shares are concatenated to reconstruct the latent vector:

$$\text{latent}_{recon} = \text{concat}(S_1, S_2, S_3, S_4) \tag{6.3}$$

- **Feature Upsampling:** The latent vector is expanded into a $16 \times 16 \times 128$ feature map and upsampled using transposed convolution layers to restore spatial resolution.
- **Residual Refinement:** Residual blocks are used to refine reconstructed features and improve image quality.
- **Output Layer:** A final transposed convolution layer with sigmoid activation produces the reconstructed grayscale image in the range $[0, 1]$.

6.4.4 Training Objective

The model is trained end-to-end using Mean Squared Error (MSE) loss:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (I_i - \hat{I}_i)^2 \tag{6.4}$$

where I_i and $\hat{I}_i$ represent original and reconstructed pixel values. The network is optimized using the Adam optimizer with learning rate $\alpha = 10^{-3}$, batch size 32, and trained for 20 epochs.

6.5 Results and Analysis

This section presents the experimental evaluation of the autoencoder-based Secret Image Sharing (SIS) scheme. The model is evaluated on grayscale images using reconstruction quality, share security, and storage efficiency as primary criteria.

6.5.1 Dataset and Training

The model is trained on the MNIST dataset, where images are resized to $128 \times 128 \times 1$ and normalized to $[0, 1]$. A total of 60,000 images are used for training and 10,000 for testing. The network is trained for 20 epochs using Mean Squared Error (MSE) loss and the Adam optimizer. The training and validation losses show consistent convergence, reaching a minimum validation loss of approximately 7.5×10^{-5}, which indicates high reconstruction accuracy.

6.5.2 Reconstruction Performance

The reconstructed images are visually close to the original inputs, with only minor pixel-level deviations. Quantitatively, the reconstructed outputs achieve PSNR values greater than 30 dB, indicating high-quality reconstruction, while SSIM values remain close to 1.0, confirming strong structural similarity. These results demonstrate that the autoencoder effectively preserves important image features during encoding and decoding.

6.5.3 Share Security Analysis

Each latent vector is divided into four independent shares. The individual shares exhibit very low similarity to the original image, with SSIM values approximately in the range of 0.05 to 0.1 and PSNR values below 10 dB. This behavior confirms that individual shares do not reveal any visible or structural information about the original image, thereby ensuring strong confidentiality.

6.5.4 Storage Efficiency

The original image size is $128 \times 128 = 16384$ pixels, which corresponds to approximately 64 KB in float representation. The latent representation reduces this to 1024 values, resulting in a total size of approximately 1 KB for all shares combined. Each individual share occupies approximately 0.25 KB. This corresponds to an overall storage reduction of about 98%, demonstrating a significant improvement compared to traditional SIS schemes where share sizes are equal to the original image.

6.5.5 Analysis Summary

The proposed method achieves high reconstruction quality with strong structural similarity, ensures security by preventing information leakage from individual shares, and significantly reduces storage requirements. However, the reconstruction is not perfectly lossless due to compression in the latent space, which introduces minor smoothing artifacts in the reconstructed images. Figure 6.2 shows the visual reconstruction results alongside the generated shares.

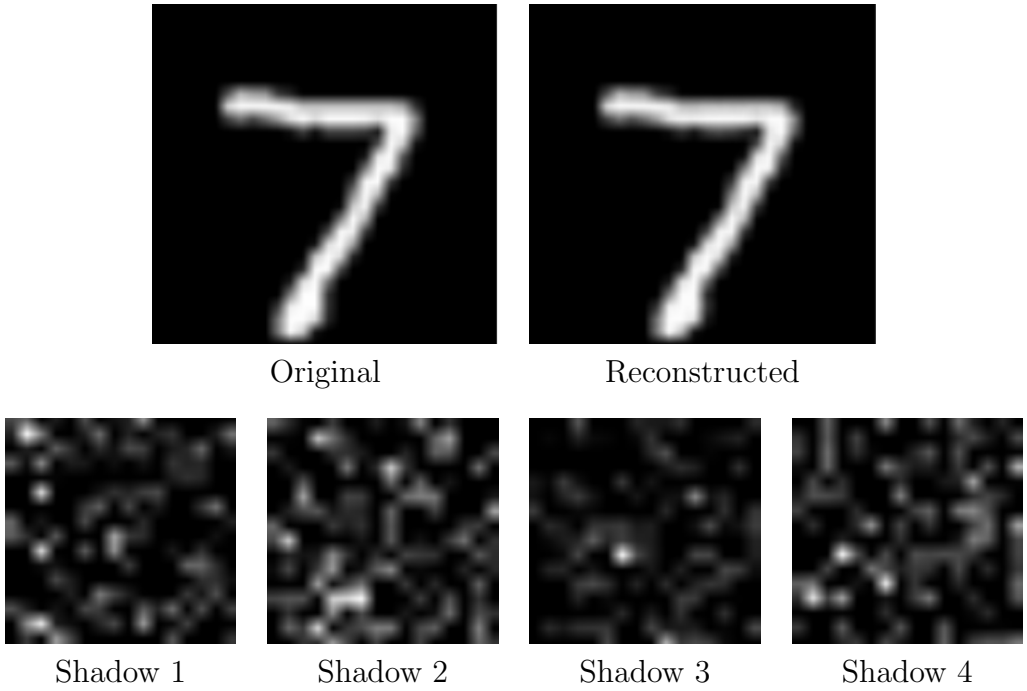


Figure 6.2: Autoencoder-based SIS results showing original image, reconstructed image, and generated shares

6.6 Summary

This work presents an autoencoder-based Secret Image Sharing (SIS) scheme that leverages deep learning for secure and storage-efficient image distribution. The proposed method encodes images into a compact latent representation and divides it into multiple independent shares, ensuring that no single share reveals meaningful information about the original image.

The model achieves high reconstruction quality (PSNR $> 30\,\mathrm{dB}$, SSIM ≈ 1.0), while individual shares remain dissimilar to the original, ensuring strong confidentiality. It also reduces storage by up to 98%.

However, the method introduces minor reconstruction loss due to the lossy nature of latent space compression. Despite this, the overall performance indicates that autoencoder-based SIS is an effective alternative to traditional schemes, offering a balance between security, compression, and reconstruction quality.

CHAPTER 7

Optimization Techniques for Secret Image Sharing Schemes

This chapter presents a comprehensive study of optimization techniques for secret image sharing schemes, focusing on reducing shadow image size while preserving security and lossless reconstruction. It reviews polynomial-based, XOR-based, and multi-secret sharing approaches, and introduces preprocessing strategies such as image differencing, Huffman coding, leading-zero removal, and digit decomposition to improve storage efficiency. The chapter also explores the integration of pooling-based methods for enhanced compression, followed by experimental evaluation demonstrating the trade-offs between compression, reconstruction quality, and computational efficiency across various schemes.

7.1 Introduction

With the growth of multimedia communication, images have become a critical carrier of information, whether for personal use or for scientific purposes. In some industries and organizations, it is preferred to store sensitive and confidential data in the form of images. Protecting these images against unauthorized access and tampering has therefore become an important research problem. Several image protection techniques, such as encryption, digital watermarking, visual cryptography, and steganography, have been successful in doing so. However, all of these techniques are dependent on centralized storage as the single information carrier. If the carrier is intercepted or destroyed, it is possible that the image is lost, then the sensitive information is also lost, and hence cannot be recovered. To prevent such occurrences, secret image sharing schemes are in practice, which instead rely on decentralized storage.

Secret Sharing (SS) provides a cryptographic mechanism in which a secret is divided into multiple shares and distributed among a number of participants. The secret can only be reconstructed when a threshold number of shares is combined, while having fewer than the threshold does not reveal any information about the se-

cret. In secure secret sharing schemes, an attacker that gains access to fewer shares of the secret than defined by the threshold gains no information about the secret. Secret sharing schemes are useful because they allow for more secure storage of highly sensitive data, including encryption keys, missile launch codes, and numbered bank accounts. By distributing the data, there is no single point of failure that can lead to its loss.

The concept of secret sharing was first introduced independently by Shamir [83] and Blakley [97] in 1979. Shamir's method, known as the threshold sharing scheme (TSS), is based on polynomial interpolation over a finite field, while Blakley's scheme relies on hyperplane geometry. Over time, researchers have proposed several variants of TSS, and the concept is now commonly called the (t, n) threshold scheme, where t is the threshold number of shares required to reconstruct the secret image from the n total shares.

With the rapid growth of digital media and network-based communication, the need arose to adapt secret sharing techniques for images. To address this, Thien and Lin [98] proposed the first Secret Image Sharing (SIS) scheme in 2002. Their method was directly based on Shamir's (t, n) scheme and allowed an image to be split into n shadow images such that any t of them could perfectly reconstruct the original image. However, there was always a scope to minimize the size of the output images as much as possible. To overcome these limitations, Wang and Su [99] introduced a novel SIS pipeline in 2006 that significantly reduced the storage size of shadow images. Their scheme incorporated image preprocessing steps, including an image differencing operation followed by Huffman coding, before applying the polynomial-based sharing mechanism. These enhancements successfully reduced the shadow image sizes by approximately 40% compared to Thien and Lin's scheme.

The XOR-based SIS approach is one of the most widely adopted and computationally efficient categories of Boolean-based SIS schemes. Despite its simplicity and efficiency, two major limitations persist in existing XOR-based schemes. The first concern the inability to effectively reduce the size of the generated shadow images, while the second pertains to the limited availability of flexible (t, n) threshold constructions, as most existing methods are confined to (n, n) configurations. To address the first limitation, we introduce a lightweight image sharing framework based on av-

erage pooling and XOR-based shadow generation. The second limitation is addressed by implementing the Kabirirad–Eslami (t, n) Multi-Secret Image Sharing Scheme [75], along with approaches to optimize the size of shadow images.

Secret Image Sharing (SIS) is an important research problem due to the need to protect sensitive images from unauthorized access and distribution. With the increasing use of digital images in various applications, including social media, e-commerce, and entertainment, the need for secure and efficient methods of image sharing is becoming more critical. Sensitive images, such as medical images or scanned copies of government documents, images of military installations, satellite images, court evidence, etc., are prone to illegal access, tampering, and distribution.

Although existing methods implement the scheme efficiently, there is always a scope to improve the efficiency of storage by minimizing the size of the share images. In addressing this challenge of reducing the size of the generated share images, it is also necessary not to compromise security and image quality. This creates an open space for further advancements in SIS algorithms that focus on generating smaller, storage-efficient shadow images and enhancing storage optimization while maintaining robust security of the secret.

7.2 Preliminaries

This chapter presents the foundational concepts and tools that form the basis for this research.

7.2.1 Lagrange Interpolation

We consider the problem of approximating a given function by a class of simpler functions, mainly polynomials. One of the main uses of interpolation is reconstructing the function $f(x)$ when it is not given explicitly and only the values of $f(x)$ and/or its certain order derivatives at a set of points, called nodes are known.

A polynomial $p(x)$ is called an interpolating polynomial if the value of $p(x)$ and/or its certain order derivatives coincides with those of $f(x)$ and/or its same order derivatives at one or more tabular points.

In general, if there are $N+1$ distinct points:

$$a = x_0 < x_1 < x_2 < \cdots < x_n = b \tag{7.1}$$

Then the problem of interpolation is to obtain $p(x)$ satisfying the conditions:

$$p(x_i) = f(x_i), \quad i = 0, 1, 2, \ldots, n \tag{7.2}$$

Substituting these conditions, we obtain the system of equations:

$$\begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = f(x_0) \\ a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_n x_1^n = f(x_1) \\ \quad \vdots \\ a_0 + a_1 x_n + a_2 x_n^2 + \cdots + a_n x_n^n = f(x_n) \end{cases} \tag{7.3}$$

This system of equations has a unique solution. The interpolation nodes are given as:

$$\begin{array}{ll} x_0 & f(x_0) = f_0 \\ x_1 & f(x_1) = f_1 \\ \vdots & \\ x_N & f(x_N) = f_N \end{array} \tag{7.4}$$

There exists only one Nth degree polynomial that passes through a given set of $N+1$ points. Its form is expressed as a power series:

$$g(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_N x^N \tag{7.5}$$

where $a_i \in \mathbb{R}$ are the unknown coefficients, $i = 0, 1, 2, \ldots, N$.

7.2.2 Huffman Coding

Huffman coding is a commonly used method for lossless data compression. It ensures that no information is lost during encoding or decoding and efficiently encodes data as binary code. The main idea is to use variable-length codes. The length of the binary code for each symbol depends on how often it appears such that frequently

used symbols get shorter codes, while less common ones get longer codes.

Construction of Huffman Tree

The Huffman coding process starts by finding the frequency of each symbol in the data. Each symbol is initially a separate node. The two nodes with the smallest frequencies are combined into a new node. The frequency of this node is the sum of the two. This is repeated until a single tree is formed, known as the Huffman tree.

Once the tree is constructed, binary codes are assigned by traversing the tree:

- Assign a '0' to each left branch and a '1' to each right branch.
- The binary code for a symbol is obtained by tracing the path from the root to the corresponding leaf node.

Average Code Length

The effectiveness of Huffman coding can be measured by the average codeword length, which is calculated using the formula:

$$L_{avg} = \frac{1}{f(T)} \sum_{i=1}^{n} d(i) \cdot f(i) \tag{7.6}$$

where:

- n is the number of characters,
- $f(T)$ is the total frequency of all characters,
- $f(i)$ is the frequency of character i,
- $d(i)$ is the length of the codeword for character i.

Huffman coding is widely used in data compression techniques like ZIP files, JPEG image compression, and MP3 audio encoding. It is a key part of many modern compression methods because of its simplicity and effectiveness.

7.2.3 Galois Field

A Galois Field (GF), also known as a finite field, is a field with a finite number of elements. For every prime number p and a positive integer n, there exists exactly one finite field with p^n elements, denoted $\mathrm{GF}(p^n)$. In general, a field with q elements is represented as $\mathbb{F}_q$. Galois Fields play a fundamental role in coding theory, cryptography, and various areas of modern computing.

Definition: A finite field $\mathrm{GF}(p^n)$ consists of p^n elements, where:

- p is a prime number, known as the characteristic of the field.
- n is a positive integer representing the degree of the field extension.

Key Properties:

1. For every finite field $\mathbb{F}_q$, each element $a \in \mathbb{F}_q$ satisfies:

$$a^q = a \tag{7.7}$$

2. The non-zero elements of a finite field form a multiplicative group of order $q-1$.
3. Every finite field of order p^n is isomorphic to the splitting field of the polynomial $x^q - x$ over $\mathbb{F}_p$.

Subfields: If $\mathbb{F}_q$ is a finite field with $q = p^n$ elements, then it has exactly one subfield with p^m elements, where m is a positive divisor of n. This unique subfield consists precisely of the roots of the polynomial $x^{p^m} - x$ over $\mathbb{F}_p$.

Galois Fields are extensively used in secure communication systems, such as secret sharing schemes, due to their strong mathematical structure and predictable behavior. They provide the foundation for operations in polynomial-based secret sharing, error-correcting codes, and cryptographic algorithms.

7.2.4 Threshold Secret Sharing (TSS) Scheme

A Threshold Secret Sharing (TSS) scheme consists of the following primary entities:

- **Secret:** A secret S is the data that must be shared between a group of participants.

- **Shares / Shadows:** The secret S is encoded into n shares / shadows, say $s_1, s_2, \ldots, s_n$, so that any of these shares alone cannot disclose any information about the secret.
- **Dealer:** The dealer D is responsible for encoding the secret into n shares/shadows and allocating these shares to the participants such that each participant receives exactly one share/shadow.
- **Participants:** The set $\mathcal{P} = \{P_i\}_{i=1}^{n}$ is used to represent the participants, who obtain the shares from the dealer.
- **Combiner:** The combiner C is responsible for decoding the secret if an approved subset of participants submits their own shares.

7.2.5 Polynomial-based Secret Image Sharing

Shamir's (k, n)-SIS

Secret sharing: The secret data is divided into n different shares. In order to ensure that any k or more shares can recover the secret data, Shamir constructed a polynomial of degree $k-1$ as follows:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{k-1} x^{k-1} \tag{7.8}$$

where a_0 represents the secret data, and $a_1, a_2, \ldots, a_{k-1}$ denotes any $k-1$ integers chosen randomly. Take n different $x_1, x_2, \ldots, x_n$, calculate $f(x)$ using Eq. (2.1), and then distribute the resulting values $f(x_1), f(x_2), \ldots, f(x_n)$ to n participants as the shares.

Secret recovery: Choose any k points $(x_1, f(x_1)), (x_2, f(x_2)), \ldots, (x_k, f(x_k))$ and use Lagrange interpolation to recover secret data a_0. The interpolation polynomial is given as:

$$f(x) = \sum_{i=1}^{k} f(x_i) \prod_{\substack{1 \le j \le k \\ j \ne i}} \frac{x - x_j}{x_i - x_j} \tag{7.9}$$

Here, the constant term of the polynomial $f(0) = a_0$ represents the recovered secret data.

Thien and Lin's (k, n)-SIS

Image sharing: The secret image S is divided into several non-overlapping blocks of equal size, with each block containing k pixels. The values of these k pixels are taken as the coefficients of a polynomial $f(x)$, as shown in Eq. (2.3):

$$f(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_{k-1} x^{k-1} \tag{7.10}$$

Let $x = 1, 2, \ldots, n$, then compute $f(x)$ for each x. Here, $f(x)$ represents the pixel value of the share image Share_i. When all blocks are processed, n shares are obtained, each with a size of $\frac{1}{k}$ of the secret image S. The pairs $(1, \text{Share}_1), (2, \text{Share}_2), \ldots, (n, \text{Share}_n)$ are distributed to the n participants.

Image recovery: Select any k shares $(1, \text{Share}_1), (2, \text{Share}_2), \ldots, (k, \text{Share}_k)$ and apply Lagrange interpolation to calculate polynomial coefficients:

$$p_j = \sum_{i=1}^{k} \text{Share}_i \prod_{\substack{1 \le m \le k \\ m \ne i}} \frac{x_j - x_m}{x_i - x_m} \tag{7.11}$$

These coefficients $p_0, p_1, \ldots, p_{k-1}$ correspond to the values of pixels within a block of the reconstructed image. Once all pixels across the shares are processed, the recovered image S is obtained.

The adjacent pixels of an image typically exhibit a high correlation. Therefore, the secret image S must be transformed before applying polynomial sharing to avoid inadvertent disclosure of sensitive information.

7.3 Literature Review

Over the years, researchers have explored various SIS schemes, including polynomial-based and XOR-based methods, to improve both security and efficiency. Recent studies have focused on reducing the size of generated shadow images to optimize storage and bandwidth usage while maintaining lossless reconstruction. This literature review examines the evolution of SIS schemes, highlighting key advancements in share generation techniques and preprocessing methods aiming to improve perfor-

mance and applicability.

In their comprehensive survey, Saha et al. [100] categorize SIS approaches into several paradigms, including polynomial-based and XOR-based schemes, each with distinct trade-offs.

Polynomial interpolation-based: Polynomial-based SIS schemes extend Shamir's Threshold Secret Sharing (TSS) [83], where pixel values or groups of pixels are treated as secret data points and embedded into a polynomial over a finite field. It used the idea of Lagrange interpolation that any t of n shares can reconstruct the polynomial, while fewer than t shares reveal no information. Thien and Lin [98] utilized Shamir's scheme to develop a (t, n)-threshold SIS scheme that reduces the share size to $\frac{1}{t}$ of the original image size. An initial permutation step was carried out to decorrelate adjacent pixel values to improve security. The initial scheme they developed was lossy. To overcome this, a lossless variant was proposed, but at the cost of an increase in the share size to $\frac{1}{t-1}$.

Polynomial CRT-Based Schemes: Some researchers have integrated the Chinese Remainder Theorem (CRT) with polynomial-based secret sharing to further reduce computational overhead and improve reconstruction accuracy. CRT-based SIS schemes [101] replace modular arithmetic over large finite fields with residue computations over smaller coprime moduli. Guo et al. [74] propose a multi-threshold SIS scheme based on the generalized CRT in which each shadow image remains the same size as the original secret image. Deshmukh et al. [71] introduced a novel approach for sharing multiple color images employing the Chinese Remainder Theorem (CRT), such that the shadow images remain the same size as the original images. This approach enables faster computations and more compact shadow images, especially when dealing with high-resolution grayscale or color images.

Progressive Secret Image Sharing (PSIS): Yan et al. in 2016 [102] introduced the Progressive SIS Scheme (PSIS), where the quality of the reconstructed image is proportional to the number of shares used to reconstruct. Prasetyo and Hsia [72] proposed an enhanced multiple secret sharing scheme using generalized chaotic image scrambling. Their approach combines XOR operations with hyperchaotic maps to enhance security and address the reconstruction issue for odd numbers of secrets, but the shadow images remain the same size as the original images. In parallel work,

Prasetyo and Hsia [73] developed lossless progressive secret sharing schemes for both grayscale and color images, enabling incremental quality recovery as more shares become available, while maintaining shadow image sizes equal to the original image dimensions.

Basic XOR Model: XOR-based SIS schemes emerged as lightweight and computationally efficient alternatives to general techniques. Wang et al. [66] proposed an (n, n) SIS method where the dealer generates $(n-1)$ random matrices and computes the nth share as the XOR of the last random matrix and the secret image. The secret can be revealed by XORing all n shares. This scheme eliminates pixel expansion, maintains perfect contrast, and supports binary, grayscale, and color images. However, the shadow image size remains the same as the original image. Chattopadhyay et al. [67] extended this approach by introducing hash-based verifiability into (n, n) secret image sharing schemes. Their method ensures that participants can detect tampered or forged shares before reconstruction. While they claim flexibility in share size when the number of shares is increased, the shadow images still remain equal in size to the original image in standard configurations.

Chen and Wu [69] proposed a multiple secret image sharing scheme (MSIS) based on Boolean operations that allows several secrets to be encoded into the same set of shares. The shadow image size remains the same as that of the original image. Building on this foundation, Chen and Wu [68] incorporated additional randomization to prevent partial information leakage when fewer than the required number of shares are available, though the shadow image size remains equal to the original image size. Nag et al. [70] proposed a Boolean-based MSIS scheme in which each share can grow to $t \times r \times c$ (i.e., multiples of the original $r \times c$ size) depending on the number of secret images and matrix structure. Lin et al. [77] proposed a key-free secret image sharing scheme using an improved Cycling-XOR mechanism, where each share is embedded into a distinct cover image to enhance visual concealment and reduce suspicion. This approach also generated shadow images identical in size to the secret images.

Few studies have addressed the limitation of large shadow image sizes in XOR-based and Boolean-based sharing schemes. Ou et al. [78] propose a user-friendly secret-image sharing scheme based on block truncation coding and error diffusion, where each meaningful share is $\frac{1}{4}$ times smaller the original secret image. Purushothama

B. R. [79] introduced work that focuses on reducing the size of generated shadow images. Unlike traditional (n, n) XOR-based methods that produce shares of size $h \times w$, their approach generates shadow images of size $\lceil h/n \rceil \times w$, significantly improving storage efficiency.

XOR-Based Multi-Secret Image Sharing (MSIS): To increase efficiency, researchers have extended the XOR-based methods to share multiple secret images simultaneously. Chen and Wu [69] proposed a scheme in which multiple images are encoded into a single set of shadow images by systematically combining bits from different images using XOR operations. Kabirirad–Eslami [75] extended MSIS to support (t, n)-threshold structures using Boolean operations, providing flexibility where t out of n shares can reconstruct the secrets. However, this approach uses t consecutive shares to backtrack and retrieve the secrets, where missing even one breaks the chain, and the shadow image size remains the same as the original image. Faraoun [76] proposed a highly secure (n, n) multi-secret image sharing scheme based on cryptographic hash functions and stream ciphers. This approach encrypts the secret image using a stream cipher with a random session key, then splits the key using XOR chaining among n shares, generating shadow images identical in size to the secret images.

7.4 Work Done

This research explores various optimized secret image sharing (SIS) schemes with the aim of developing a methodology to reduce the size of shadow images. To achieve this, we initially implemented the classical polynomial-based scheme and the XOR-based scheme to gain a comprehensive understanding of their concepts and mechanisms. Building on this foundation, we introduce preprocessing steps that effectively reduce the net storage space, while maintaining the property of lossless reconstruction of the secret image.

7.4.1 Implementation of Polynomial-based SIS with Smaller Shadow Images

As part of this chapter, we implemented the methodology proposed in the paper "Secret Image Sharing with Smaller Shadow Images" by Ran-Zan Wang and Chin-Hui Su (*Pattern Recognition Letters*, 2006) [99].

The paper addresses the problem of protecting sensitive images by dividing them into multiple shadow images, such that the original image can only be reconstructed when a threshold number of these shadow images are combined. This approach offers higher fault tolerance and security compared to traditional centralized storage mechanisms such as encryption or steganography. In this work, we reproduced and implemented the key stages of the proposed system, which are as follows:

1. **Image Differencing:** The secret image represented as a t-bit $SE = \{se_{ij}\}$, where $1 \leq i \leq m$ and $1 \leq j \leq n$ is converted to a difference image $DIFF = \{diff_{ij}\}$ of SE according to the equation below:

$$diff_{ij} = \begin{cases} se_{ij}, & \text{if } i = 0 \text{ and } j = 0, \\ se_{ij} - se_{(i-1)j}, & \text{if } i \neq 0 \text{ and } j = 0, \\ se_{ij} - se_{i(j-1)}, & \text{otherwise.} \end{cases} \tag{7.12}$$

   This step aims to destroy the correlation between neighboring pixel values, thereby reducing redundancy and hiding the image structure.

2. **Huffman Coding for Compression:** The difference image was encoded using Huffman coding to further reduce the data size before generating shadow images. This step plays a crucial role in not only producing smaller shadow images, but also makes the image unintelligible.

3. **Shadow Image Generation (Sharing Phase):** The encoded data was processed using a polynomial-based sharing scheme on a Galois field of power-of-two, GF(2^t).

   - The method takes the threshold number of coefficients (r) and generates n shadow images.

- Any r out of n shadow images can be used to reconstruct the secret, ensuring both redundancy and fault tolerance.

4. **Reconstruction (Reveal Phase):** During reconstruction, we combined the shadow images using Lagrange interpolation to retrieve the encoded data. The Huffman decoder was then applied to reconstruct the difference image, followed by inverse differencing to perfectly obtain the original secret image.

Outcome

- The implementation successfully generated smaller shadow images, approximately 40% smaller than those produced by the classical Thien and Lin (2002) method, as reported in the paper.
- The system ensured a perfect reconstruction of the original image when at least the threshold number of shadow images were provided.
- This implementation serves as the foundation for further enhancements, such as the application of different preprocessing techniques to further improve storage and transmission efficiency.

7.4.2 Optimization Techniques

Considering the methodology of the Wang and Su paper as the baseline approach, we explored several preprocessing techniques to further reduce the size of the generated shadow images. Each of these techniques was implemented before the first step of the baseline methodology, and further evaluated for the size of shadow images, while not compromising on the security of the secret image. The rest of the section deals with each technique in detail.

Removal of Leading Zeros in Binary Representation of Pixels (RLZ technique)

The first optimization approach involved the removal of leading zeros from the binary representation of pixel values, thus constructing a bitstream that contains exactly the number of bits required to depict a pixel value.

Motivation

In an 8-bit grayscale image, each pixel is typically represented using a fixed-length 8-bit binary format. However, in many cases, the most significant bits (MSBs) are zeros, especially for darker pixels with lower intensity values.

Example: Pixel value 4 $\rightarrow$ Binary: `00000100` (five leading zeros)

These redundant zeros contribute to an increase in bitstream size, which in turn leads to larger shadow images. To address this, we developed a strategy to remove leading zeros from each pixel's binary representation before proceeding with the difference image computation.

Proposed Approach

The preprocessing consists of the following steps:

1. **Conversion to Binary (Variable Length):** Each pixel is converted to binary form, but the leading zeros are removed. *Example:* `00000100` $\rightarrow$ `100`

2. **Bitstream Generation:** The shortened binary values of all pixels are concatenated into a continuous bitstream, significantly smaller than the 8-bit fixed-length representation.

3. **Padding and Reshaping:** The bitstream is padded (if needed) so that its length becomes a multiple of 8. Then it is split into 8-bit chunks and converted back into decimal values, which are reshaped into a square matrix. This matrix becomes the input for the sharing phase.

4. **Recording Metadata:** The count of zeros removed for each pixel is recorded in a separate matrix; the data regarding padding and Huffman table are also integrated as metadata, which is crucial for an accurate reconstruction of the original pixel values during the reveal phase.

Illustration

Consider a simple 5×5 block of the original image, as shown in Table 7.1:

In the original scheme, the difference image is computed directly from the grayscale pixel values. In our optimized method, the new matrix generated after zero-stripping replaces the original image for subsequent steps of the base paper methodology.

Impact

Table 7.1: Example of leading-zero removal for representative pixel values.

Pixel Value	Binary (8-bit)	After Removing Zeros	Zeros Removed
4	00000100	100	5
7	00000111	111	5
128	10000000	10000000	0

- **Reduction in Bitstream Size:** Removing leading zeros significantly compressed the initial bitstream, resulting in a smaller secret image without any loss of information.
- **Perfect Reconstruction:** During the reveal phase, the zero removed matrix ensures an accurate reconstruction by adding the appropriate number of leading zeros for each pixel.

Digit Decomposition of Pixel Values (DDP technique)

After experimenting with leading-zero removal, we explored another preprocessing technique to further reduce the size of the shadow images before applying Wang and Su's sharing scheme. This method involves splitting each pixel's grayscale value into its decimal digits, thereby working with smaller numbers and producing a more compact bitstream.

Motivation

In an 8-bit grayscale image, each pixel has an intensity value between 0 and 255. Traditionally, these values are processed directly as 8-bit integers. However, treating all pixels uniformly as 8-bit values introduces redundant information. By decomposing each pixel into its hundreds, tens, and ones digits:

1. The image is split into three smaller matrices with much smaller ranges:
   - Hundreds matrix: values 0–2
   - Tens matrix: values 0–9
   - Ones matrix: values 0–9
2. These smaller values are encoded using variable-length binary representations, resulting in a shorter overall bitstream.

Table 7.2: Example of digit decomposition for representative pixel values.

Pixel Value	Hundreds (H)	Tens (T)	Ones (O)
247	2	4	7
093	0	9	3
158	1	5	8

Proposed Approach

The preprocessing pipeline works as follows:

1. **Splitting into Digit Matrices:** Each pixel P is decomposed as:

$$H = \lfloor P/100 \rfloor, \quad T = \lfloor (P/10) \bmod 10 \rfloor, \quad O = P \bmod 10 \tag{7.13}$$

Example: $P = 247 \Rightarrow H = 2, T = 4, O = 7$

2. **Conversion to Bitstreams:** Each digit matrix is flattened and converted to binary without padding, generating three separate bitstreams:

$$H_{\text{stream}}, \quad T_{\text{stream}}, \quad O_{\text{stream}} \tag{7.14}$$

3. **Ultimate Bitstream Formation:** These streams are concatenated to form a single ultimate bitstream:

$$U = H_{\text{stream}} \| T_{\text{stream}} \| O_{\text{stream}} \tag{7.15}$$

4. **Reshaping into Matrix:** The ultimate bitstream is split into 8-bit chunks and converted to integers (0–255). These values are reshaped into the smallest square matrix possible, which serves as the input for the sharing phase.

5. **Recording Metadata:** The data regarding the bitstreams, Huffman table and the data regarding padding are also integrated as metadata, which is crucial for an accurate reconstruction of the original pixel values during the reveal phase.

Illustration of Digit Splitting

Table 7.2 illustrates the digit decomposition for representative pixel values:

Integration with Wang and Su's Method

In the original scheme, the grayscale image directly undergoes the difference image computation. In our workflow, the new matrix created from the ultimate bitstream replaces the original image for subsequent steps of the base paper methodology.

Impact

- **Reduced Original Image Size:** Working with digit-level data significantly compresses the bitstream, producing a smaller original image.
- **No Information Loss:** Metadata ensures perfect reconstruction of the original pixel values.
- **Efficient Encoding:** Most digits are small (0–9), which are highly efficient for binary encoding.

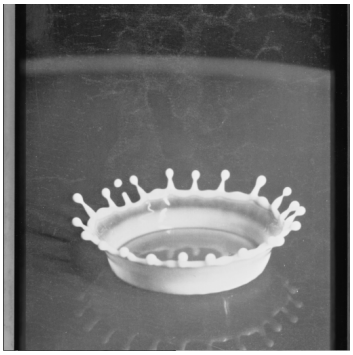
7.5 Results and Analysis

This section presents the experimental results obtained by implementing the techniques discussed previously. The polynomial-based Secret Image Sharing (SIS) scheme proposed by Wang and Su was implemented using a $(2, 4)$ threshold configuration, which means that any two out of four generated shadow images are sufficient to perfectly reconstruct the original secret image. All computations were performed with the irreducible polynomial $x^8 + x^5 + x^3 + x^2 + 1$ in the Galois Field GF(2^8), as specified in the paper. The XOR-based SIS scheme optimized by pooling and permutation was computed using $(4, 4)$ threshold configuration, whereas the (t, n) XOR-based SIS scheme was implemented in $(2, 4)$ configuration.

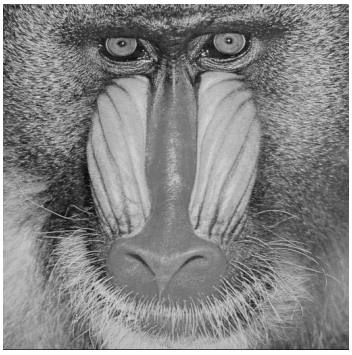
The experiments were carried out on standard benchmark images of size 512×512 pixels, that is, a size of 256.00 KB. The images used include Jet, Lena, and Milk, as described in the original paper, along with widely used test images such as Baboon, Barbara, and Bridge. Figure 7.1 depicts the test images used for various schemes in this paper. The goal was to analyze the shadow image sizes generated by the baseline polynomial-based SIS method and to compare them with the optimized preprocessing techniques explored in this research.

(a) Jet

(b) Lena



(c) Milk



(d) Baboon

(e) Barbara

(f) Bridge

Figure 7.1: Test images used for evaluation

Since the reconstruction process in all polynomial and XOR-based experiments was lossless, the Peak Signal-to-Noise Ratio (PSNR) was consistently ∞ and the Structural Similarity Index Measure (SSIM) remained 1.0 for all test images. Thus, the focus of the analysis is primarily on the storage efficiency achieved through share size reduction.

The results of autoencoders-based SIS include focus on the shares size reduction and the reconstruction quality using SSIM and PSNR measures, as it is not a perfectly lossless scheme.

7.5.1 Experimental Results of Base Paper Implementation

The experimental evaluation of the base paper, **Secret Image Sharing with Smaller Shadow Images** (SIS), was carried out to serve as a baseline to compare the results of the optimization techniques explored in this work. Table 7.3 presents the results

obtained from the implementation. It lists the Huffman table entries for each unique pixel difference value, the size of each generated shadow image, and the combined shadow size, that is, the total storage required for the four generated shadow images.

Table 7.3: Experimental results for all test images using the baseline polynomial-based SIS scheme

Image	Huffman Entries	Shadow Size (bytes)	Combined Size (bytes)
Jet	245	74653	298612
Lena	233	82574	330296
Milk	213	68819	275276
Baboon	270	105062	420248
Barbara	331	99044	396176
Bridge	156	72195	288780

Observations

- **Variation in Huffman Table Entries:** Images with higher texture complexity, such as Barbara and Baboon, required more Huffman table entries due to increased pixel variation, whereas simpler images like Bridge and Milk had fewer entries.
- **Shadow Image Size:** Shadow image size varies depending on the entropy of the input image. For instance, Baboon, known for its highly complex texture, produced the largest shadow size (102.60 KB), while Milk had the smallest (67.21 KB).
- **Efficiency of the scheme:** As expected from the baseline paper, each shadow image is less than $\frac{1}{r}$ the size of the secret image, that is, less than $\frac{1}{2}$ of the original image or 128 KB. Compared to traditional (n, n) XOR schemes also, this (t, n) scheme generates shadow images that are smaller than the original secret image, leading to significant savings in storage space.

Figure 7.2 illustrates the results of the baseline SIS scheme on the Milk image, showing the original image, the difference image that highlights the loss in correlation between pixels, and the four generated shadow images produced during the secret image sharing process.

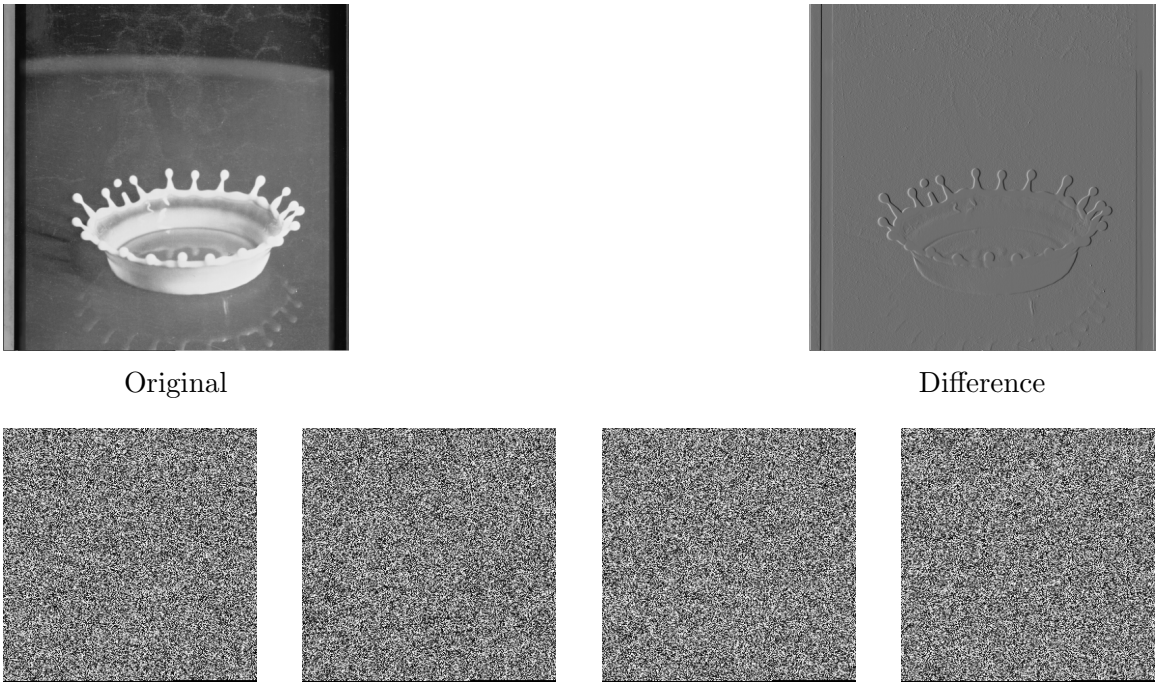


Figure 7.2: Base paper implementation pipeline

7.5.2 Performance Evaluation of Polynomial-based Optimization Techniques

This section deals with the experimental results of the various optimization techniques explored in the research, along with the improvements and drawbacks found in each method.

Removal of Leading Zeros in Pixel Binary Representation (RLZ Technique)

In this section, we evaluate the results of the first optimization technique in which the leading zeros from the binary representation of the pixel values, so that the size of the bitstream used for generating the difference image is reduced, compared to the base paper approach. Table 7.4 shows the results of this approach, listing the sizes of the processed image and shadow images.

Table 7.4: Experimental results for all test images using the polynomial-based SIS scheme improvised using RLZ technique

Image	Preprocessed Size (B)	Shadow Size (B)	Combined Size (B)
Jet	256041	105779	423117
Lena	240068	117450	469883
Milk	225570	116243	464693
Baboon	244053	128294	513434
Barbara	235077	121876	487651
Bridge	216205	113670	454682

Observations

- **Preprocessed Image Size:** The preprocessing step resulted in a compressed representation of the original image, with sizes ranging from 211.16 KB (Bridge) to 250.04 KB (Jet). It is also noted that images with higher texture complexity, such as Baboon and Jet, produced larger preprocessed sizes due to greater variability in pixel values.
- **Increase in Huffman Entries:** The modifications in the bitstream as a result of preprocessing contribute to a higher number of unique pixel values. Therefore, the number of Huffman table entries reached 509 for all images, except for the Bridge image that had 505 Huffman codings.
- **Shadow Size Trend:** Due to an increase in the Huffman table entries, each generated shadow image is larger than in the baseline scheme.

Figure 7.3 illustrates the results of the baseline SIS scheme improved by the RLZ technique on the Jet image, showing the original image, the preprocessed image, the difference image, and the four generated shadow images produced during the secret image sharing process.

The Removal of Leading Zeros from binary representation technique failed in achieving the objective of reducing the size of shadow images.

Digit Decomposition of Pixel Values (DDP Technique)

In this section, we evaluate the results of the **Digit Decomposition of Pixels** optimization technique in which each pixel value is split into individual digits, and

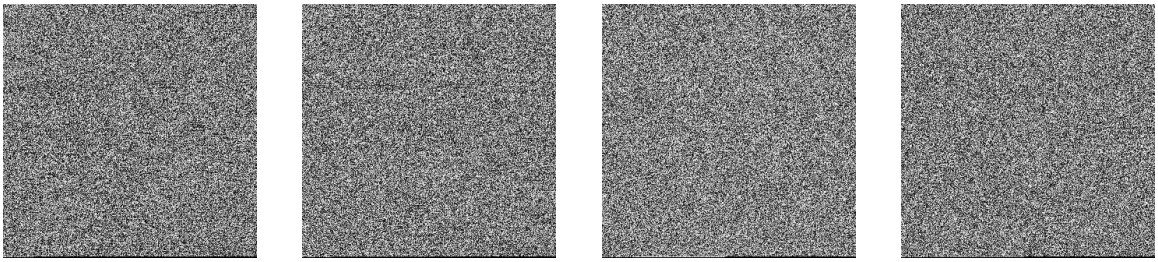


Figure 7.3: Removal of Leading Zeros (RLZ) technique pipeline

each digit is used in three separate matrices. The bitstreams generated from each matrix is concatenated to form the ultimate bitstream, that is used to generate the preprocessed image matrix on which the SIS scheme is performed. Table 7.5 shows the results of this approach, listing the sizes of the processed image and the shadow images.

Observations:

- **Preprocessed Image Size:** The preprocessing stage resulted in noticeably smaller image representations than the original image, with sizes ranging from 191.65 KB (Milk) to 212.07 KB (Bridge). An improvement was seen in this approach as the size of the preprocessed images were smaller than in the RLZ optimization technique.

Table 7.5: Experimental results for all test images using the polynomial-based SIS scheme improvised using DDP technique

Image	Preprocessed Size (B)	Shadow Size (B)	Combined Size (B)
Jet	202496	92472	370933
Lena	206122	102972	411919
Milk	196698	91379	365529
Baboon	207019	108800	435197
Barbara	205210	104806	419240
Bridge	217144	93411	372847

- **Consistent, but high Huffman Table Entries:** For all test images, the Huffman table entries reached a maximum of 511.
- **Shadow Image Size Trend:** The shadow images generated were, in fact, larger than in the baseline methodology. Although this approach generated the maximum number of Huffman entries, the shadow images were smaller than in the RLZ optimization technique, providing the insight that a higher number of Huffman entries does not mean a larger image.

Figure 7.4 illustrates the results of the baseline SIS scheme improved by the DDP technique on the Lena image showing the original image, the preprocessed image, the difference image, and the four generated shadow images produced during the secret image.

The Digit Decomposition of Pixels technique also failed to achieve the objective of reducing the size of the shadow images compared to the baseline approach.

7.6 Summary

This chapter addressed the challenge of reducing the size of shadow images generated in Secret Image Sharing (SIS) schemes without compromising the perfect recovery of the original secret. Beginning with the implementation of the classical polynomial-based SIS method, we systematically explored preprocessing methodologies such as binary representation optimization, digit-based pixel decomposition, and average pooling with residual mapping. Furthermore, the integration of the XOR-based scheme with pooling techniques provided a computationally efficient approach

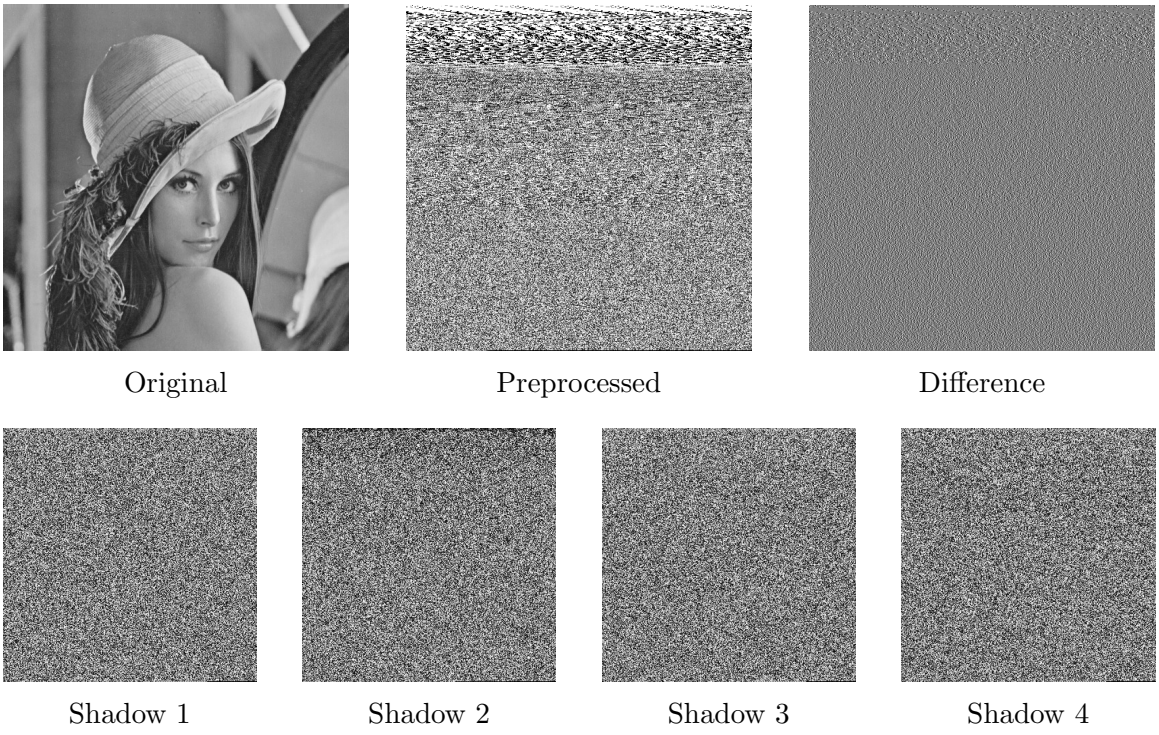


Figure 7.4: Digit Decomposition of Pixels (DDP) technique pipeline

to image sharing. Experimental evaluations using PSNR and SSIM confirmed that the proposed methods not only achieve exact reconstruction but also substantially reduce the storage requirements compared to traditional SIS schemes. The (t, n) Boolean-based MSIS scheme represents a significant advancement, enabling deterministic and lossless recovery of the original images using any t or more consecutive shares. However, the scheme does not achieve reduction in the size of the generated shares, leaving room for optimization in storage efficiency. The autoencoder-based SIS model achieved extremely high compression of shadow images along with near-perfect reconstruction of the secret. Hence, the objective of generating smaller shadow images while maintaining security and lossless recovery was successfully achieved.

CHAPTER 8

Conclusion and Future Works

8.1 Conclusion

This thesis has been composed of a wide range of research studies covering two main themes: log-based anomaly detection based on representation learning; and the development and evaluation of image encryption and secret image sharing techniques. These seven chapters have covered many vital issues concerning system security and confidential data management.

A role-aware representation learning framework for detecting anomalies using logs is presented in Chapter 1. This framework learns token roles without requiring any annotations or parsing of tokens manually to overcome the limitation of treating all tokens uniformly, which exists in previous methods like NeuralLog. The proposed framework has obtained an unprecedentedly high F1-score of 0.98–0.99 on four public datasets.

The attacks on chaos-based image encryption algorithms were discussed in Chapters 2 and 3, in which the medical image encryption algorithm of Jain et al. is entirely broken in Chapter 2 using no more than $n+1$ oracle queries with an exact reconstruction of the plaintext image with a maximal pixel difference of 0. This is regardless of the good statistical properties claimed by the algorithm in terms of entropy and NPCR/UACI values. The same line of work is continued in Chapter 3 by breaking the MMCBIE image encryption algorithm that uses the Hénon map, cyclic key mixing, and two-dimensional logistic map with at most 256 oracle queries by leveraging inadequate inter-pixel and inter-channel diffusion.

Pooling-based Dimension Reduction for Secret Image Sharing (SiS): The pooling-based dimension reduction technique is presented in Chapter 4 in the form of Pool-LiteSIS, which reduces shadow images storage by 75%. A lossless reconstruction process was confirmed using PSNR $= \infty$ and SSIM $= 1.0$, while a residual permutation scheme was applied to improve security.

The robustness of XOR visual secret sharing schemes against learning-based attacks is investigated in Chapter 5. It is demonstrated that a neural network adver-

sary could reconstruct the original image nearly perfectly (SSIM = 0.9720, PSNR = 33.87 dB) using all shares. This indicates that learning-based methods are capable of learning the entire reconstruction function. Nevertheless, in the missing share scenario, the quality of reconstructed images degrades substantially.

The image-sharing method in Chapter 6 used autoencoders to compress images into latent space and then distribute the latent vectors into different shares. The PSNR was over 30 dB and the SSIM score was close to 1.0. There was about 98% compression from the original image size. However, each share contains little information about the original image.

The optimization strategies for secret image sharing were discussed in Chapter 7 by using the binary encoding scheme, digit decomposition, averaging pooling along with residual mapping, and XOR-based methods that use pooling. The MSIS method based on (t, n) Boolean was useful to provide deterministic and perfect recovery through the usage of any t consecutive shares or more than t. The autoencoder model proved to give maximum compression ratio without degrading the performance, thus fulfilling the aim of creating small shadows.

In totality, these contributions represent an advancement in the field of both anomaly detection and image security, but always point out the difference between performance-based evaluation measures and security properties.

8.2 Future Works

This thesis introduces many research directions that could further expand the proposed methodology of anomaly detection as well as image security through enhancing adaptability and robustness properties along with developing theoretical foundations.

For the first line of research, regarding log-based anomaly detection, there is much space for improvements of the proposed role-aware learning methodology that would make the framework more adaptable and efficient. In particular, learning the optimal number of roles and using automatic adjustment for changing log format without the need for retraining could increase the applicability and flexibility of the approach. Furthermore, using multi-modal system data, such as metrics and execution traces, as well as hierarchical roles for better system state representation, and even cross-system

pretraining should be considered in the future.

The findings of cryptanalytic research call for enhanced consideration in the analysis and development of chaotic cryptosystems. Future research may explore the possibility of applying the suggested cryptanalysis techniques to a larger variety of cryptographic structures, from multidimensional chaos generators to permutations-diffusion schemes. The discovered vulnerabilities can be used to create secure encryption schemes by implementing robust diffusers, adding channel dependency, and preventing deterministic keystream construction. The development of an effective assessment technique involving statistical tests and protection against structure-based attacks will substantially increase their safety.

Pooling-based secret image sharing schemes can adopt adaptive algorithms that would balance well between compression ratio and reconstruction quality. The efficiency of data-driven reconstruction attacks implies the importance of developing schemes that would be resistant to such attacks. It might be possible by employing share obfuscation methods or training schemes against adversarial attacks. Methods based on autoencoders would benefit from the use of perceptual losses and generative learning to achieve high-quality reconstructions while retaining high compression ratios. Video data sharing would be another avenue to explore using the same methods to exploit temporal redundancies. In terms of threshold schemes, the reduction of shares' size while maintaining perfect reconstruction properties could become the area of improvement.

In all contributions, one recurring issue is the mismatch between the empirically observed metrics and theoretical guarantees. The future directions of research must concentrate on designing methods for evaluating solutions in light of these two factors. Furthermore, implementation on constrained hardware, including edge computing and IoT platforms, necessitates more efficient algorithms.

Publication Details

- The research paper titled ***Role-Aware Log Representations for Enhanced Anomaly Detection via Latent Token Role Induction*** has been **accepted for publication** in **Lecture Notes in Networks and Systems, Springer Nature** (SCOPUS-indexed proceedings of the **6th International Conference on Computer Vision and Robotics (CVR 2026)**). For more information: `https://scrs.in/conference/cvr2026`

- The research paper titled ***PoolLiteSIS: A Lightweight Secret Image Sharing Using Pooling-Based Dimensional Reduction*** has been **submitted** to the **Journal of Cyber Security Technology**. For more information: `https://www.tandfonline.com/journals/tsec20`

- The research paper titled ***Vulnerability Assessment of XOR-Based Secret Image Sharing Schemes Against Autoencoder Based Reconstruction Attacks*** has been **submitted** to the **4th International Conference on Networks and Cryptology (NETCRYPT 2026)**. For more information: `https://www.netcrypt.org.in/`

- The research paper titled ***Cryptanalysis of a Medical Image Encryption Scheme Using Multiple Chaotic Maps*** has been **submitted** to the **Journal of Cryptologia**. For more information: `https://www.tandfonline.com/journals/ucry20`

- The research paper titled ***A Cryptanalysis of Multi-Chaos-Based Image Encryption for IoT Using Chosen Plaintext Attacks*** has been **submitted** to the **Journal of Cryptologia**. For more information: `https://www.tandfonline.com/journals/ucry20`

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[2] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*. ACM, 2009, pp. 117–132.

[3] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," in *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIXATC'10. USA: USENIX Association, 2010, p. 24.

[4] A. Makanju, A. N. Zincir-Heywood, and E. E. Milios, "Clustering event logs using iterative partitioning," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2009, pp. 1255–1264.

[5] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE International Conference on Web Services (ICWS)*. IEEE, 2017, pp. 33–40.

[6] R. Vaarandi and M. Pihelgas, "LogCluster: A data clustering and pattern mining algorithm for event logs," in *2015 11th International Conference on Network and Service Management (CNSM)*. IEEE, 2015, pp. 1–7.

[7] J. Zhu, S. He, J. Liu, P. He, Q. Xia, M. R. Lyu, and D. Zhang, "Tools and benchmarks for automated log parsing," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 121–130.

[8] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1285–1298.

[9] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[10] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li *et al.*, "Robust log-based anomaly detection on unstable log data," in *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2019, pp. 807–817.

[11] F. Meng, Z. Li, Y. Liu, and L. Fang, "Log2Vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2020, pp. 1777–1794.

[12] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 448–460.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[14] M. Schuster and K. Nakajima, "Japanese and korean voice search," in *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2012, pp. 5149–5152.

[15] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," *Applied Soft Computing*, vol. 146, p. 110689, 2023.

[16] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, and Q. Tian, "LogFormer: A pre-train and tuning pipeline for log anomaly detection," *arXiv preprint arXiv:2401.04749*, 2024.

[17] Y. Wu, Z. Zhong, M. Pan, W. Cheng, and X. Yang, "Effectiveness of ChatGPT for log parsing: An empirical study," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2023.

[18] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.

[19] Z. Ma, A. Xu, P. He, Y. Li, Y. Sun, Z. Zheng, T. Chen, and Y. Liu, "LLM-Parser: An exploratory study on using large language models for log parsing," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2024.

[20] Z. Jiang, J. Liu, Z. He, Y. Chen, J. Gu, and M. R. Lyu, "LILAC: Log parsing using LLMs with adaptive parsing cache," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2024.

[21] J. Xu, R. Huang, Y. Xue, S. He, and Z. Zheng, "DivLog: Log parsing with prompt enhanced in-context learning," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2024.

[22] L. Xiao, Z. Zheng, S. He, J. Liu, Y. Chen, and M. R. Lyu, "LogBatcher: A LLM-based log parsing approach without training or labeling," in *Proceedings of the 33rd ACM International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024.

[23] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023.

[24] P. Bodik, M. Goldszmidt, A. Fox, D. B. Woodard, and H. Andersen, "Fingerprinting the datacenter: Automated classification of performance crises," in *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*. ACM, 2010, pp. 111–124.

[25] M. Chen, A. X. Zheng, J. Lloyd, M. I. Jordan, and E. Brewer, "Failure diagnosis using decision trees," in *Proceedings of the First International Conference on Autonomic Computing (ICAC)*. IEEE, 2004, pp. 36–43.

[26] J. Fridrich, "Symmetric ciphers based on two-dimensional chaotic maps," *International Journal of Bifurcation and Chaos*, vol. 8, no. 6, pp. 1259–1284, 1998.

[27] Z.-H. Guan, F. Huang, and W. Guan, "Chaos-based image encryption algorithm," *Physics Letters A*, vol. 346, no. 1–3, pp. 153–157, 2005.

[28] Y. Wu, J. P. Noonan, and S. Agaian, "Image encryption using the two-dimensional logistic chaotic map," *Journal of Electronic Imaging*, vol. 21, no. 1, p. 013014, 2012.

[29] M. F. M. Mursi, H. E.-D. H. Ahmed, F. E. A. El-Samie, and A. H. A. El-Aziem, "Image encryption based on fractional Fourier transform and Hénon chaotic map," *International Journal of Computer Applications*, vol. 91, no. 8, pp. 11–17, 2014.

[30] Z. Hua, F. Jin, B. Xu, and H. Huang, "2D logistic-sine-coupling map for image encryption," *Signal Processing*, vol. 149, pp. 148–161, 2018.

[31] K. Jain, A. Aji, and P. Krishnan, "Medical image encryption scheme using multiple chaotic maps," *Pattern Recognition Letters*, vol. 152, pp. 356–364, 2021.

[32] C. Cokal and E. Solak, "Cryptanalysis of a chaos-based image encryption algorithm," *Physics Letters A*, vol. 373, no. 15, pp. 1357–1360, 2009.

[33] E. Y. Xie, C. Li, S. Yu, and J. Lü, "On the cryptanalysis of Fridrich's chaotic image encryption scheme," *Signal Processing*, vol. 132, no. 3, pp. 150–154, 2017.

[34] H. Wen and Y. Lin, "Cryptanalysis of an image encryption scheme using multiple DNA operations," *Expert Systems with Applications*, vol. 223, p. 119831, 2023.

[35] ——, "Cryptanalysis of a chaotic image encryption scheme," *Expert Systems with Applications*, vol. 250, p. 123748, 2024.

[36] ——, "Cryptanalysis of color image encryption based on DNA synthetic image and chaos," *Expert Systems with Applications*, vol. 218, p. 119552, 2023.

[37] K. Jain, B. Titus, P. Krishnan, S. Sudevan, P. Prabu, and A. S. Alluhaidan, "A lightweight multi-chaos-based image encryption scheme for IoT networks," *IEEE Access*, vol. 12, pp. 62 118–62 148, 2024.

[38] M. Hénon, "A two-dimensional mapping with a strange attractor," *Communications in Mathematical Physics*, vol. 50, no. 1, pp. 69–77, 1976.

[39] Z. Hua, Y. Zhou, and H. Huang, "Cosine-transform-based chaotic system for image encryption," *Information Sciences*, vol. 480, pp. 403–419, 2019.

[40] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[41] K. Shafique, B. A. Khawaja, F. Sabir, S. Qazi, and M. Mustaqim, "Internet of things (IoT) for next-generation smart systems: A review of current challenges, future trends and prospects for emerging 5G-IoT scenarios," *IEEE Access*, vol. 8, pp. 23 022–23 040, 2020.

[42] A. Nauman, Y. A. Qadri, M. Amjad, Y. B. Zikria, M. K. Afzal, and S. W. Kim, "Multimedia Internet of Things: A comprehensive survey," *IEEE Access*, vol. 8, pp. 8202–8250, 2020.

[43] S. Heron, "Encryption: Advanced encryption standard (aes)," *Network Security archive*, vol. 2009, pp. 8–12, 2009.

[44] W. C. Barker, "Recommendation for the triple data encryption algorithm (TDEA) block cipher," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-67, 2004.

[45] S. Roy, U. Rawat, H. A. Sareen, and S. K. Nayak, "IECA: An efficient IoT friendly image encryption technique using programmable cellular automata," *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, no. 11, pp. 5083–5102, 2020.

[46] Z. Gu, H. Li, S. Khan, L. Deng, X. Du, M. Guizani, and Z. Tian, "IEPSBP: A cost-efficient image encryption algorithm based on parallel chaotic system for green IoT," *IEEE Transactions on Green Communications and Networking*, vol. 6, no. 1, pp. 89–106, 2022.

[47] P. Kumari and B. Mondal, "An encryption scheme based on grain stream cipher and chaos for privacy protection of image data on IoT network," *Wireless Personal Communications*, vol. 130, no. 3, pp. 2261–2280, 2023.

[48] M. Gupta, V. P. Singh, K. K. Gupta, and P. K. Shukla, "An efficient image encryption technique based on two-level security for Internet of Things," *Multimedia Tools and Applications*, vol. 82, no. 4, pp. 5091–5111, 2023.

[49] R. Lan, J. He, S. Wang, T. Gu, and X. Luo, "Integrated chaotic systems for image encryption," *Signal Processing*, vol. 147, pp. 133–145, 2018.

[50] S. Noshadian, A. Ebrahimzade, and S. J. Kazemitabar, "Optimizing chaos-based image encryption," *Multimedia Tools and Applications*, vol. 77, no. 19, pp. 25 569–25 590, 2018.

[51] W. Alexan, Y.-L. Chen, L. Y. Por, and M. Gabr, "Hyperchaotic maps and the single neuron model: A novel framework for chaos-based image encryption," *Symmetry*, vol. 15, no. 5, p. 1081, 2023.

[52] Z. Hua, Y. Chen, H. Bao, and Y. Zhou, "Two-dimensional parametric polynomial chaotic system," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 7, pp. 4402–4414, 2022.

[53] Y. Zhang, Z. Hua, H. Bao, H. Huang, and Y. Zhou, "Generation of n-dimensional hyperchaotic maps using Gershgorin-type theorem and its application," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 10, pp. 6516–6529, 2023.

[54] Z. Hua, J. Li, Y. Chen, and S. Yi, "Design and application of an S-box using complete Latin square," *Nonlinear Dynamics*, vol. 104, no. 1, pp. 807–825, 2021.

[55] H. Wen, Y. Huang, and Y. Lin, "High-quality color image compression-encryption using chaos and block permutation," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 8, p. 101660, 2023.

[56] H. Wen and Y. Lin, "Cryptanalyzing an image cipher using multiple chaos and DNA operations," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 7, p. 101612, 2023.

[57] K. Qian, W. Feng, Z. Qin, J. Zhang, X. Luo, and Z. Zhu, "A novel image encryption scheme based on memristive chaotic system and combining bidirectional bit-level cyclic shift and dynamic DNA-level diffusion," *Frontiers in Physics*, vol. 10, p. 963795, 2022.

[58] H. Li, S. Yu, W. Feng, Y. Chen, J. Zhang, Z. Qin, Z. Zhu, and M. Wozniak, "Exploiting dynamic vector-level operations and a 2D-enhanced logistic modular map for efficient chaotic image encryption," *Entropy*, vol. 25, no. 8, p. 1147, 2023.

[59] J. Choi and N. Y. Yu, "Secure image encryption based on compressed sensing and scrambling for Internet-of-Multimedia Things," *IEEE Access*, vol. 10, pp. 10 706–10 718, 2022.

[60] R. Hedayati and S. Mostafavi, "A lightweight image encryption algorithm for secure communications in multimedia Internet of Things," *Wireless Personal Communications*, vol. 123, pp. 1121–1143, 2022.

[61] R. Durga, E. Poovammal, K. Ramana, R. H. Jhaveri, S. Singh, and B.-J. Yoon, "CES blocks: A novel chaotic encryption schemes-based blockchain system for an IoT environment," *IEEE Access*, vol. 10, pp. 11 354–11 371, 2022.

[62] D. A. Trujillo-Toledo, O. R. López-Bonilla, E. E. García-Guerrero, E. Tlelo-Cuautle, D. López-Mancilla, O. Guillén-Fernández, and E. Inzunza-González, "Real-time RGB image encryption for IoT applications using enhanced sequences from chaotic maps," *Chaos, Solitons & Fractals*, vol. 153, p. 111506, 2021.

[63] K. N. Singh, O. P. Singh, N. Baranwal, and A. K. Singh, "An efficient chaos-based image encryption algorithm using real-time object detection for smart city applications," *Sustainable Energy Technologies and Assessments*, vol. 53, p. 102566, 2022.

[64] H. S. Shahhoseini, E. Saleh Kandzi, and M. Mollajafari, "Nonflat surface level pyramid: A high-connectivity multidimensional interconnection network," *Journal of Supercomputing*, vol. 67, no. 1, pp. 31–46, 2014.

[65] S. M. Mohtavipour, M. Mollajafari, and A. Naseri, "A novel packet exchanging strategy for preventing HoL-blocking in fat-trees," *Cluster Computing*, vol. 23, no. 2, pp. 461–482, 2020.

[66] D. Wang, L. Zhang, N. Ma, and X. Li, "Two secret sharing schemes based on Boolean operations," *Pattern Recognition*, vol. 40, no. 10, pp. 2776–2785, 2007.

[67] A. K. Chattopadhyay, D. Ghosh, P. Maitra, A. Nag, and H. N. Saha, "A verifiable (n, n) secret image sharing scheme using XOR operations," in *2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*. IEEE, 2018, pp. 1025–1031.

[68] C.-C. Chen and W.-J. Wu, "A secure Boolean-based multi-secret image sharing scheme," *Journal of Systems and Software*, vol. 92, pp. 107–114, 2014.

[69] T.-H. Chen and C.-S. Wu, "Efficient multi-secret image sharing based on Boolean operations," *Signal Processing*, vol. 91, no. 1, pp. 90–97, 2011.

[70] A. Nag, J. P. Singh, and A. K. Singh, "An efficient Boolean-based multi-secret image sharing scheme," *Multimedia Tools and Applications*, vol. 79, no. 23, pp. 16 219–16 243, 2020.

[71] M. Deshmukh, N. Nain, and M. Ahmed, "A novel approach for sharing multiple color images by employing Chinese Remainder Theorem," *Journal of Visual Communication and Image Representation*, vol. 49, pp. 291–302, 2017.

[72] H. Prasetyo and C.-H. Hsia, "Improved multiple secret sharing using generalized chaotic image scrambling," *Multimedia Tools and Applications*, vol. 78, no. 20, pp. 29 089–29 120, 2019.

[73] ——, "Lossless progressive secret sharing for grayscale and color images," *Multimedia Tools and Applications*, vol. 78, no. 17, pp. 24 837–24 862, 2019.

[74] C. Guo, H. Zhang, Q. Song, and M. Li, "A multi-threshold secret image sharing scheme based on the generalized Chinese Remainder Theorem," *Multimedia Tools and Applications*, vol. 75, no. 18, pp. 11 577–11 594, 2016.

[75] S. Kabirirad and Z. Eslami, "A (t, n)-multi secret image sharing scheme based on Boolean operations," *Journal of Visual Communication and Image Representation*, vol. 57, pp. 39–47, 2018.

[76] K. M. Faraoun, "Highly secure (n, n) multi-secret image sharing scheme based on cryptographic hash function and stream cipher," *Information Sciences*, vol. 512, pp. 1054–1070, 2020.

[77] J.-Y. Lin, J.-H. Horng, and C.-C. Chang, "Secret image sharing with distinct covers based on improved Cycling-XOR," *Journal of Visual Communication and Image Representation*, vol. 104, p. 104282, 2024.

[78] D. Ou, L. Ye, and W. Sun, "User-friendly secret image sharing scheme with verification ability based on block truncation coding and error diffusion," *Journal of Visual Communication and Image Representation*, vol. 29, pp. 15–27, 2015.

[79] B. R. Purushothama, " ESIS-SSI: Efficient (n, n) Secret Image Sharing with Shrinking Shadow Images ," in *2024 OITS International Conference on Information Technology (OCIT)*. Los Alamitos, CA, USA: IEEE Computer Society, Dec. 2024, pp. 609–616.

[80] A. K. Chattopadhyay, A. Nag, and J. P. Singh, "An efficient verifiable (t, n)-threshold secret image sharing scheme with ultralight shares," *Multimedia Tools and Applications*, vol. 81, no. 24, pp. 34 969–34 999, 2022.

[81] S. Kabirirad and Z. Eslami, "A verifiable threshold secret image sharing scheme based on XOR and GF(2^8)," *Journal of Information Security and Applications*, vol. 43, pp. 132–143, 2019.

[82] A. V. Soreng and S. Kandar, "A verifiable threshold secret image sharing (SIS) scheme with combiner verification and cheater identification," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 8, pp. 10 631–10 655, 2023.

[83] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[84] M. Naor and A. Shamir, "Visual cryptography," in *Advances in Cryptology – EUROCRYPT 1994*, ser. Lecture Notes in Computer Science, vol. 950. Springer, 1995, pp. 1–12.

[85] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[86] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[87] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 448–456.

[88] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, ser. Lecture Notes in Computer Science, vol. 9351. Springer, 2015, pp. 234–241.

[89] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Computer Vision – ECCV 2016*, ser. Lecture Notes in Computer Science, vol. 9906. Springer, 2016, pp. 694–711.

[90] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[91] D. Wang, L. Zhang, N. Ma, and X. Li, "Two secret sharing schemes based on boolean operations," *Pattern Recognition*, vol. 40, no. 10, pp. 2776–2785, 2007.

[92] A. K. Chattopadhyay, A. Nag, J. P. Singh, and A. K. Singh, "A verifiable multi-secret image sharing scheme using xor operation and hash function," *Multimedia Tools Appl.*, vol. 80, no. 28–29, p. 35051–35080, Nov. 2021.

[93] T.-H. Chen and C.-S. Wu, "Efficient multi-secret image sharing based on boolean operations," *Signal Processing*, vol. 91, no. 1, pp. 90–97, 2011.

[94] C.-C. Chen and W.-J. Wu, "A secure boolean-based multi-secret image sharing scheme," *Journal of Systems and Software*, vol. 92, pp. 107–114, 2014.

[95] A. Nag, J. Singh, and A. Singh, "An efficient boolean based multi-secret image sharing scheme," *Multimedia Tools and Applications*, vol. 79, pp. 1–25, 06 2020.

[96] L. Pasi and B. R. Purushothama, *Secret Image Sharing with Reduced Shadow Image Size Without Combiner's Secret*, 01 2026, pp. 375–386.

[97] G. R. Blakley, "Safeguarding cryptographic keys," in *Proceedings of the 1979 AFIPS National Computer Conference*, vol. 48. AFIPS Press, 1979, pp. 313–317.

[98] C.-C. Thien and J.-C. Lin, "Secret image sharing," *Computers & Graphics*, vol. 26, no. 5, pp. 765–770, 2002.

[99] R.-Z. Wang and C.-H. Su, "Secret image sharing with smaller shadow images," *Pattern Recognition Letters*, vol. 27, no. 6, pp. 551–555, 2006.

[100] S. Saha, A. K. Chattopadhyay, A. K. Barman, A. Nag, and S. Nandi, "Secret image sharing schemes: A comprehensive survey," *IEEE Access*, vol. 11, pp. 98 333–98 361, 2023.

[101] S. J. Shyu and Y.-R. Chen, "Threshold secret image sharing by Chinese Remainder Theorem," in *2008 IEEE Asia-Pacific Services Computing Conference (APSCC)*. IEEE, 2008, pp. 1332–1337.

[102] X. Yan, S. Wang, and X. Niu, "Threshold progressive visual cryptography construction with unexpanded shares," *Multimedia Tools and Applications*, vol. 75, no. 14, pp. 8657–8674, 2016.